



河南大學
HENAN UNIVERSITY

第三章 存储系统

3.1 存储系统概述

➤ 3.1.1 存储器的分级结构

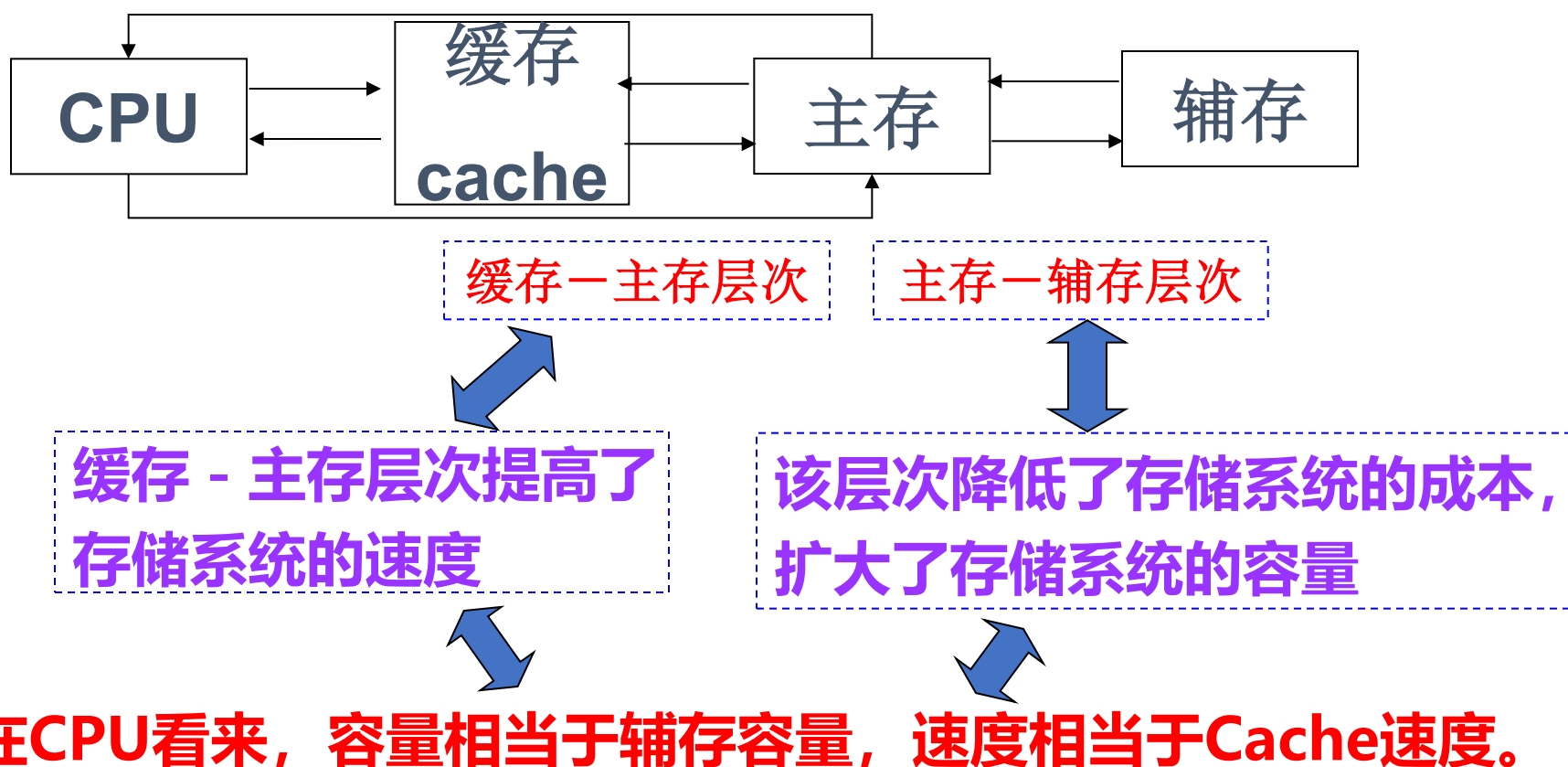
➤ 3.1.2 存储器分类

➤ 3.1.3 存储器的编址和端模式

➤ 3.1.4 存储器的技术指标

>>> 3.1.1 存储系统的层次结构

➤ 三级存储系统结构（主板上的存储系统结构）



>>> 3.1.3 存储器的编址和端模式

- **字存储单元**：存放一个机器字的存储单元称为字存储单元，相应的单元地址叫字地址。
- **字节存储单元**：存放一个字节的单元称为字节存储单元，相应的地址称为字节地址。
- **存储器地址编址方式**：存储器地址的组织方式，一般在设计处理器时就已经确定了



3.1.3 存储器的编址和端模式

➤ 编址方式：根据计算机中的最小单位编址来确定

□ 编址的最小单位是字存储单元-----按字编址

□ 编址的最小单位是字节存储单元-----按字节编址

➤ 考虑：如果一个机器字包含多个字节，如何进行编址

□ 一个16位二进制的字存储单元包含两个字节

✓ 可以按字地址寻址，也可以按字节地址寻址

✓ 采用字节编址：占两个字节地址

每个存储单元有一个地址：

按字节编址

按字编址

按半字编址

按1/4字编址

>>> 3.1.3 存储器的编址和端模式

➤ 端模式：多字节数据的字节顺序，如何对该字进行存储的排列方式

□ 大端 big-endian

✓ 高低低高（高字节数据放内存的低地址端，低字节数据放在内存的高地址端）

□ 小端 little-endian

✓ 高高低低（低字节数据存放在低地址端，高字节数据存放在内存高地址端）

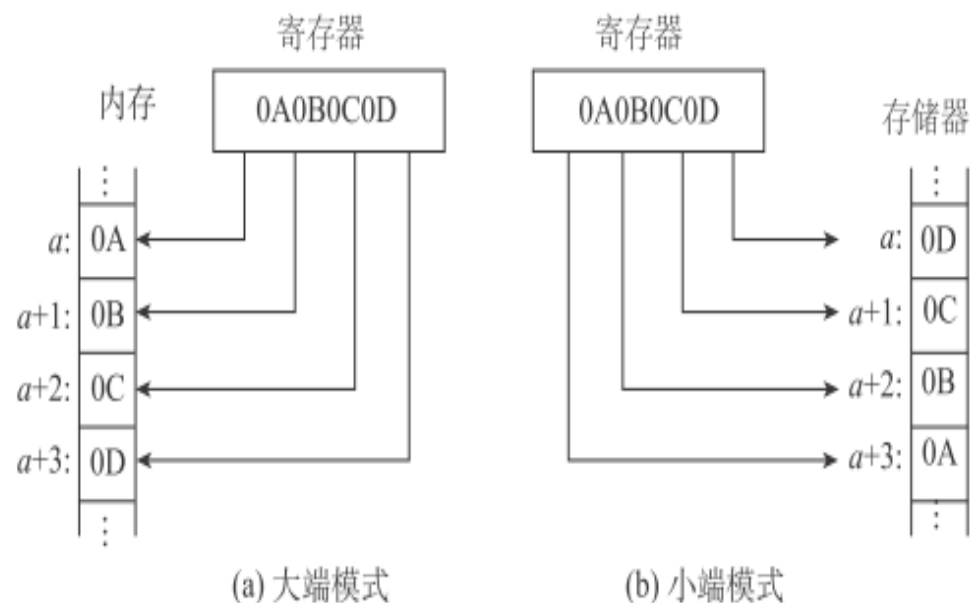
➤ 例：数字 0x0A0B0C0D 在内存中

□ 大端：

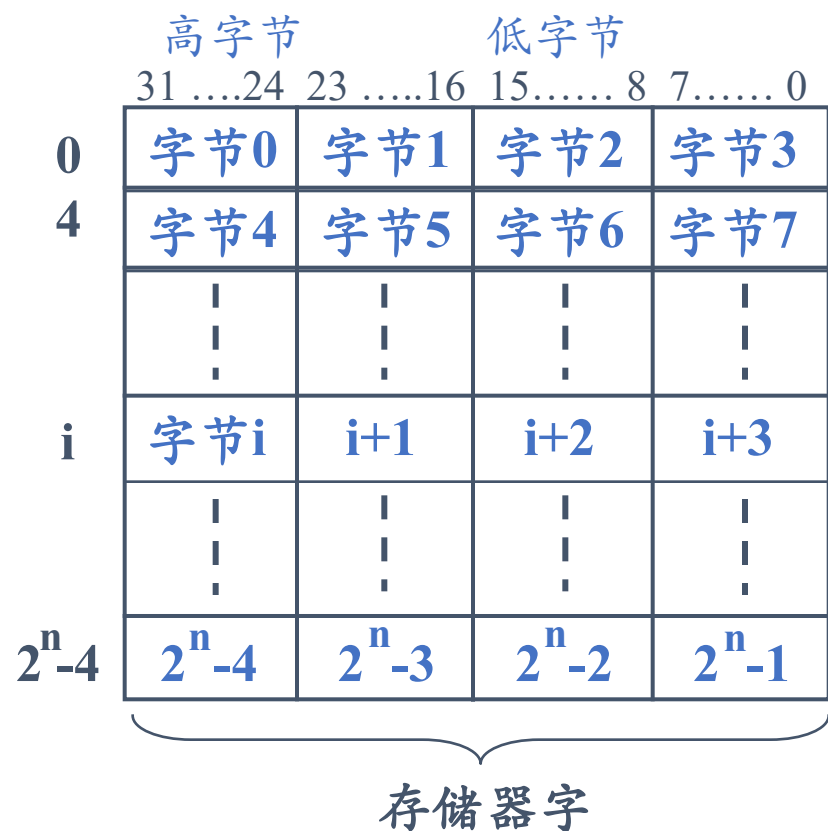
✓ 低地址 -----> 高地址
0x0A | 0x0B | 0x0C | 0x0D

□ 小端：

✓ 低地址 -----> 高地址
0x0D | 0x0C | 0x0B | 0x0A



>>> 字节地址—大端模式



字长=4字节

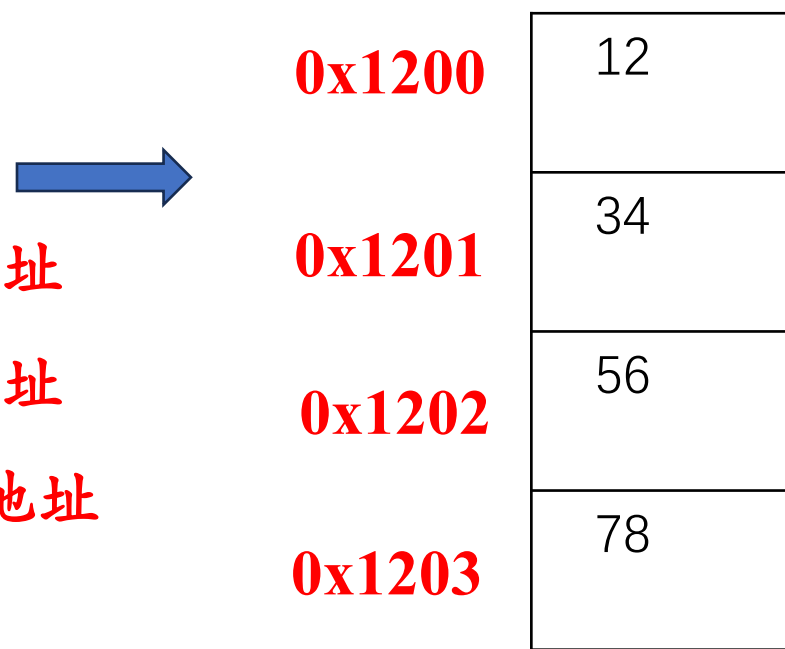
低字节——高地址

高字节——低地址

字地址=高字节地址

大端
Big Endian

例如: `int a=0x12345678h`

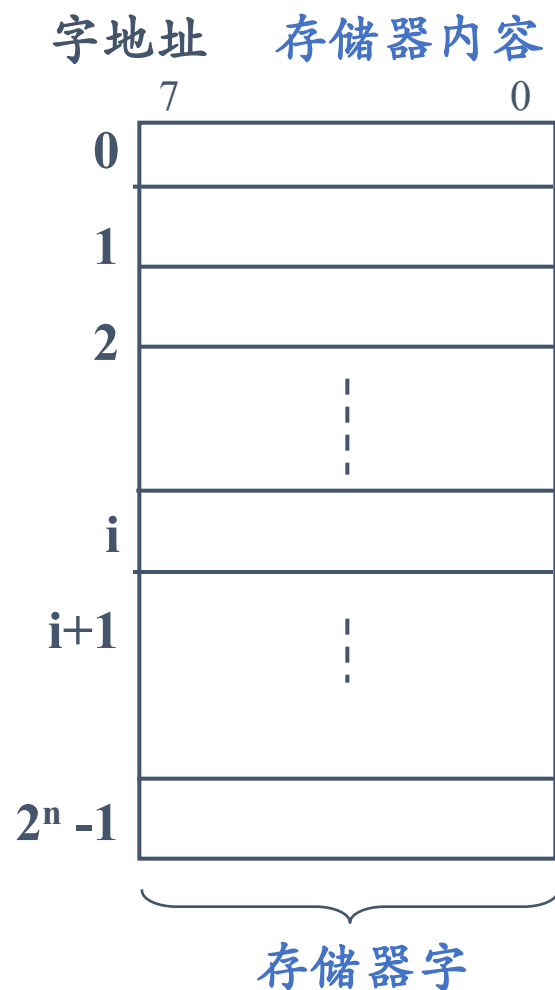


0x1200 0x1201 0x1202 0x1203

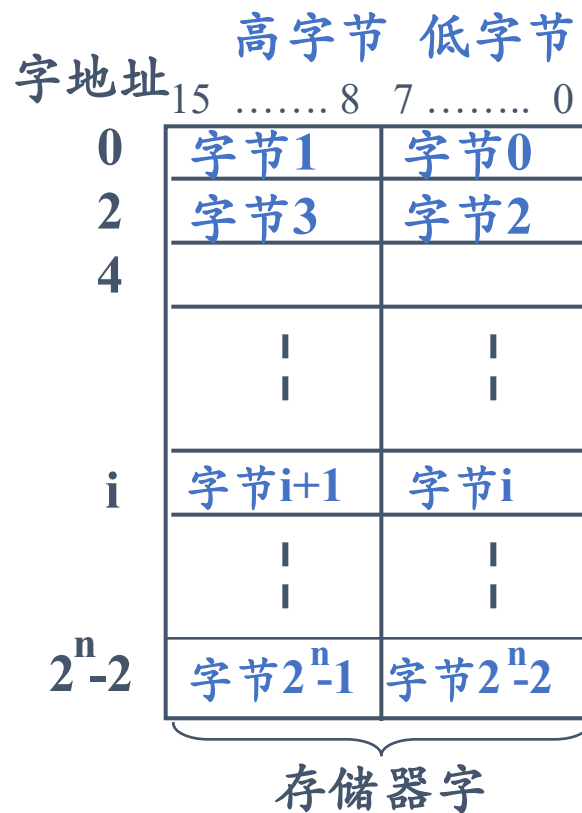
12	34	56	78
----	----	----	----

大端模式:方便人阅读

>>> 字节地址—小端模式

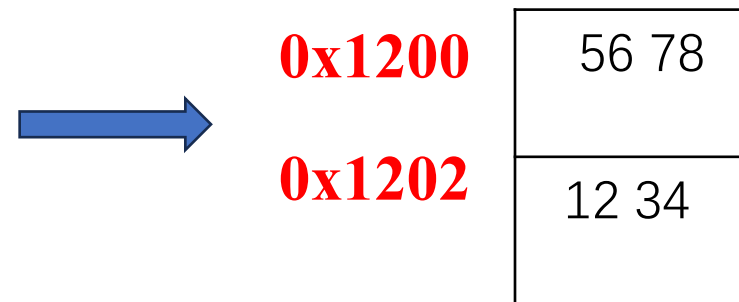


小端
Little Endian



低字节——低地址
高字节——高地址
字地址=低字节地址
字长=2字节

例如: `int a=0x12345678h`



0x1200 0x1201 0x1202 0x1203

78	56	34	12
----	----	----	----

小端模式:机器处理方便

>>> 3.1.4 主存储器的技术指标——存储容量 (1)

➤ **存储容量：指存储器能存放的信息比特数。**

□ **存储容量=存储单元个数×存储字长**

✓ **用 $a \times b$ 表示**

□ **存储容量=存储单元个数×存储字长/8**

✓ **单位为B（字节）**

$1\text{KB} = 2^{10}\text{B} = 1024\text{个字节}$ ； $1\text{MB} = 2^{20}\text{B} = 1024\text{K个字节}$

$1\text{GB} = 2^{30}\text{B} = 1024\text{M个字节}$ ； $1\text{TB} = 2^{40}\text{B} = 1024\text{G个字节}$

➤ **要求：**

已知存储容量，能计算出该存储器的地址线和数据线的根数。

➤ **例如**

□ **某机存储容量为 $2\text{K} \times 16$ ，则该系统所需的地址线为11根，数据线位数为16根。**

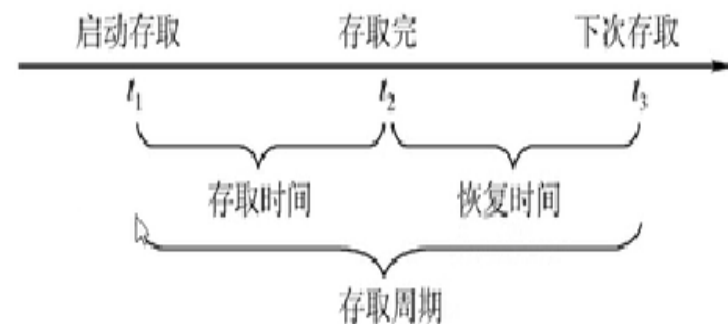
>>> 3.1.4 主存储器的技术指标——存储速度 (2)

➤ 存取时间(访问时间 T_a)

- 从存储器接收到读/写命令到信息被读出或写入完成所需的时间;
- 或: 从启动一次存储器操作到完成该操作为止所经历的时间;
- 决定于存储介质的物理特性和寻址部件的结构
- 以ns为单位, 存取时间又分读出时间、写入时间两种。

➤ 存取周期(T_m)

- 在存储器连续读写过程中一次完整的存取操作所需的时间
- CPU连续两次访问存储器的最小时间间隔
- 它是指存储器进行一次完整的读写操作所需的全部时间, 又称读写周期或者访问周期
- 以ns为单位, 存取周期=存取时间+复原时间。



>>> 3.1.4 主存储器的技术指标——存储速度 (3)

➤ 存储器带宽 (通常可以用数据传输率来代替)

- 每秒从存储器进出信息的最大数量;
- 单位为位/秒或者字节/秒。

➤ 数据传送速率 (频宽) B_m :

- 单位时间内能够传送的信息量
- 若系统的总线宽度为 W bit, 则 $B_m = W/T_m$ (b/s)

>>> 求存储器带宽的例子

➤ 设某存储系统的存取周期为500ns，每个存取周期可访问16位，则该存储器的带宽是多少？

□ 存储带宽 = 每周期的信息量 / 周期时长

$$= 16 \text{位} / (500 \times 10^{-9}) \text{秒}$$

$$= 3.2 \times 10^7 \text{位/秒}$$

$$= 32 \times 10^6 \text{位/秒} = 32 \text{M位/秒}$$

>>> 3.3.2 DRAM芯片的逻辑结构

➤ 外部地址引脚比SRAM减少一半;

- 存储芯片集成度高, 体积小;
- 将地址分为行列, 行列地址位数相同, 行列均分地址 (行地址: 高位地址)
- 送地址信息时, 分行地址和列地址分时传送, 目的是减少地址引脚的个数, 达到地址线复用的目的;

➤ 内部结构: 比SRAM复杂

□ 刷新电路

- ✓ 用于存储元上的信息刷新, 以行为单位;
- ✓ 刷新计数器的位数与行译码器的输出位数相同;

□ 行、列地址锁存器

- ✓ 用于保存完整的地址信息;
- ✓ 使用行选通信号 $\overline{RAS}$ 和列选通信号 $\overline{CAS}$ 锁存地址;

>>> 3.3.3 刷新周期

➤ 刷新

- DRAM的基本存储元——电容，会随着时间和温度而减少；
- 必须定期地对所有存储元（即使未读写的）刷新，以保持原来的信息。
- 在固定时间内对所有存储单元，通过“**读出**(不输出)—**写入**”的方式恢复信息的操作过程；

➤ 刷新方式

- 以存储矩阵的行为单位刷新，优先于访存，但访存周期内不能打断读写而刷新；
- 每块DRAM芯片内的存储矩阵按行刷新，每次刷新一行
- 一个存储器各个芯片同时刷新。

刷新过程中存储器不能进行正常的读写访问，刷新优先于读写

➤ 刷新周期

- 从上一次对整个M刷新结束到下一次对整个M全部刷新一遍为止的时间，一般为2ms。

>>> 3.3.3 刷新周期

➤ 刷新周期

□ 例如某DRAM得刷新周期为2ms，其存储矩阵的行数为256，则在2ms内至少进行256次刷新操作。

➤ 刷新周期需要考虑的问题

□ 多久刷新一次？ 2ms

□ 每次刷新多少单元？ 以行为单位，每次刷新所有芯片的同一行

□ 如何刷新？ 通常在读出一行信息后重新写入，占用1个读写周期

>>> DRAM的刷新方式

➤ 集中式刷新

- 在一个刷新周期内，利用一段**固定时间**，依次对存储矩阵的所有行逐一刷新，在此期间停止对存储器的读/写操作；
- 存在**死区时间**，会影响CPU的访存操作；

➤ 分散式刷新

- 有教材也称异步式刷新；
- 在一个刷新周期内，分散地刷新存储器的所有行；
- 既不会产生明显的读写停顿，也不会延长系统的存取周期；

➤➤➤【例】设某存储器的存储矩阵为 128×128 ，存取周期为 $0.5\mu\text{s}$ ，RAM刷新周期为 2ms ，若采用**集中式刷新方式**，试分析其刷新过程。

1、**集中式刷新**： 2ms 内集中安排时间全部刷新一次，一般是刷新周期的最后一段时间。

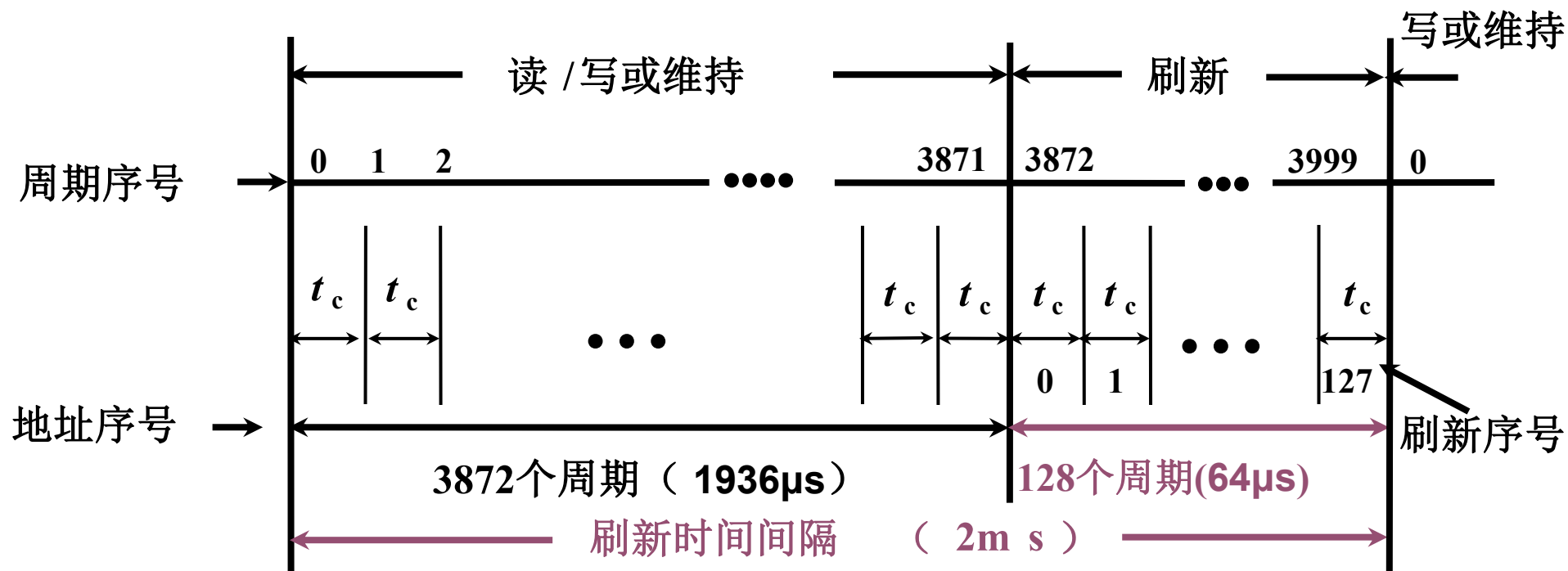
刷新一行的时间 = 存取周期 = $0.5\mu\text{s}$

2、系统的存取周期 $0.5\mu\text{s}$ ，其中有段时间专门用于刷新而无法访问存储器，称为访存“死区”。

3、**刷新周期包含的存取周期数**： $2\text{ms} \div 0.5\mu\text{s} = 4000$ 个周期。

4、**128行需要刷新时间**： $128 \times 0.5 = 64\mu\text{s}$

➤➤➤ 【例】设某存储器的存储矩阵为 128×128 ，存取周期为 $0.5\mu\text{s}$ ，RAM刷新周期为 2ms ，若采用**集中式刷新方式**，试分析其刷新过程。



“死区” 时间为 $0.5\mu\text{s} \times 128 = 64\mu\text{s}$

“死时间率” 为 $128/4000 \times 100\% = 3.2\%$

优点：主存利用率高，控制简单

缺点：刷新过程中，不能使用存储器，因而形成一段死区

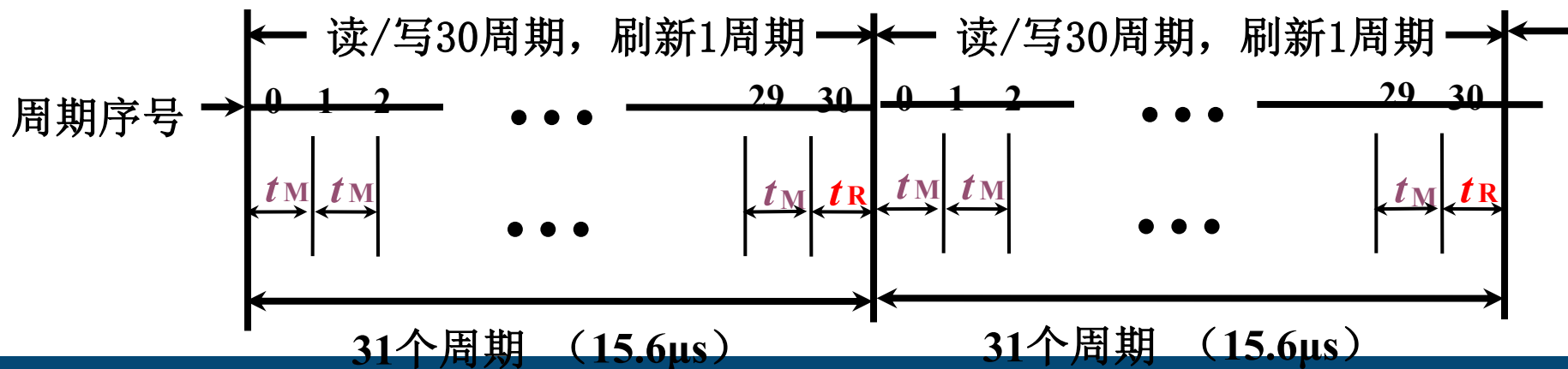
➤➤➤ 【例】 设某存储器的存储矩阵为 128×128 ，存取周期为 $0.5 \mu\text{s}$ ，RAM刷新周期为 2ms ，若采用**分散式刷新方式**，试分析其刷新过程。

- 每一个正常读写周期后安排一次刷新周期
- 两次刷新之间的间隔相同，都是间隔一个 t_m （一个存取周期），因此刷新周期和存取周期对半，各2000个存取周期（ $2\text{ms} \div 0.5 \mu\text{s} = 4000$ 个周期）
- 存取周期由原来的 $0.5 \mu\text{s}$ 变为 $1.0 \mu\text{s}$ ，实际存取的效率降低了
- 时序控制虽然简单且没有较长的死区，但主存利用率低，速度降低且刷新过于频繁，适合用在低速系统之中。



➤➤➤【例】设某存储器的存储矩阵为 128×128 ，存取周期为 $0.5\mu\text{s}$ ，RAM刷新周期为 2ms ，若采用**异步式刷新方式**，试分析其刷新过程。

- 结合前两种， 2ms 内刷新128行
- 若每隔 $2\text{ms}/128=15.6\mu\text{s}$ 刷新一行（一次），每一行的刷新操作被均匀地分配到刷新周期内，每行每隔 2ms 刷新一次，也就是刷新128行后特定的某行再被刷新一次（定时刷新）
- 每隔 $15.6\mu\text{s}$ 产生一个刷新请求信号，其中只有 $0.5\mu\text{s}$ 用于刷新时间（“死区时间”）
 - 每 $15.6/0.5 = 31.2$ （ ≈ 31 ）个工作周期中做刷新一行存储器的操作。



>>> 关于异步式刷新

➤ 刷新周期

- 动态存储元能够维持信息的最大时间长度;

➤ 刷新信号周期

- 存储矩阵相邻两行的刷新时间间隔;

- 计算: $\lfloor \text{刷新周期} / \text{存储矩阵行数} \rfloor$

- ✓ 向下取整, 保证刷新周期内所有行都能刷新完毕;

➤ 实际刷新时间 (实际刷新周期)

- 存储矩阵完成一遍刷新的实际时间;

- 计算: $\text{刷新信号周期} * \text{存储矩阵行数}$

- ✓ 其值一般略小于刷新周期;



动态 RAM 和静态 RAM 的比较

	DRAM	SRAM
存储原理	电容	触发器
集成度	高	低
芯片地址引脚	少	多
功耗	小	大
价格	低	高
速度	慢	快
刷新	有	无
运送行列地址	分时，先行后列	同时
破坏性读出	是	否

2014年考研统考第15题

15、某容量为256MB的存储器由若干4M×8位的DRAM芯片构成，该DRAM芯片的地址引脚和数据引脚总数是（ ）

- ☒ A 19 ☐ B 22
☐ C 30 ☐ D 36

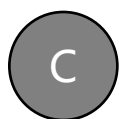
提交

下列有关RAM和ROM的叙述中，正确的是（ ）

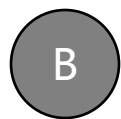
- I、 RAM是易失性存储器，ROM是非易失性存储器
- II、 RAM和ROM都是采用随机存取的方式进行信息访问
- III、 RAM和ROM都可用作Cache
- IV、 RAM和ROM都需要进行刷新



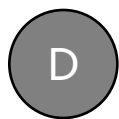
A 仅I和II



C 仅I,II, III



B 仅II和III



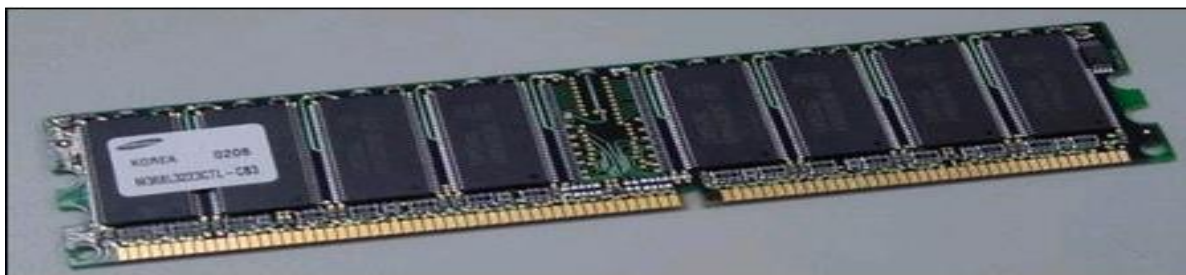
D 仅II, III, IV

提交

>>> 3.3.4 存储器容量的扩充

SRAM、DRAM、ROM
均可进行容量扩展

- 单个存储芯片的容量有限，实际存储器由多个芯片扩展而成；



- 存储器（存储芯片）与CPU的连接

- 数据、地址、控制三总线连接；

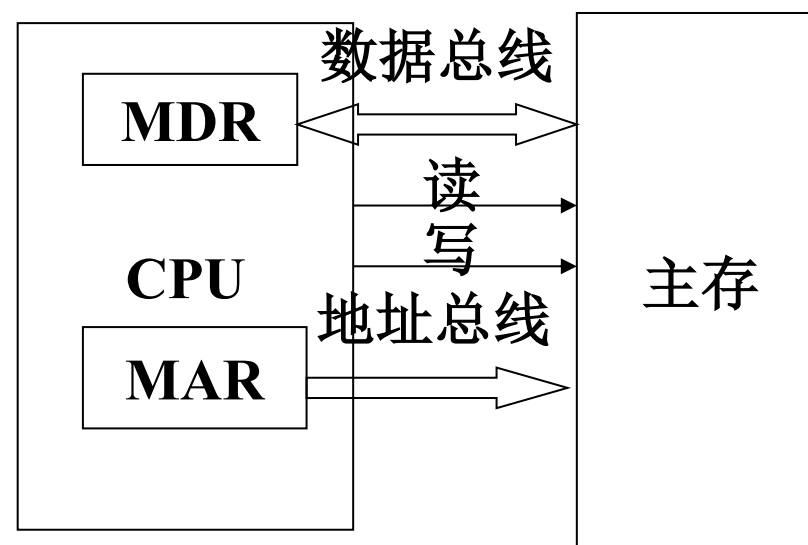
- 多个存储芯片 ↔ CPU

- ✓ 不是一一对应连接

- 关注存储芯片与CPU的外部引脚

- 存储器容量扩充方式

- 位扩展、字扩展、字位扩展



>>> 3.2.4 存储器容量扩充

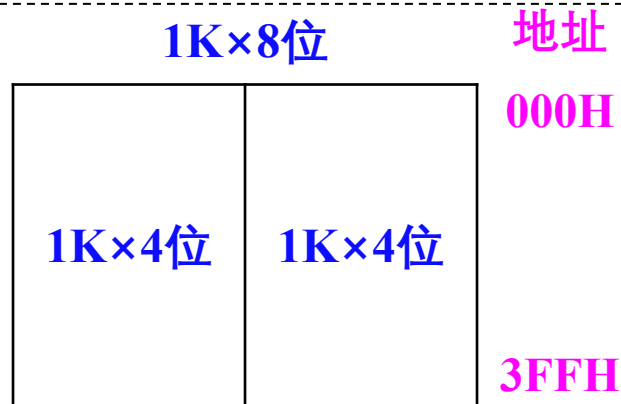
➤ 实际存储器由多个芯片扩展而成;

□ SRAM、DRAM、ROM均可进行容量扩展;

➤ 存储器容量扩充方式:

位 扩 展

存储单元数目不变, 存储字长增加



所有位扩展芯片
相同地址单元的同时访问

字 扩 展

存储单元数目增加, 存储字长不变



对某一字扩展芯片的
一个单元访问

字位扩展

存储单元数目和存储字长均增加



>>> 存储芯片与CPU的引脚

➤ 存储芯片的外部引脚

- **数据总线**：位数与存储单元字长相同，用于传送数据信息；
- **地址总线**：位数与存储单元个数为 2^n 关系，用于选择存储单元；
- **读写信号/WE**：决定当前对芯片的访问类型；
- **片选信号/CS**：决定当前芯片是否正在被访问；

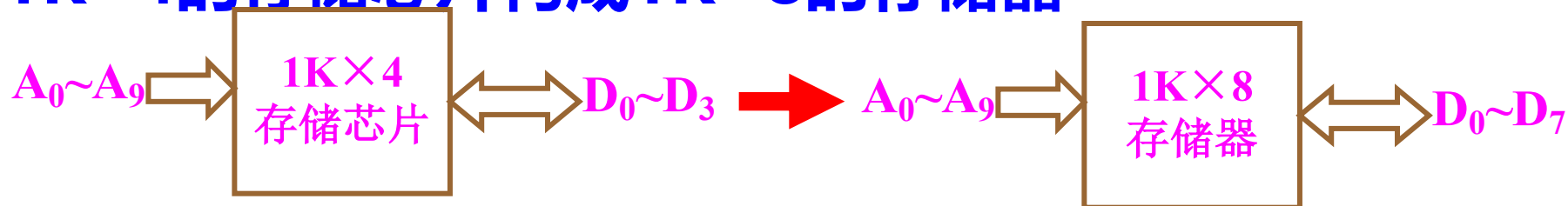
➤ CPU与存储器连接的外部引脚

- **数据总线**：位数与机器字长相同，用于传送数据信息；
- **地址总线**：位数与系统中可访问单元个数为 2^n 的关系；
- **读写信号/WE**：决定当前CPU的访问类型；
- **访存允许信号/MREQ**：决定是否允许CPU访问存储器；

>>> 存储器容量的位扩展

➤ 存储单元数不变，每个单元的位数（字长）增加；

➤ 例如：由 $1K \times 4$ 的存储芯片构成 $1K \times 8$ 的存储器



➤ 存储芯片与CPU的引脚连接方法：

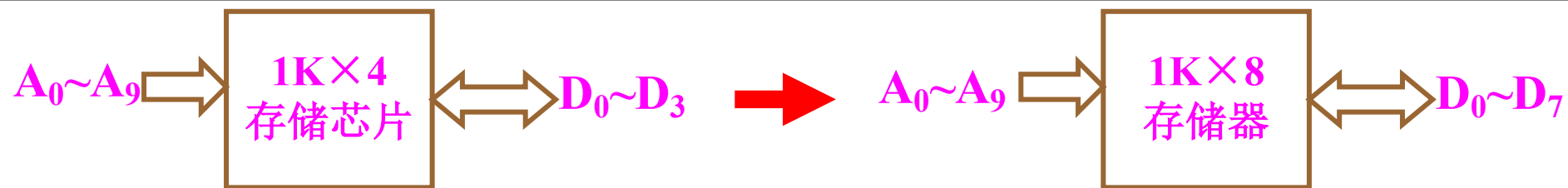
□ 地址线：各芯片的地址线直接与CPU地址线连接；

□ 数据线：各芯片的数据线分别与CPU数据线的不同位连接；

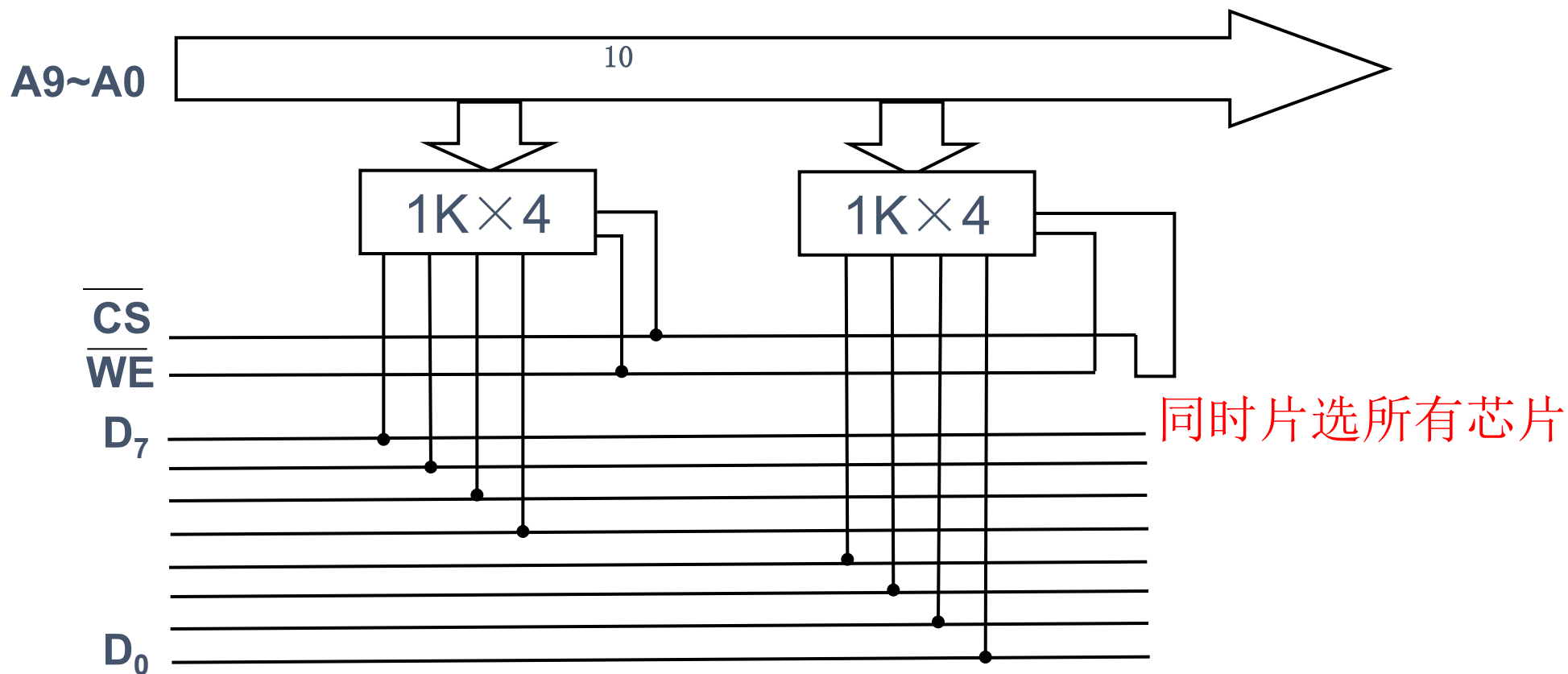
□ 片选及读写线：各芯片的片选及读写信号直接与CPU的访存及读写信号连接；同时片选所有芯片

➤ CPU对该存储器的访问是对各位扩展芯片相同地址单元的同时访问。

>>> 位扩展：由 $1K \times 4$ 的存储芯片构成 $1K \times 8$ 的存储器



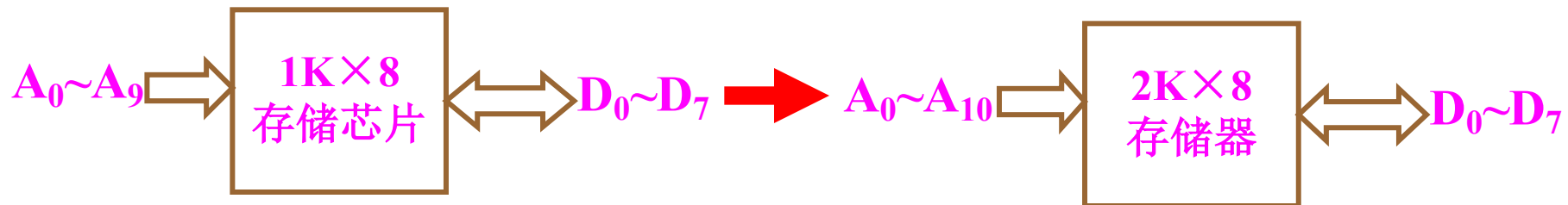
➤ 地址线、
读写线、
片选线
并联即可



>>> 存储器容量的字扩展

➤ 字扩展：每个单元位数不变，总的单元个数增加。

➤ 例如：用 $1\text{K} \times 8$ 的存储芯片构成 $2\text{K} \times 8$ 的存储器



➤ 存储芯片与CPU的引脚连接方法（考虑CPU的引脚个数多少）：

□ 地址线：各芯片的地址线与CPU的低位地址线直接连接；

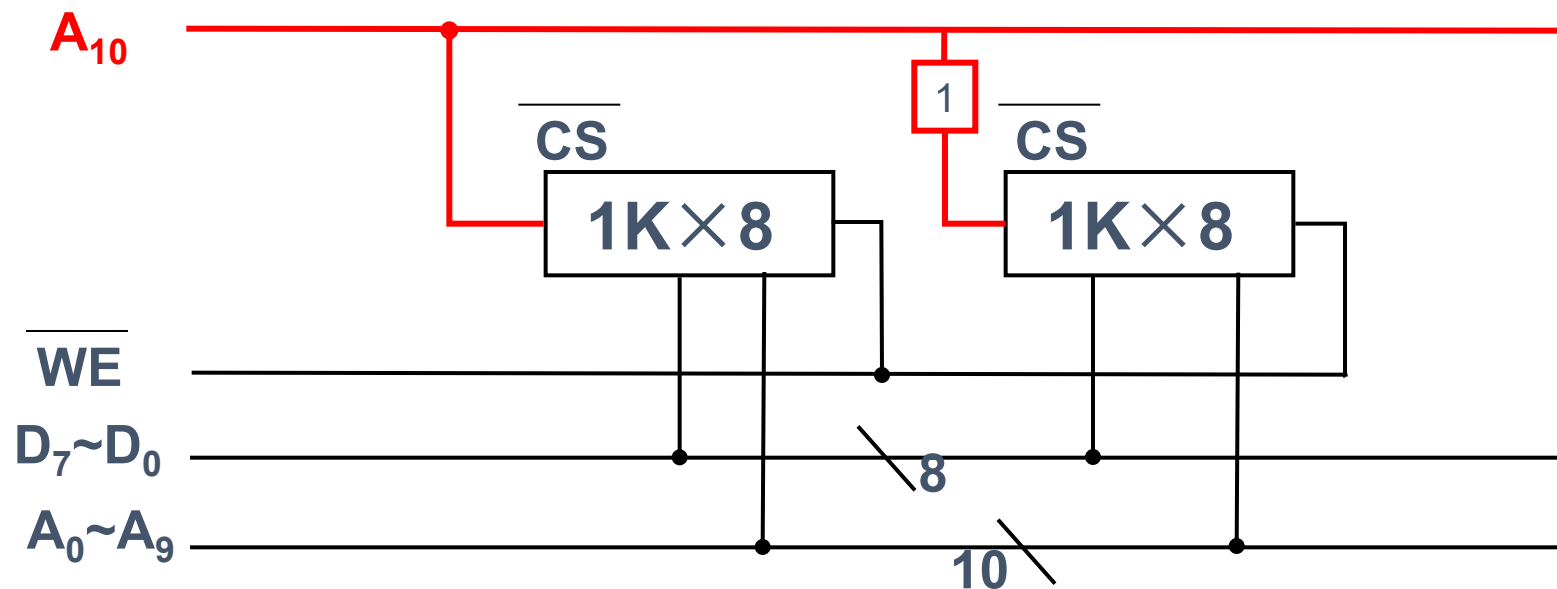
□ 数据线：各芯片的数据线直接与CPU数据线连接；

□ 读写线：各芯片的读写信号直接与CPU的读写信号连接；

□ 片选信号：各芯片的片选信号由CPU的高位地址和访存信号产生；

➤ CPU对该存储器的访问是对某一字扩展芯片的一个单元访问。

>>> 字扩展：由 $1\text{K} \times 8$ 的存储芯片构成 $2\text{K} \times 8$ 的存储器



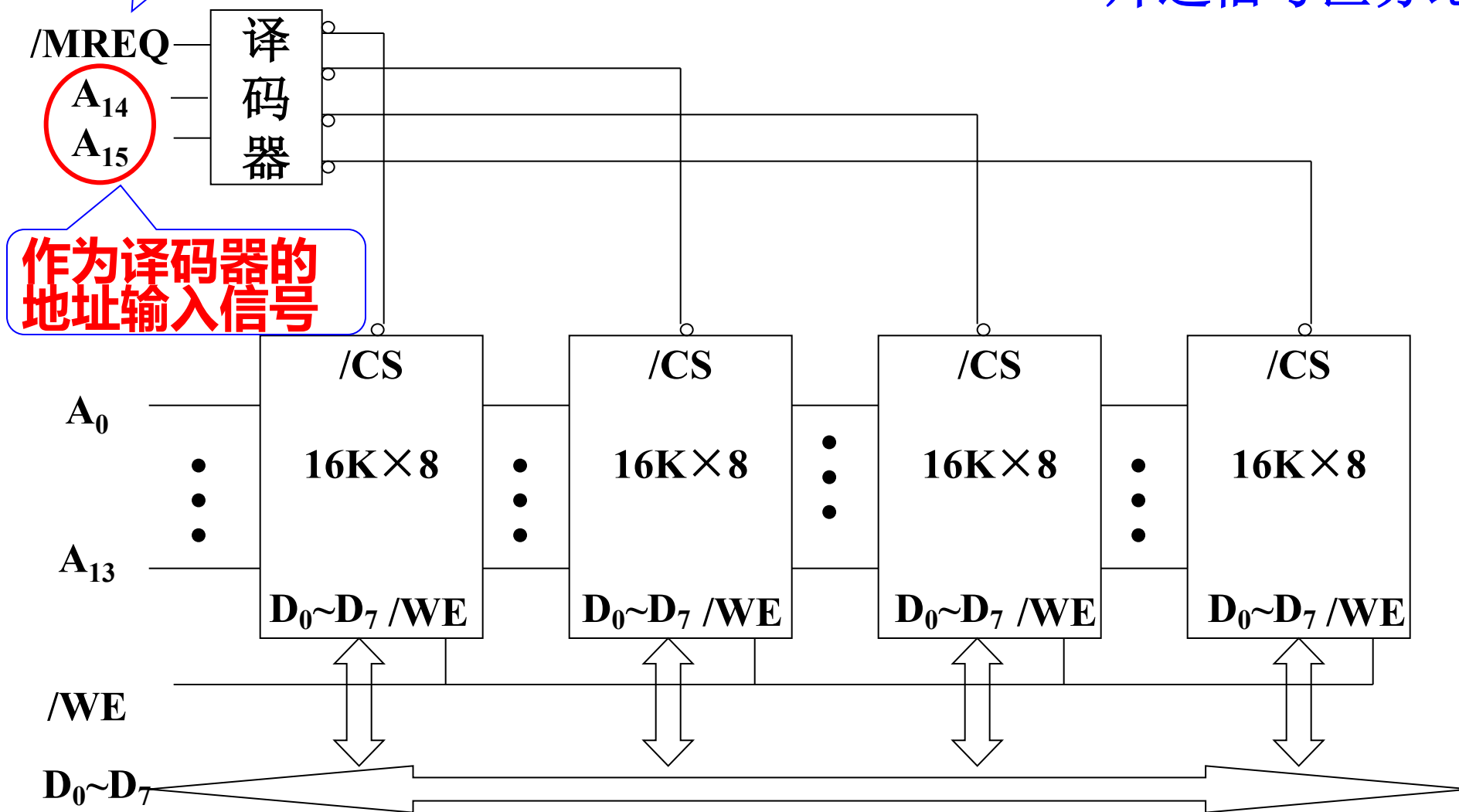
- 低位的地址线与各芯片的地址线并联，直接连接；
- 多余的高位地址线用来产生相应的片选信号。
- 用同一根高位地址线作为片选信号（设计简化）



作为译码器的
使能信号

存储芯片的字扩展连接图

- 片选信号区分地址空间



>>> 用16K×8的芯片构成64K×8的存储器

- 16K×8的存储芯片：地址线14根，数据线8根，/CS，/WE
- CPU的引脚：地址线16根，数据线8根，/MREQ（访存控制信号），/WE
- CPU的最高2位地址和/MREQ信号产生4个芯片的片选信号；
- 4个存储芯片构成存储器的地址分配：

第1片	00	00 0000 0000 0000	
	00	11 1111 1111 1111	即 0000H ~ 3FFFH
第2片	01	00 0000 0000 0000	
	01	11 1111 1111 1111	即 4000H ~ 7FFFH
第3片	10	00 0000 0000 0000	
	10	11 1111 1111 1111	即 8000H ~ BFFFH
第4片	11	00 0000 0000 0000	
	11	11 1111 1111 1111	即 C000H ~ FFFFH

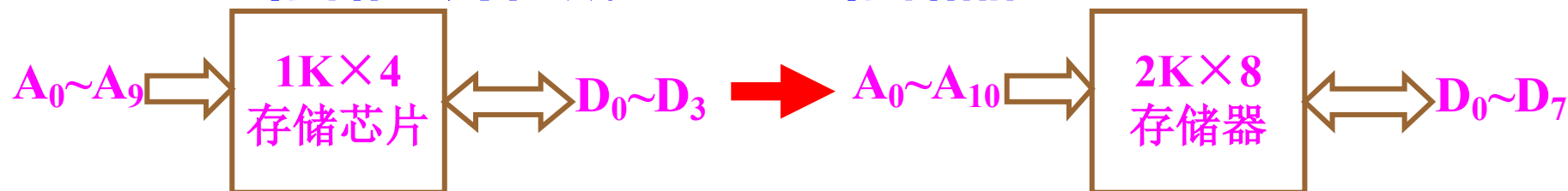
0000H	16K×8
3FFFH	
4000H	16K×8
7FFFH	
8000H	16K×8
0BFFFH	
0C000H	16K×8
0FFFFH	

注意反向思考：由地址空间推导选择芯片

>>> 存储芯片的字位扩展

➤ 字位扩展：每个单元位数和总的单元个数都增加。

➤ 例如：用 $1\text{K} \times 4$ 的存储芯片构成 $2\text{K} \times 8$ 的存储器



➤ 扩展方法

□ 先进行位扩展，形成满足位要求的存储芯片组；

□ 再使用存储芯片组进行字扩展。

➤ 要求：能够计算出字位扩展所需的存储芯片的数目。

□ 例如：用 $L \times K$ 的芯片构成 $M \times N$ 的存储系统；

✓ 所需芯片总数为 $M/L \times N/K$ 片。

>>> 【练习】 用 $2\text{M} \times 8$ 的SRAM芯片构成一个 $16\text{M} \times 8$ 的存储器，请回答以下问题：

1. 共需要几块芯片，进行如何扩展？

□ 8片 $2\text{M} \times 8$ 的SRAM芯片进行字扩展；

2. 各芯片的数据线怎么连？

□ 各芯片的数据线均直接与8位系统数据总线连接；

3. 各芯片的地址线怎么连？

□ 各芯片的地址线均直接与最低21位的系统地址线连接；

4. 各芯片的读写、片选信号线怎么连？

□ 各芯片的读写信号直接与系统的读写控制线连接；

□ 各芯片的片选信号由剩余的高3位系统地址线和访存允许信号/ MREQ 译码产生；

✓ 保证CPU发出不同地址，访问不同芯片；

>>> 例1. 设CPU有16根地址线，8根数据线，用/MREQ作访存控制

—— 现有下列芯片：1K×4RAM；4K×8RAM；8K×8RAM；2K×8ROM；4K×8ROM；8K×8ROM及74LS138等电路

要求：构成地址为6000~67FFH的系统程序区、地址为6800~6BFFH的用户程序区，选择芯片并画出逻辑连接图。

➤ 先存储器地址段分析：

	A ₁₅	...	A ₁₁	A ₁₀	A ₉	...	...	A ₀	
6000H	0	1	1	0	0	0	0	0000	} 系统程序区 2K×8位
67FFH	0	1	1	0	1	1	1	1111	
6800H	0	1	1	0	1	0	0	0000	} 用户程序区 1K×8位
6BFFH	0	1	1	0	1	1	1	1111	

➤ 存储芯片选择

□ 系统程序区：1片2K×8ROM

□ 用户程序区：2片1K×4RAM，做位扩展

} 再做字扩展



例1.设CPU有16根地址线，8根数据线，用/MREQ作访存控制

现有下列芯片：1K×4RAM；4K×8RAM；8K×8RAM；
2K×8ROM；4K×8ROM；8K×8ROM及74LS138等电路

要求：构成地址为6000~67FFH的系统程序区、地址为
6800~6BFFH的用户程序区，选择芯片并画出逻辑连接图。

➤存储器地址段分析：

	A ₁₅	...	A ₁₁	A ₁₀	A ₉	...	...	A ₀	
6000H	0	1	1	0	0	0	0	0000	系统程序区 2K×8位
67FFH	0	1	1	0	1	1	1	1111	
6800H	0	1	1	0	1	0	0	0000	用户程序区 1K×8位
6BFFH	0	1	1	0	1	1	1	1111	

➤译码芯片选择规则

- 用剩下的地址位选择
- 以低位占用的地址线较多的高位作为译码器的输入

作为译码器的输入

A₁₀无法作为译码器的输入

芯片及引脚分析

➤ 2K×8ROM

□地址线: $A_0 \sim A_{10}$

□数据线: $D_0 \sim D_7$

□控制线: $/CS$

	A_{15}	...	A_{11}	A_{10}	A_9	...	...	A_0	
6000H	0	1	0	0	0	0	0000	0000	系统程序区 2K×8位
67FFH	0	1	0	1	1	1	1111	1111	
6800H	0	1	0	1	0	0	0000	0000	用户程序区 1K×8位
6BFFH	0	1	0	1	1	1	1111	1111	

$A_0 \sim A_{10}$

剩余 $A_{15} \sim A_{11}$
0110 0

2K×8
ROM

$D_0 \sim D_7$

$/CS$

➤ 1K×4RAM

□地址线: $A_0 \sim A_9$

□数据线: $D_0 \sim D_3$

□控制线: $/CS$ 、 $/WE$

剩余 $A_{15} \sim A_{10}$
0110 10

$A_0 \sim A_9$

1K×4
RAM

$D_0 \sim D_3$

$/CS$

$/WE$

➤ CPU

□地址线: $A_0 \sim A_{15}$

□数据线: $D_0 \sim D_7$

□控制线: $/WE$ 、 $/MREQ$

剩余 $A_{15} \sim A_{12}$
0110

根据上文分析应使用
 $A_{13} \sim A_{11}$ 作为地址译
码信号, 产生各存储
芯片的 $/CS$



➤ 存储器地址段分析:

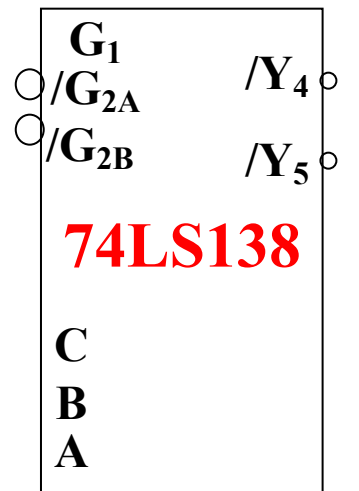
	A_{15}	...	A_{11}	A_{10}	A_9	...	...	A_0	
6000H	0	1	1	0	0	0	0000	0000	系统程序区 2K×8位
67FFH	0	1	1	0	1	1	1111	1111	
6800H	0	1	1	0	0	0	0000	0000	用户程序区 1K×8位
6BFFH	0	1	1	0	1	1	1111	1111	

➤ 译码芯片选择规则

作为译码器的输入

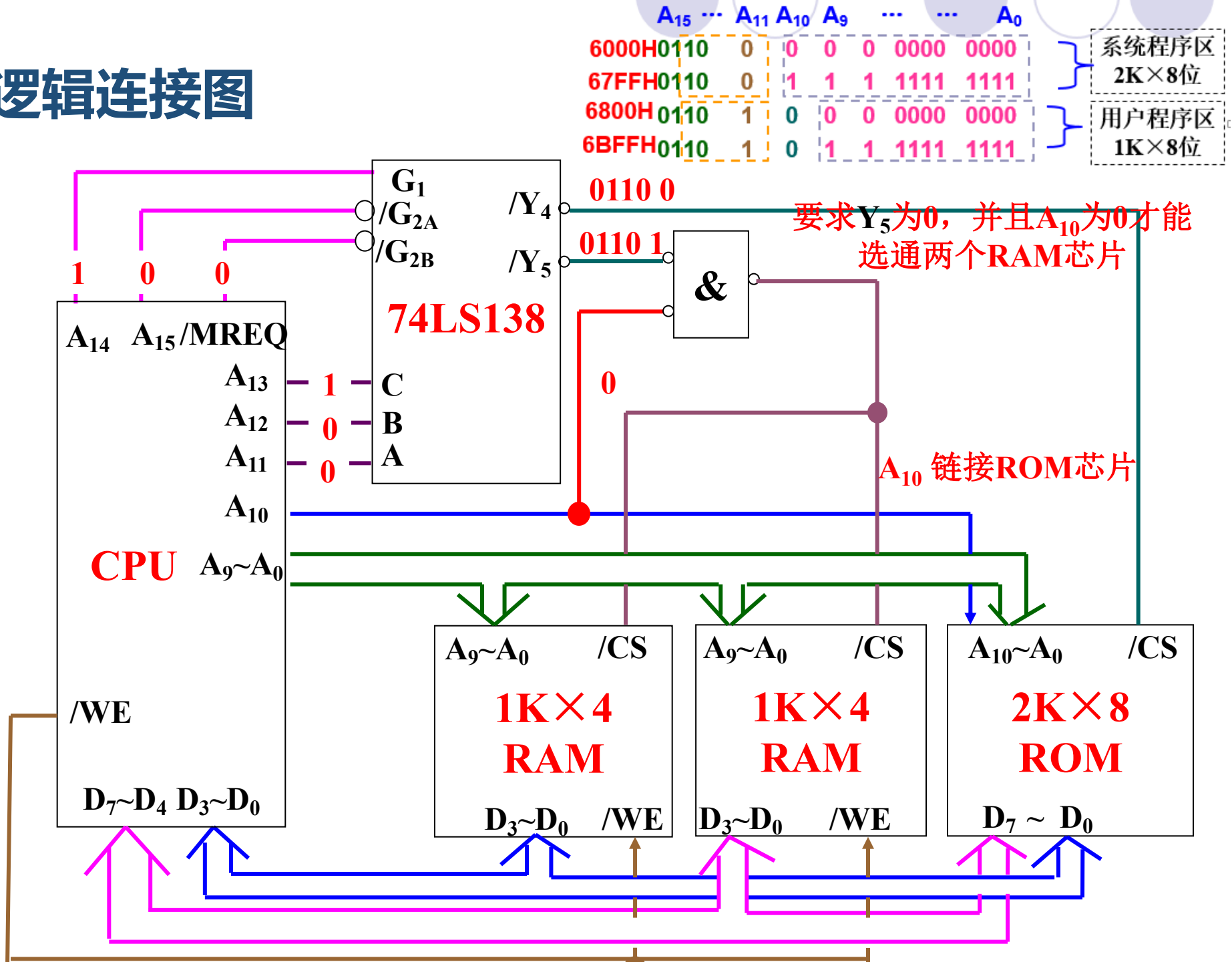
- 从剩下的地址位选择
- 以低位占用的地址线较多的高位作为译码器的输入及其访存控制信号的链接。

思考: $A_{15} \cdots A_{11}$ 如何分配? 注意还有 A_{10}
当 $A_{13}A_{12}A_{11}=100$, 译码器CBA=100, 即 Y_4 有效
当 $A_{13}A_{12}A_{11}=101$, 译码器CBA=101, 即 Y_5 有效



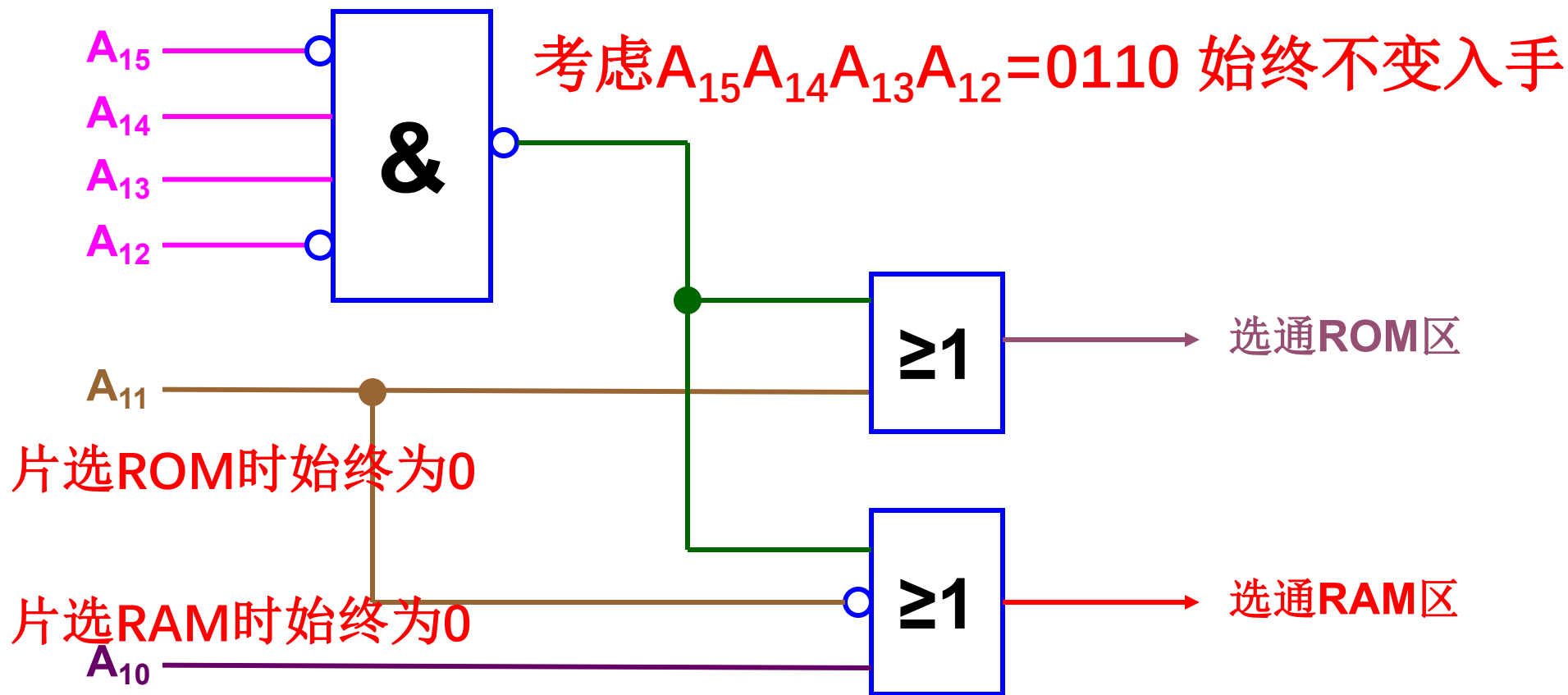


逻辑连接图



>>> 译码设计方案2

	A_{15}	...	A_{11}	A_{10}	A_9	...	...	A_0	
6000H	0	1	1	0	0	0	0000	0000	系统程序区 2K×8位
67FFH	0	1	1	0	1	1	1111	1111	
6800H	0	1	1	0	0	0	0000	0000	用户程序区 1K×8位
6BFFH	0	1	1	0	1	1	1111	1111	





例2. CPU有16根地址线，8根数据线，要求主存地址空间满足：最小8K为系统程序区，与其相邻的16K地址为用户程序区，最大4K地址空间为系统程序工作区，划出逻辑图及指出芯片种类及片数。

➤ 可选存储芯片：

1K×4RAM; 4K×8RAM; 8K×8RAM;
2K×8ROM; 4K×8ROM; 8K×8ROM;

0010 0000 0000 0000
0011 1111 1111 1111

0100 0000 0000 0000
0101 1111 1111 1111

➤ 存储器地址分析：

□ 最小8K系统程序区

1片8K×8ROM，高3位地址为000

0000 0000 0000 0000 ~ 0001 1111 1111 1111

□ 接下来的16K用户程序区

2片8K×8RAM，高3位地址为001、010

0010 0000 0000 0000 ~ 0011 1111 1111 1111

0100 0000 0000 0000 ~ 0101 1111 1111 1111

□ 最大4K系统程序工作区

1片4K×8RAM，高4位地址为1111

1111 0000 0000 0000 ~ 1111 1111 1111 1111



逻辑连接图

存储器地址分析:

最小8K系统程序区

0000 0000 0000 0000 ~ 0001 1111 1111 1111

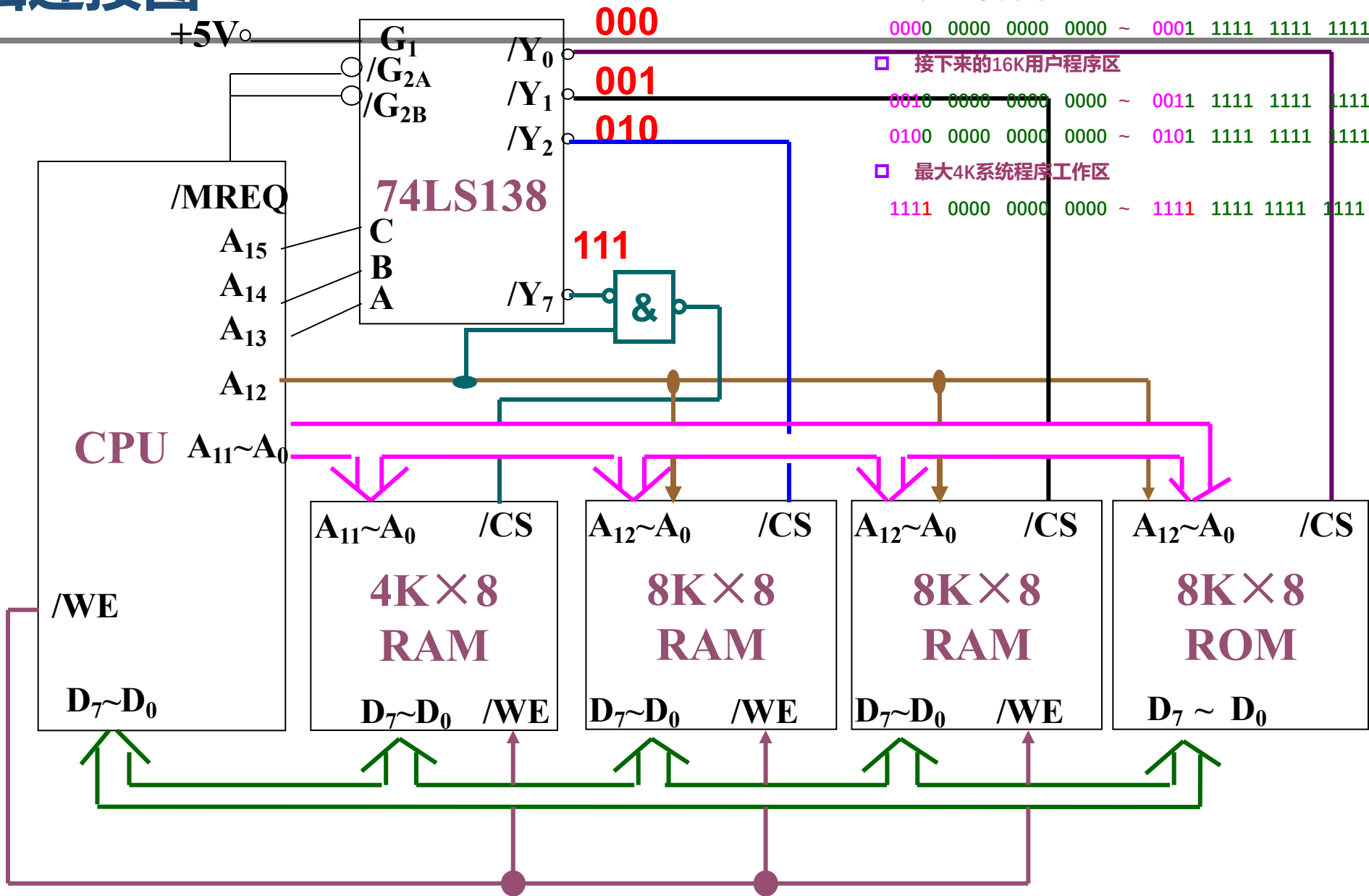
接下来的16K用户程序区

0010 0000 0000 0000 ~ 0011 1111 1111 1111

0100 0000 0000 0000 ~ 0101 1111 1111 1111

最大4K系统程序工作区

1111 0000 0000 0000 ~ 1111 1111 1111 1111



>>> 例题总结

- 根据CPU先确认数据线与地址线
- 分析CPU要求的地址空间，如何选择ROM与ROM
- 然后确定存储芯片如何进行字位扩展
- 确认译码器的输入端，确认剩余的其他地址位如何直接或间接地与其他所有的芯片链接
- 如果有其他题中提到MAR, MDR, 需要明白的是二者逻辑上是属于存储器的，但被硬件设计到了CPU内。

>>> 位扩展与字扩展的区别

➤ 片选

- 位扩展时片选选中所有扩展芯片；
- 字扩展时根据译码输出选通线决定所选中的芯片

➤ 访问

- 位扩展访问访问是对各位扩展芯片相同地址单元的同时访问。
- 字扩展访问是对某一字扩展芯片的一个单元访问。

>>> 存储器设计的连接要点

➤ 地址线的连接

- 用CPU的低位地址线与芯片地址线直接连接；

➤ 数据线的连接

- 用CPU的对应位数据线与芯片的数据线直接连接；

➤ 读/写控制信号线的连接

- 用CPU的读/写控制信号线直接与存储芯片直接连接；

➤ 片选线的连接

- 一般使用CPU的高位地址线的和CPU的访存允许控制信号线/MREQ，经译码器译码后产生各芯片的片选信号。

- **关键点：**保证从每个芯片的角度来看，都完全使用了CPU引脚（直接或间接连接）；

>>> 存储器与CPU的连接补充例子

做题思路：

- 审题确定所需扩展的类型，选择合适的存储芯片；
 - 原则：尽量作简单的扩展（位扩展—字扩展—字位扩展）
- 分析存储芯片和CPU的引脚特性（地址范围、地址线数目、容量要求等），确定引脚的连接；
 - 尤其是在进行字扩展时，特别注意片选信号的产生。
 - ✓ 3-8译码器74LS138、双2-4译码器74LS139
- 画出逻辑连接图，作必要的分析说明。

2009年考研统考第15题

某计算机主存容量为64KB，其中ROM区为4KB，其余为RAM区，按字节编址，现要用 $2K \times 8$ 位的ROM芯片和 $4K \times 4$ 位的RAM芯片来设计该存储器，则需要上述规格的ROM芯片数和RAM芯片数分别是（ ）

- | | | | |
|-------------------------|------|------------------------------------|------|
| ☐ A | 1、15 | ☐ C | 1、30 |
| ☐ B | 2、15 | ☒ D | 2、30 |

提交

某计算机主存容量为**64KB**，其中**ROM**区为**4KB**，其余为**RAM**区，按字节编址，现要用**2K×8**位的**ROM**芯片和**4K×4**位的**RAM**芯片来设计该存储器，则需要上述规格的**ROM**芯片数和**RAM**芯片数分别是（ ）

- ☐ A 1、15 ☐ C 1、30
☐ B 2、15 ☒ D 2、30

提交

解答： 本题主要考查主存储器的组成。题中已知计算机的主存容量为**64KB**，其中**ROM**为**4KB**，其余为**RAM**，可知**RAM**大小为**60KB**。所以，用**2K×8**位的**ROM**芯片构成**4KB**（同时可知存储器位数是**8**位），需要**4KB/2KB=2**片，而用**4K×4**位的**RAM**芯片构成**60KB**的**RAM**区，需要**60K×8位/4K×4位=30**片（进行位扩展）。

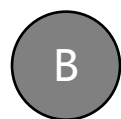
2010年考研408真题 第15题

15、假定用若干个 $2K \times 4$ 位芯片组成一个 $8K \times 8$ 位的存储器，则地址0B1FH所在芯片的最小地址是（ ）



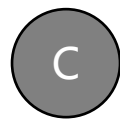
A

0000H



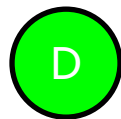
B

0600H



C

0700H



D

0800H

提交

2010年考研408真题 第15题

15、假定用若干个 $2K \times 4$ 位芯片组成一个 $8K \times 8$ 位的存储器，则地址0B1FH所在芯片的最小地址是（ 0800H ）

用 $2K \times 4$ 位的芯片组成一个 $8K \times 8$ 位存储器，共需8片 $2K \times 4$ 位的芯片，分为4组，每组由2片 $2K \times 4$ 位的芯片并联组成 $2K \times 8$ 位的芯片，各组芯片的地址分配如下：

第一组（2个芯片并联）：0000H~07FFH。

第二组（2个芯片并联）：0800H~0FFFH。

第三组（2个芯片并联）：1000H~17FFH。

第四组（2个芯片并联）：1800H~1FFFH。

地址0B1FH所在的芯片属于第二组，故其所在芯片的最小地址为0800H。

2011年考研统考真题 第15题

15、某计算机存储器按字节编址，主存地址空间大小为64MB，现用 $4\text{M} \times 8$ 位的RAM芯片组成32MB的主存储器，则存储器地址寄存器MAR的位数至少是（ ）

A 22位

B 23位

C 25位

D 26位

提交

2011年考研统考真题 第15题

15、某计算机存储器按字节编址，主存地址空间大小为64MB，现用4M×8位的RAM芯片组成32MB的主存储器，则存储器地址寄存器MAR的位数至少是（ 26位 ）

主存按字节编址，地址空间大小为64MB，MAR的寻址范围为 $64\text{M} = 2^{26}$ ，故为26位。实际的主存容量32MB不能代表MAR的位数，考虑到存储器扩展的需要，MAR应保证访问到整个主存地址空间，反过来，MAR的位数决定了主存地址空间的大小。

2016年考研统考真题 第16题

16、某存储器容量为64KB，按字节编址，地址4000H-5FFFH为ROM区，其余为RAM区，若用8K×4位的SRAM芯片设计，则需要该芯片的数量为（ ）

A 7

B 8

C 14

D 16

提交

5FFF-4000 + 1 = 2000H，即 ROM 区容量为 $2^{13}\text{B} = 8\text{ KB}$ ($2000\text{H} = 2 \times 16^3 = 2^{13}$)，RAM 区容量为 56KB (64 KB-8 KB=56KB)，则需要 8 K×4位的 SRAM 芯片的数量为 14 (56KB/8 K×4位 = 14)。

2021年考研统考真题 第15题

15、某计算机的存储器总线中有24位地址线和32位数据线，按字编址，机器字长32位。如果000000H-3FFFFFFH为RAM区，那么需要512K×8位的RAM芯片数为（ ）。

A

8

B

16

C

32

D

64

提交

000000~3FFFF，共有 $3FFFFFFH - 000000H + 1H = 400000H = 2^{22}$ 个地址，按字编址，字长为32位（4B），因此RAM区大小为 $2^{22} \times 4B = 2^{22} \times 32\text{bit}$ 。每个RAM芯片的容量为 $512K \times 8\text{bit} = 2^{19} \times 8\text{bit}$ ，所以需要RAM芯片的数量为 $(2^{22} \times 32\text{bit}) / (2^{19} \times 8\text{bit}) = 32$ 。

>>> 2021年考研题

➤4、某计算机的存储总线中有24位地址线和32位数据线，按字编址，字长是32位，若000000-3FFFFFFH RAM区，则需要512K*8位的RAM芯片数为（ ）

➤解答：

➤但存储系统有24位地址线和32位数据线，需要现有的芯片进行字位扩展。

➤但系统是按字编址，即每个地址对应一个字。一个字=32bit=4B，000000-3FFFFFFH,需要22根地址线，即 2^{22} 个地址22位,即 $2^{22} * 32\text{bit}$ 。

➤每块芯片大小为 $2^{19} * 8\text{bit} = 2^{17} * 32\text{bit}$,共需 $2^{22} * 32\text{bit} / 2^{17} * 32\text{bit} = 32$ 块芯片

>>> 2010年考研统考第15、16题

15. 假定用若干个 $2K \times 4$ 位芯片组成一个 $8K \times 8$ 位的存储器，则地址0B1FH所在芯片的最小地址是 (D)

A. 0000H B. 0600H C. 0700H D. 0800H

16. 下列有关RAM和ROM的叙述中，正确的是 (A)

I、 RAM是易失性存储器，ROM是非易失性存储器

II、 RAM和ROM都是采用随机存取的方式进行信息访问

III、 RAM和ROM都可用作Cache

IV、 RAM和ROM都需要进行刷新

A. 仅I和II B. 仅II和III C. 仅I, II, III D. 仅II, III, IV

➤➤➤15. 假定用若干个2K×4位芯片组成一个8K×8位的存储器，则地址0B1FH所在芯片的最小地址是（ 0800H ）

➤解答：0B1FH——0000 1011 0001 1111 B

➤所需芯片：(8K×8) / (2K×4) = 8 个，先位扩展，再字扩展

地址范围：第一组

0 0000 0000 0000
0 0111 1111 1111



第二组

0 1000 0000 0000
0 1111 1111 1111 →

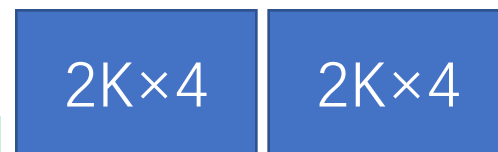
0800H



第三组

1 0000 0000 0000 →
1 0111 1111 1111

10000H



第四组

1 1000 0000 0000 →
1 1111 1111 1111

1800H



>>> 2014年考研统考第15题

15. 某容量为256MB的存储器由若干 $4\text{M} \times 8$ 位的DRAM芯片构成，该DRAM芯片的地址引脚和数据引脚总数是（ A ）

A. 19

B. 22

C. 30

D. 36

【解答】

$4\text{M} \times 8$ 位的存储芯片，SRAM需要地址引脚22根，数据引脚8根；

DRAM地址引脚减半，故总引脚数为 $11+8=19$ 根；

若是SRAM芯片，则总引脚数目为30根；

2023年考研统考真题 第15题

15、某计算机的 CPU 有 30 根地址线，按字节编址，CPU 和主存芯片连接时，要求主存芯片占满所有可能存储地址空间，并且 RAM 区和 ROM 区所分配的空间大小比为 3:1，若 RAM 在连续低地址区，ROM 在连续高地址区，则 ROM 的地址范围（ ）。

A

0000 0000H~0FFF FFFFH

B

10000000H~2FFFFFFFH

C

3000 0000H~3FFF FFFFH

D

40000000H~4FFFFFFFH

提交

2023年考研统考真题 第15题

15、某计算机的 CPU 有 30 根地址线，按字节编址，CPU 和主存芯片连接时，要求主存芯片占满所有可能存储地址空间，并且 RAM 区和 ROM 区所分配的空间大小比为 3:1，若 RAM 在连续低地址区，ROM 在连续高地址区，则 ROM 的地址范围（ ）。 **3000 0000H~3FFF FFFFH**

C。【解析】由于 CPU 有 30 根地址线，每根地址线可以表示 2 个不同的状态（0 或 1）。因此，CPU 的地址线共有 2^{30} 个不同的地址。根据 RAM 和 ROM 的分配比，RAM 区占据整个地址空间的 $\frac{3}{4}$ ，而 ROM 区占据剩下的 $\frac{1}{4}$ ，我们可以选取高位的两根地址线，00、01、10 分配给连续低地址的 RAM，则高位只剩下 11 可以分配，所以可以分配的地址范围为 110...00(28个0) ~ 11...1(30个1)，十六进制表示为 3000 0000H ~ 3FFF FFFFH。故本题的正确选项为 C。

>>> 3.5 并行存储器

➤ 引入原因

- CPU与主存之间速度不匹配(以及容量、速度、价格的要求)
- 为了提高二者之间的数据交换速度，在不同层次上采用不同的技术
 - ✓ **芯片技术**：选用材料，或芯片内部结构或接口，例如突发传输技术，同步DRAM等
 - ✓ **结构技术**：改进CPU与主存之间的链接方式（本节）
 - ✓ **系统结构技术**：系统分层，见本章第一节

>>> 3.5 并行存储器

- 3.5.1 双端口存储器（空间并行技术）
- 3.5.2 多模块交叉存储器（时间并行技术）

>>> 3.5.1 双端口存储器

➤ **早期**计算机系统以CPU为中心，机器内的各个部件、外设与主存之间信息交互都要经过CPU运算器，影响了CPU效率。

□ **改进：**产生了以内存为中心的系统取代了以CPU为中心的系统。

□ 多个部件可以和内存直接交互，减轻了CPU的负担，增加访存次数

✓ **改进：**传统存储器在任意时刻只能进行一个读写的弊端，为扩大主存的信息交互能力，提出了多口存储器结构。



3.5.1 双端口存储器DPRAM

➤ 双端口存储器采用空间并行技术：

□ 同一个存储体使用两组相互独立的读写控制线路，可并行操作。

➤ 读写特点（两种情况）

□ 无冲突读写

✓ 访问的存储单元不同，可并行读写存储体；

□ 有冲突读写

✓ 两个端口同时存储同一个单元，且至少一个端口写，冲突产生

✓ 同时访问同一存储单元，片上的判断逻辑电路决定了访问优先权

✓ 被延时的端口可使用/BUSY信号控制暂时关闭此端口

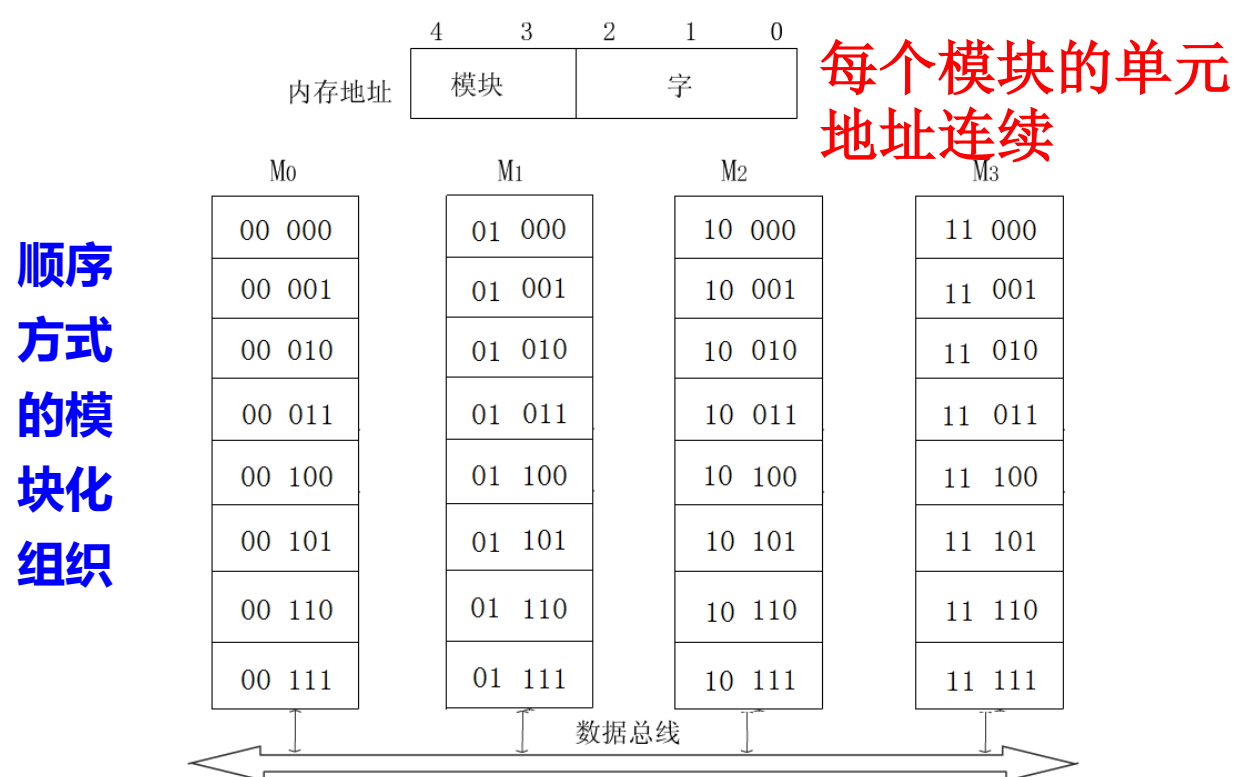
➤ 显卡上的存储器一般都是双端口存储器。

➤ 缺点：硬件设计比较复杂，需要两套电路

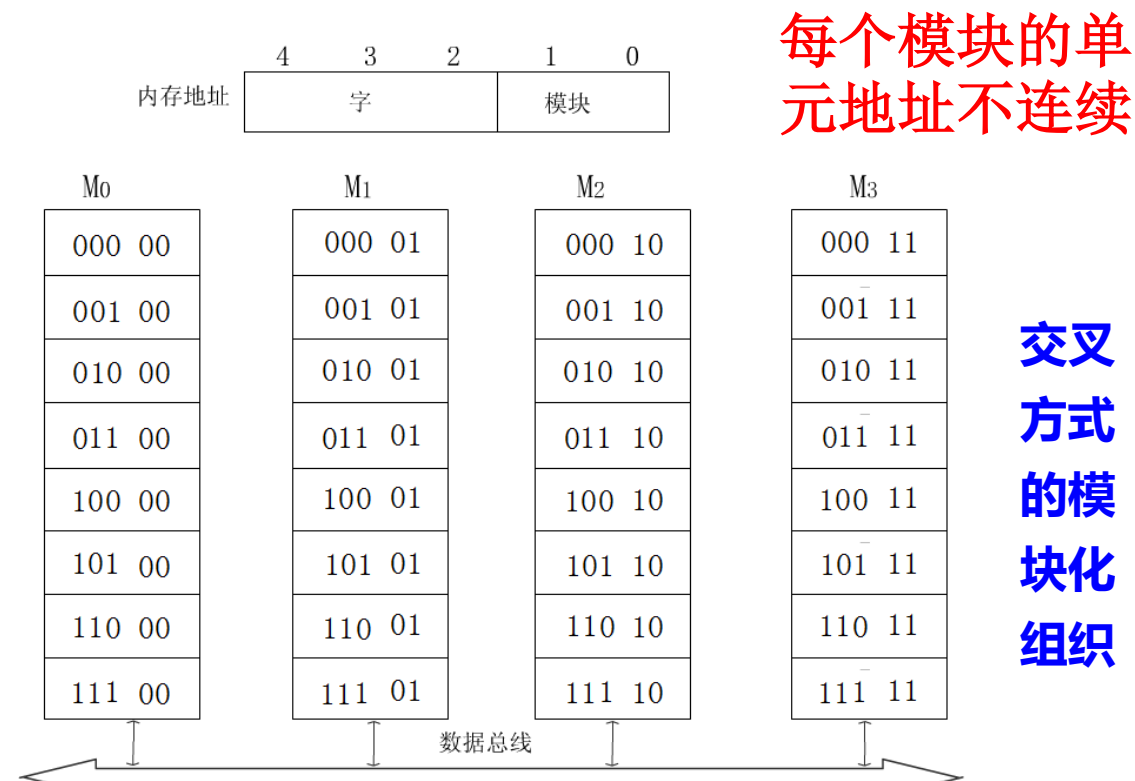
➤ 但适用于一些特殊场合，比如两个CPU交换数据

>>> 3.5.2 多模块交叉存储器

- 多模块交叉存储器由多**若干个模块**组成，采用**时间**并行技术。其工作方式依赖于它的模块组织方式（或者地址编码方式）
- 存储器的模块化组织方式（地址都是线性的）----顺序方式、交叉方式



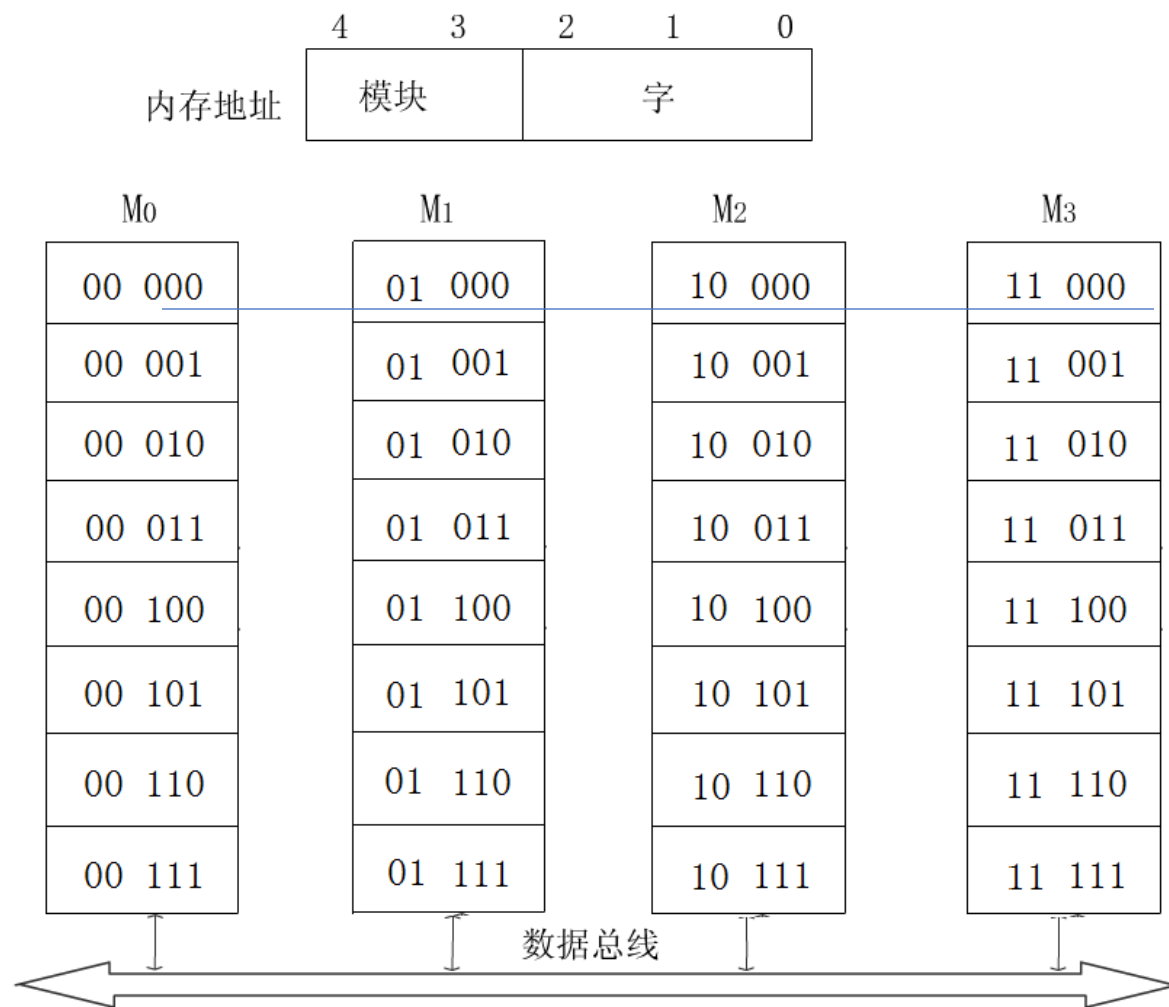
- ◆ 优点：直接增添模块扩充存储器容量比较方便；
- ◆ 缺点：各模块串行工作，存储器的带宽受到了限制。



- ◆ 优点：块数据传送时，可大大提高存储器的带宽；
- ◆ 缺点：模块间依赖性强，不易扩展存储容量。

>>> 多模块交叉存储器——顺序方式（高位交叉）

- 每个模块中的单元地址是连续的、线性的；
- 某个模块进行存取时，其他模块不工作，某一模块出现故障时，其他模块可以照常工作；
- 存储单元地址
 - 高位——模块号；
 - 低位——模块内的字号；
 - 右方图中共32个字



>>> 多模块交叉存储器——顺序方式（高位交叉）

➤ 每个模块中的单元

地址是连续的；

顺序访问：设存储周期为 T ,访问5个存储单元

假设连续访问

00 000

00 001

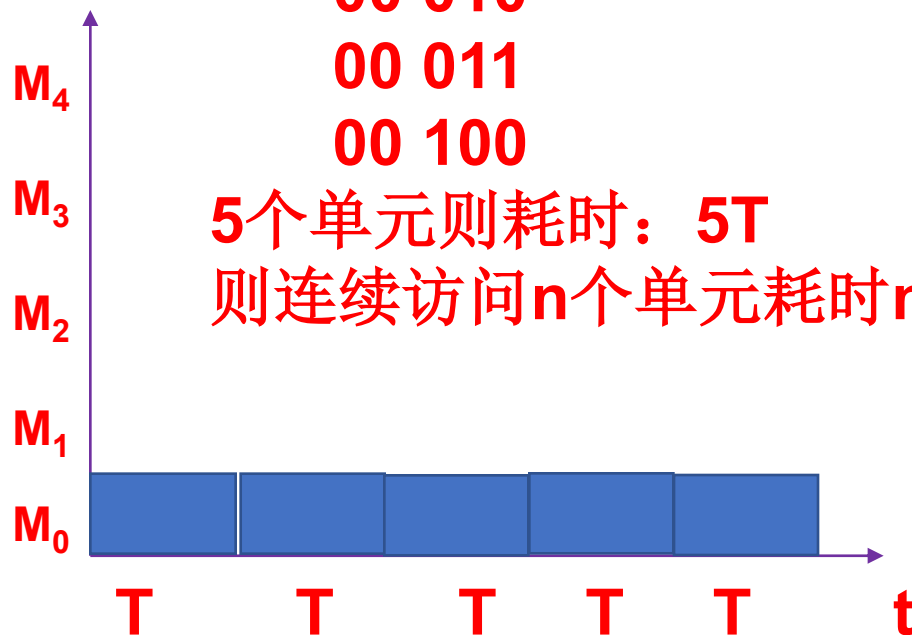
00 010

00 011

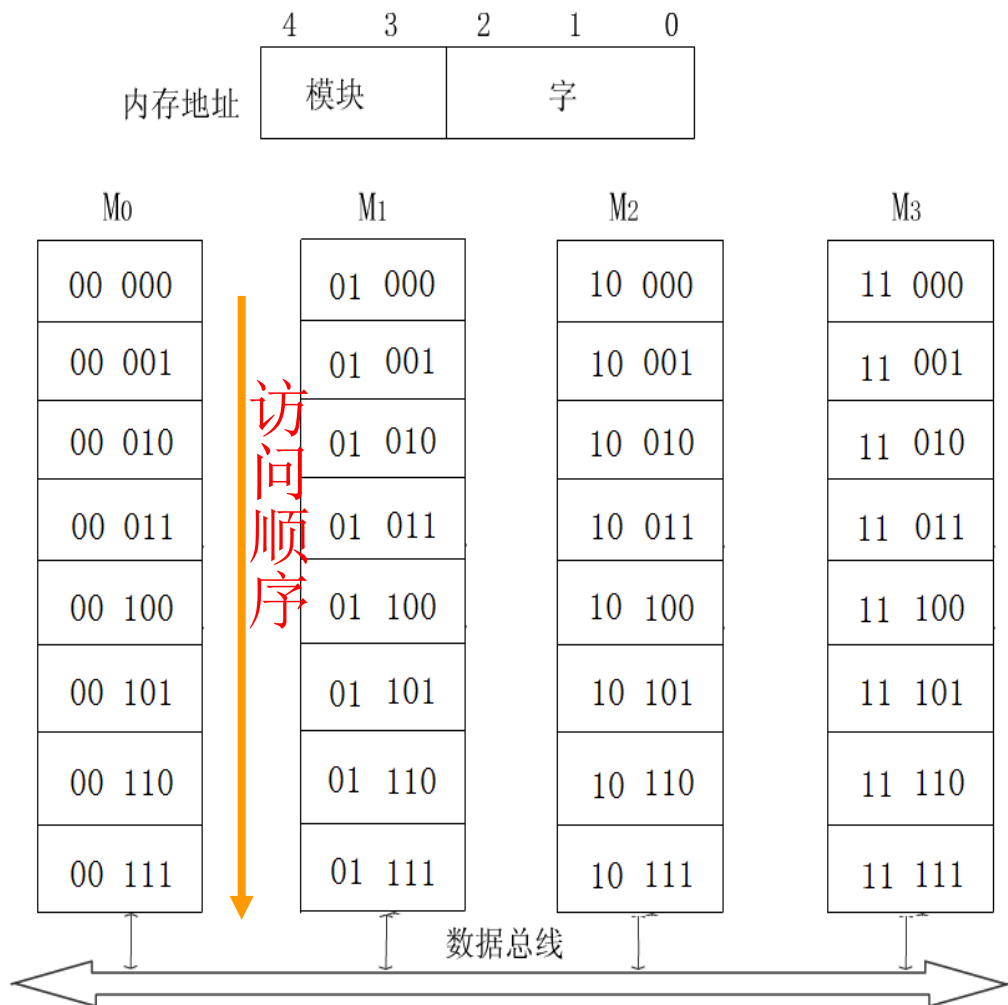
00 100

5个单元则耗时： $5T$

则连续访问 n 个单元耗时 nT

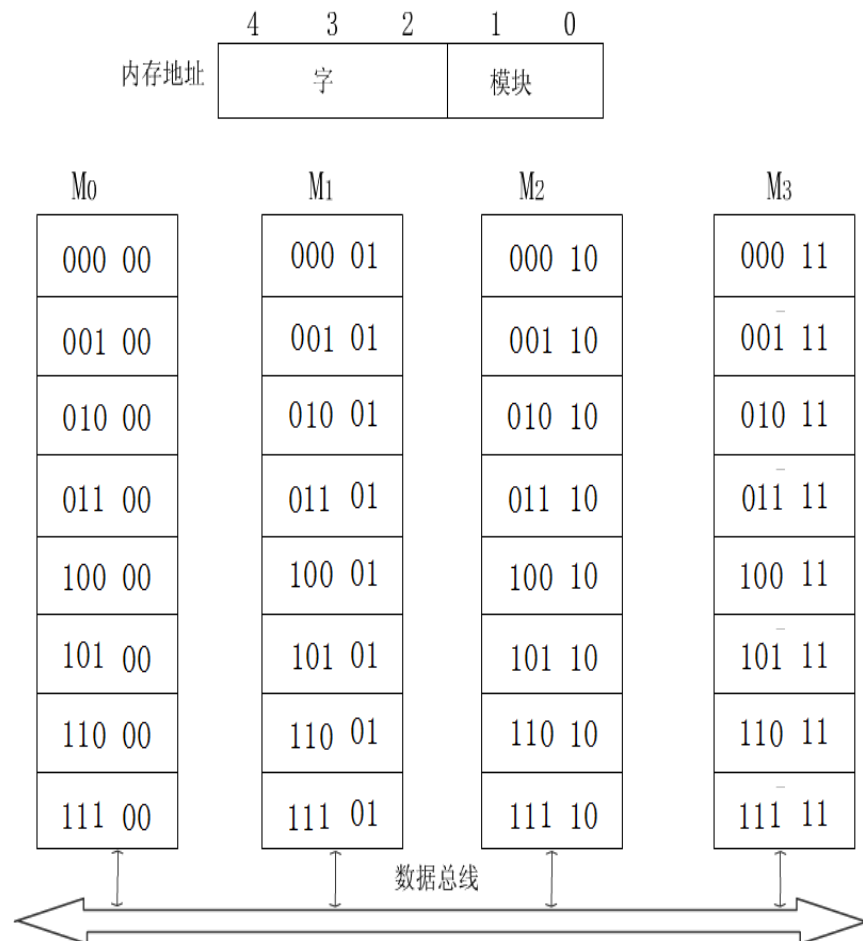


特点：类似于扩容



>>> 多模块交叉存储器——交叉方式（低位交叉）

- 每个模块的单元地址是不连续的；连续地址分布在相邻的不同模块内。每个模块都有自己的读写控制电路等。
- 对于数据的成块传送，各模块可以实现多模块流水式并行存取；（第一个模块读取完毕在恢复时间第二个模块开始工作的模式）
- 存储单元地址
 - 低位——模块号；
 - 高位——模块内的字号；



>>> 多模块交叉存储器——交叉方式（低位交叉）

➤ 每个模块的单元地址是不连续的；

➤ 连续地址分布在相邻的不同模块内。

交叉访问：设存储周期为 T ，连续访问4个存储单元则需要不同存储体，如下图

假设连续访问

000 00

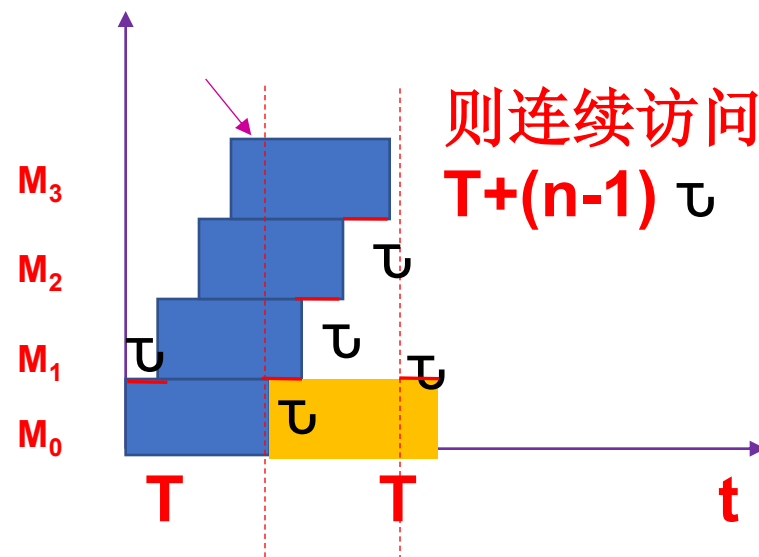
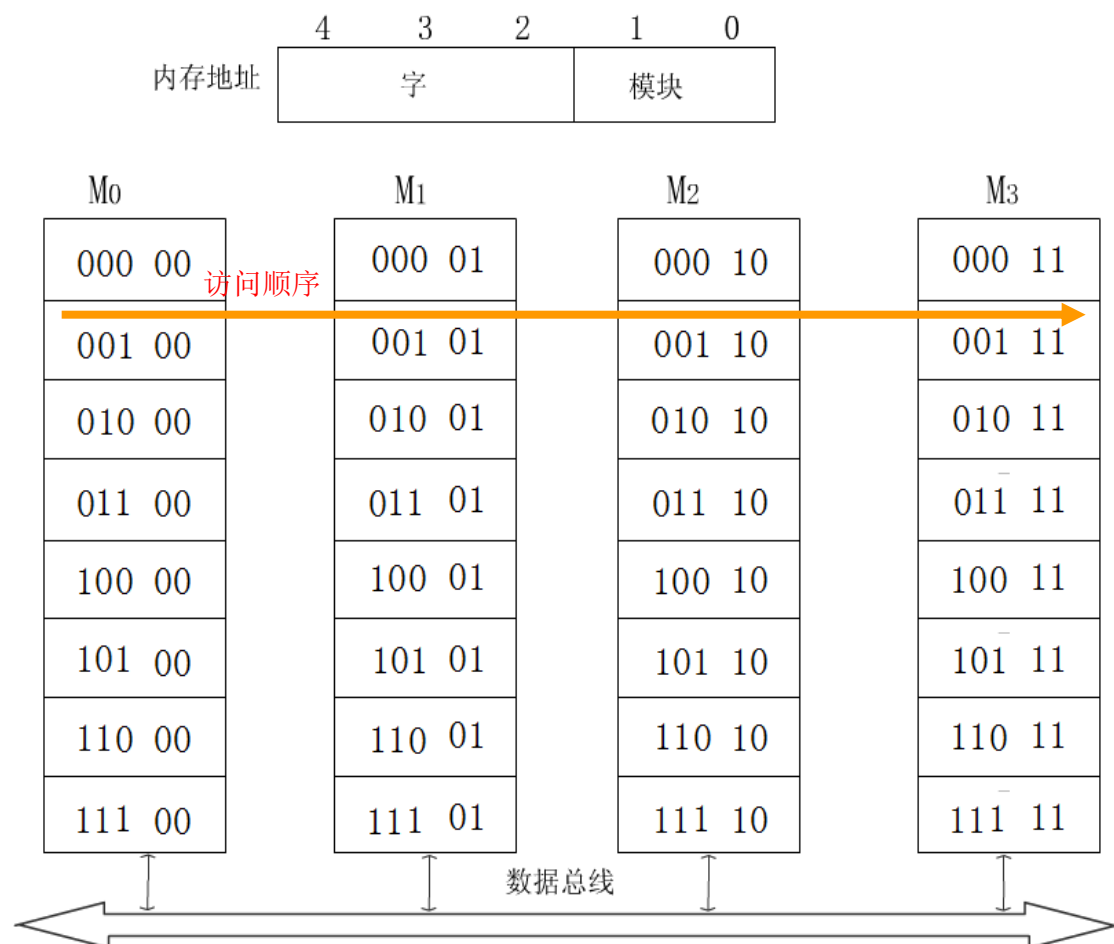
000 01

000 10

000 11

4个单元则耗时： $T+3\tau$

则连续访问 n 个单元耗时
 $T+(n-1)\tau$



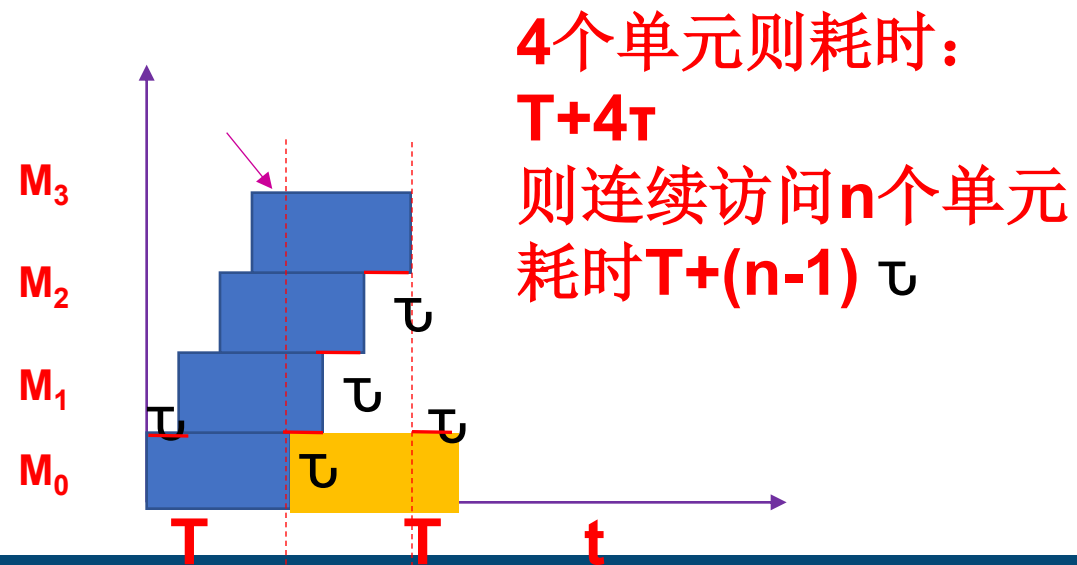
>>> 多模块交叉存储器——交叉方式（低位交叉）

- 虽然M0, M1, M2, M3在一个T周期内分别读取一个字, 但只能在同一时间往总线上传送一个字
- 从M1取到字需要等待M0模块取到的字在总线上传送完毕, 即需要等待 τ
- 后续每个字需要等待一个 τ 传送时间

顺序访问: 设存储周期为T, 连续访问4个存储单元则需要不同存储体, 如下图

假设连续访问

000 00
000 01
000 10
000 11

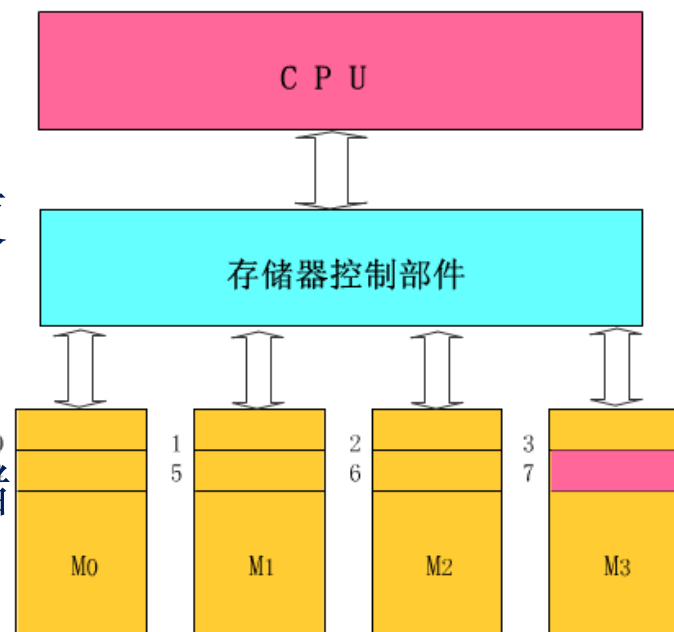


>>> 多模块交叉存储器

- **默认或理想状态下是连续访问多个字**
- **每个模块各自以等同的方式与CPU传送信息。**
- **CPU同时访问四个模块，由存储器控制部件控制它们分时使用数据总线进行信息传递。**

对每一个模块来说，从CPU给出访存命令直到读出信息仍然使用了一个存取周期时间但却访问了四个模块如右图；

对CPU来说，交叉模式，它可以在一个存取周期中连续访问4个模块；各模块的读写过程重叠进行，所以这是一种并行存储器结构。



>>> 多模块交叉存储器——交叉方式

➤ 定量分析（最优模块数）

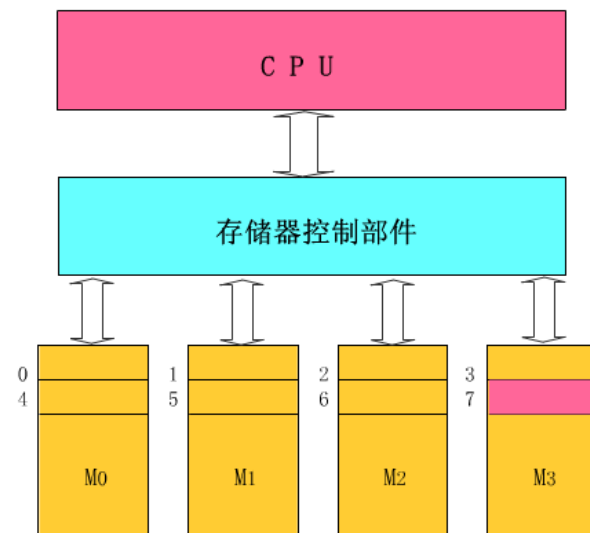
➤ 假设：交叉方式模块字长=数据总线宽度,存取周期 T ,总线传送周期为 τ ，存储器模块数为 m ，为了实现流水线方式存取，应当满足： $T \leq m\tau$

□ 即成块传送可按 τ 间隔流水方式进行，也就是每经过 τ 时间延迟后启动下一个模块。

□ 其中 $m_{\min} = T/\tau$,称为交叉存取度，因此交叉存储器要求模块数必须大于或等于 $m_{\min}$ 以保证启动某模块后经过 $m\tau$ 时间再次启动该模块时，它的上次存取操作已经完成。这样连续读取 m 个字所需的时间为： $t_1 = T + (m-1)\tau$ $m_{\min} = T/\tau$

➤ 而顺序方式存储器连续读取 m 个字所需时间为 $t_2 = mT$

由于 $t_1 < t_2$ ，交叉存储器的带宽确实大大提高了。





课本P91【例5】

设存储器容量为32字,字长64位,模块数 $m=4$,连续访问4个字分别用顺序方式和交叉方式进行组织。存储周期 $T=200\text{ns}$,数据总线宽度为64位,总线传送周期 $\tau=50\text{ns}$ 。问顺序存储器和交叉存储器的带宽各是多少?

- 顺序存储器和交叉存储器连续读出 $m=4$ 个字的数据信息量为

$$q=4 \times 64=256\text{位}$$

- 顺序存储器所需要的时间为

$$t_1=m \times T=4 \times 200\text{ns}=800\text{ns}=8 \times 10^{-7}\text{s}$$

- 故顺序存储器的带宽为

$$W_1=q/t_1=256/(8 \times 10^{-7})=32 \times 10^7[\text{bit/s}]$$

- 交叉存储器所需要的时间为

$$t_2=T+(m-1) \times \tau=200\text{ns}+(4-1) \times 50\text{ns}=350\text{ns}=3.5 \times 10^{-7}\text{s}$$

- 故交叉存储器的带宽为

$$W_1=q/t_1=256/(3.5 \times 10^{-7})=73 \times 10^7[\text{bit/s}]$$

总线传送周期 τ , 即每隔 τ 时间启动下个存储体

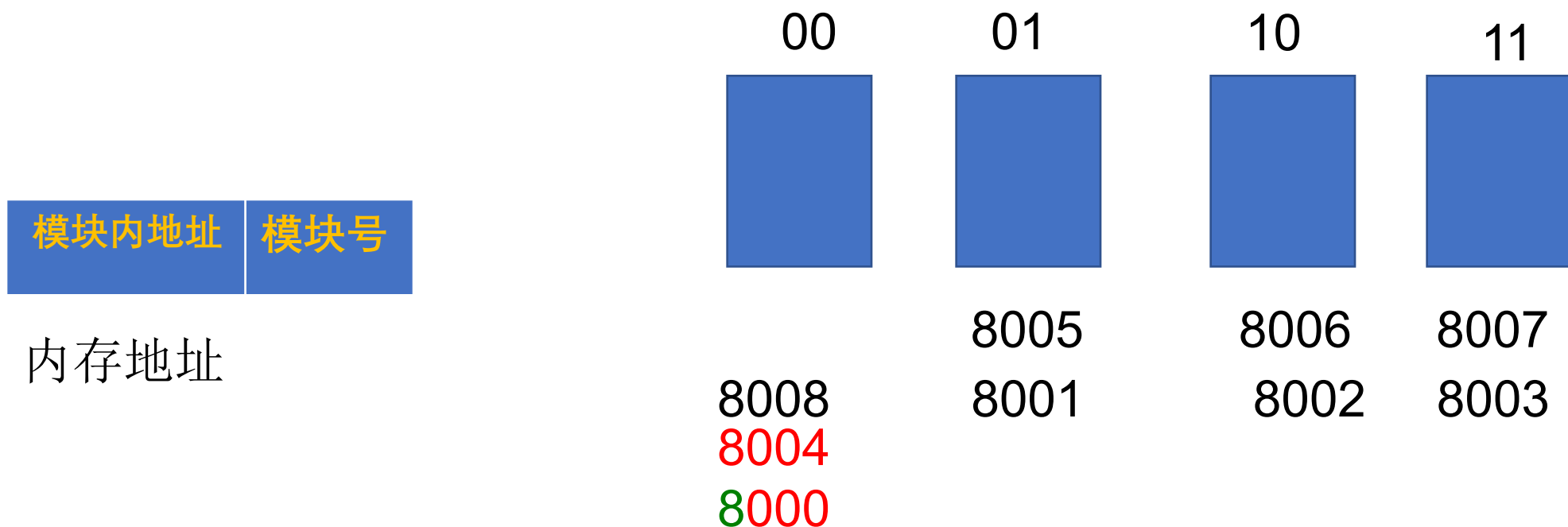
带宽=数据量/所需时间
 $=m \times 64/\tau$

(2015年) 某计算机使用4体交叉编址存储器, 修定在存储器总线上出现的主存地址(十进制) 序列为8005, 8006, 8007, 8008, 8001, 8002, 8003, 8004, 8000, 则可能发生访存冲突的地址是 ()

- ☐ A 8004和8008
- ☐ B 8002和8007
- ☐ C 8001和8008
- ☒ D 8000和8004

某计算机使用4体交叉编址存储器，修定在存储器总线上出现的主存地址（十进制）序列为8005, 8006, 8007, 8008, 8001, 8002, 8003, 8004, 8000, 则可能发生访存冲突的地址是（ ）

解答：5—101, 6—110, 7—111, 8—100, 1—001, 2—010 3—011, 4—100, 0—000 最低两位就是模块编号—交叉方式



2017考研题：某计算机主存按字节编址，由4个64M×8位的DRAM芯片采用交叉编址方式构成，并与宽度为32位的存储器总线相连，主存每次最多读写32位数据。若double型变量x的主存地址为804 001AH，则读取x需要的存储周期数是（）

A 1

B 2

C 3

D 4

提交

2017考研题：某计算机主存按字节编址，由4个64M×8位的DRAM芯片采用交叉编址方式构成，并与宽度为32位的存储器总线相连，主存每次最多读写32位数据。若double型变量x的主存地址为804 001AH，则读取x需要的存储周期数是（）

解答：考察知识点：

- 1、芯片扩展：位扩展，同时选中所有4个芯片。2、交叉编址：低位位存储体编号。
3、编址：字节编址，题目提供的字长为4个字节，因此访问的时候是一次从4个存储芯片的一个单元读取4个字节的内容，而开始读取的是2号存储体，而变量x是8个字节。因此需要3个周期（读8次）

804 001AH: *****10

模块内地址	模块号
-------	-----

内存地址

	00	01	10	11
T1			√	√
T2	√	√	√	√
T3	√	√		
T4				

>>> 3.6 Cache存储器

3.6.1 Cache基本原理

3.6.3 替换策略

3.6.4 Cache的写操作策略

3.6.2 主存与Cache的地址映射

>>> 3.6 Cache存储器

从主存结构上来说，多体交叉存储器确实可以提高带宽但优化后的速度依然与**CPU**有很大差距

进一步优化每个存储单元的设计，但带来的是---价格**高**，容量**小**

后来，编程中发现，程序实际更多时候使用的是很小一块的空间，于是提出了**cache**

程序局部原理：在最近的未来要用到的指令和数据（信息），很可能与现在正在使用的信息在存储空间是邻近的，例如数组 $a[i] = a[i]++$ 访存两次（空间），例如循环（时间）

存储体系改善：分层次

>>> 3.6.1 Cache基本原理

动画演示:

Cache的功能.swf

➤使用Cache的原因

- CPU速度越来越快，主存储器与CPU的速度差距越来越大，影响CPU的工作效率。

➤Cache的作用

- 在CPU和主存之间加一块高速的SRAM（Cache）；
- 主存中将要被访问或者频繁被访问的数据提前送到Cache中，作为主存中对应数据的副本；
- CPU访存时，先访问Cache，若没有再访问主存进行数据调度。

➤使用Cache的依据

程序访问的局部性原理(空间和时间)

- 在一段时间内，CPU所执行的程序和访问的数据大部分都在某一段地址范围内，而该段范围外的地址访问很少；例如数组的访问。
- 最近要访问的信息，可能就是现在正在访问的信息，例如循环。

>>> 1、Cache的基本功能

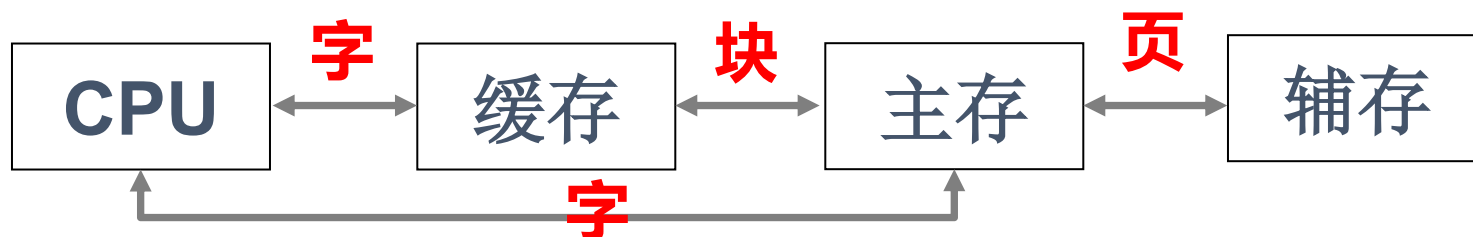
➤ 结构模块化

- CPU访问Cache或主存时，以字/字节为单位；
- Cache和主存交换信息时，以块为单位，一次读入一块或多块内容；
 - ✓ 每块由若干个字组成，是定长的；
- Cache的每行都设置有标记，CPU访问程序或数据时，先访问标记。

Cache的一块，
也称为一行

➤ 此结构全部由硬件实现；

➤ Cache对程序员是透明的，即程序员不必知道是否存在Cache。



>>> 关于块/行

假设某机器主存容量为1MB，Cache容量为8KB，存储器以字节编址，每块512B。

存储单元地址	主存		块地址
00000H		512B	000H
00001H			
...	...		
001FFH			
00200H		512B	001H
00201H			
...	...		
003FFH			
00400H			...
...	...	...	
FFFFFH		...	

Cache		行地址
标记		
主存块号	512B	00H
主存块号	512B	01H
...	...	...
主存块号	512B	0FH

Cache行数远小于主存块数

>>> 关于块/行

假设某机器主存容量为1MB，Cache容量为8KB，存储器以字节编址，每块512B。

存储单元地址	主存		块地址
00000H		512B	000H
00001H			
...	...		
001FFH			
00200H		512B	001H
00201H			
...	...		
003FFH			
00400H			...
...	...	...	
FFFFFH		...	

Cache		行地址
标记		
主存块号	512B	00H
主存块号	512B	01H
...	...	...
主存块号	512B	0FH

Cache行数远小于主存块数

>>> 3、Cache的命中率

- 命中率是指CPU要访问的信息在Cache中的比率；

$$\text{命中率}_h = \frac{\text{访问信息在Cache中的次数}}{\text{访问总次数}} \times 100\% \quad \text{一般} > 95\%$$

- 访问总次数 = 访问cache次数 + 访问主存次数
- 失效率 = 1 - 命中率，即访存率指CPU要访问的信息在主存的比率
- 影响命中率的主要因素
 - Cache 容量： 过小时，局部信息装不完，命中率低。
过大时，对提高效率不明显，且成本高。
 - Cache中块的大小：
一般用一个主存周期所能调出的单元数（字或字节）作为一个块大小。

>>> 主存系统的平均访问时间

➤ Cache/主存系统的平均访问时间 t_a 为

$$t_a = ht_c + (1-h)t_m$$

t_c ——命中时的Cache访问时间

t_m ——未命中时的主存访问时间

h ——命中率

ht_c ——命中Cache时的平均访问时间

$(1-h)t_m$ ——未命中Cache时对主存的平均访问时间

➤ 设主存与Cache的速度倍率 $r = t_m/t_c$, 则系统的访问效率 e (指从cache的性能考虑) 为

$$e = \frac{t_c}{t_a} = \frac{t_c}{ht_c + (1-h)t_m} = \frac{1}{h + (1-h)r} = \frac{1}{r + (1-r)h}$$

>>> 课本例子

CPU执行一段程序时，Cache完成存取的次数为1900次，主存完成存取的次数为100次，已知Cache存取周期为50ns，主存存取周期为250ns，求Cache/主存系统的效率和平均访问时间。

➤ 命中率

$$\square h = N_c / (N_c + N_m) = 1900 / (1900 + 100) = 0.95$$

➤ 主存与Cache的速度倍率（从存取周期考虑）

$$\square r = t_m / t_c = 250\text{ns} / 50\text{ns} = 5$$

➤ 访问效率

$$\square e = \frac{1}{r + (1-r)h} = \frac{1}{5 + (1-5)0.95} = 83.3\%$$

➤ 平均访问时间

$$\square t_a = t_c / e = 50\text{ns} / 0.833 = 60\text{ns}$$

2009年考研统考题目 第21题

21、假设某计算机的存储系统由Cache和主存组成。某程序执行过程中访存1000次，其中访问cache缺失（未命中）50次，则Cache的命中率是（ ）

☐ A 5%

☐ B 9.5%

☐ C 50%

☒ D 95%

提交

>>> 4、cache 结构设计必须解决的问题

➤由CPU的工作原理可以看出，cache 的设计需要遵循两个原则：

□cache 的命中率尽可能高，实际应接近于 1；

□cache 对 CPU 而言是透明的，即不论是否有 cache，CPU 访存的方法都是一样的，软件不需增加任何指令就可以访问 cache。

➤解决了命中率和透明性问题，就 CPU 访存的角度而言，内存将具有主存的容量和接近 cache 的速度。为此，必须增加一定的硬件电路完成控制功能，即 cache 控制器。

在设计 cache 结构时，必须解决后面4个问题。

>>> 4、Cache结构设计必须解决的问题

- ①主存的内容调入 cache 时如何存放?
- ②访存时如何找到 cache 中的信息?
- ③当cache空间不足时如何替换 cache 中已有的内容
- ④需要写操作时如何改写 cache 的内容?

① ②是相互关联的，即如何将主存信息定位在 cache 中，如何将主存地址映射为 cache 地址。

➤ **地址映射**：为了把主存块放到 cache 中，必须应用某种方法把主存地址定位到 cache 中，称为地址映射。“映射”一词的物理含义是确定位置的对应关系，并用硬件来实现。

这样当 CPU 访问存储器时，它所给出的一个字的内存地址会自动变换成 cache 的地址，即 cache 地址变换。

>>> 4、Cache的读操作

➤ CPU发出有效的主存地址

- 查找相联存储器/块表，判断所要访问的信息是否在Cache中；
- 若命中，经地址变换机构，将主存地址变换为相应的Cache地址；
 - CPU直接读取Cache获取数据；
- 若未命中，则CPU访问主存并完成读取，并判断Cache是否已满；
 - 若Cache未满，将该数据所在块从主存中调入Cache（有可能调入成功或失败）；
 - 若Cache已满，使用某种替换机制，使用当前数据块替换掉Cache中的某些块。

>>> 5、Cache的写操作

➤ CPU发出有效的主存地址;

- 查找相联存储器/块表, 判断所要访问的信息是否在Cache中;
- 若命中, 经地址变换机构, 将主存地址变换为相应的Cache地址;
 - 使用某种写策略将数据写入Cache或主存。
- 若未命中, 则使CPU直接写主存数据;
 - 同时根据写策略, 决定是否将该块内容调入Cache;

>>> 3.6.2 主存与Cache的地址映射

➤ 信息从主存→Cache中，如何定位？

□ Cache的容量小于主存，需要采用某种算法确定主存和Cache中块的对应关系；

➤ 地址映射

□ 主存中数据块调入Cache中时，主存数据块与Cache行之间的**映射**关系；

➤ 地址变换

□ CPU访存时，将主存地址**按映射函数关系**变换成Cache地址的过程；

➤ 地址映射的方式

□ 全相联映射、直接映射、组相联映射；

□ 即将主存中的块放到Cache中的哪个位置？

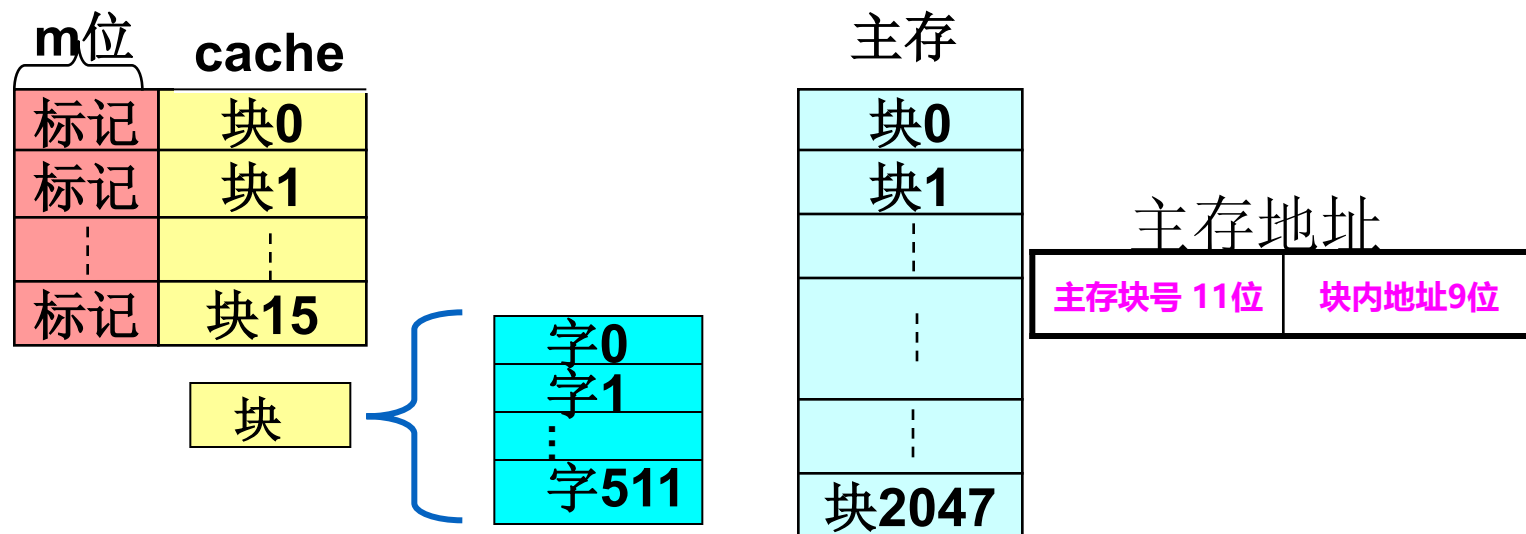
>>> 关于主存的块与cache的行

➤ cache和主存机械等分为相同大小的块，每一块是由若干个字（或字节）组成。

例：某机主存容量为1MB，每块512B，划分为2048块；主存地址共20位，块内地址9位，主存块号11位。

cache容量为8KB，每块512B，划分为16块（也称为行）。主存的块与cache块或行对应。

由于cache的块（行）数远小于主存的块数，因此一个cache不能唯一地、永久地只对应一个存储块，在cache中，每一块外加有一个标记，指明它是主存的哪一块的副本（拷贝）。



>>> 1、全相联映射 (Associative Mapping)

➤ 映射关系

□ 主存中的任意字块可调进Cache的任一行中;

➤ 地址映射

□ 主存中数据块调入Cache时, 可以调入Cache的任一空行;

□ 调入的同时, 将主存标记(除去块内地址)和Cache的行号同时写入块表;

✓ 将主存标记保存于调入Cache行的对应标记位

✓ 主存标记=主存块号

➤ 地址变换

□ CPU访存时, 发出主存地址;

□ 将主存标记作为关键字, 送入块表中检索每一个单元;

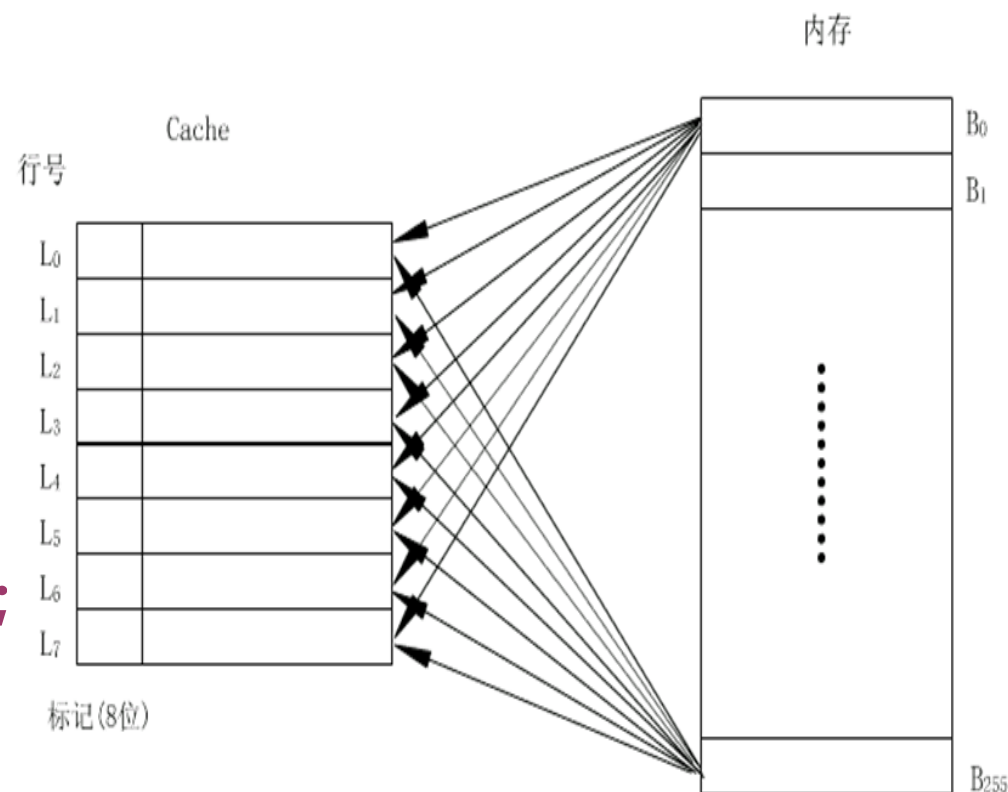
□ 命中时, 读出对应单元中的Cache行号;

□ 使用Cache行号和主存地址中的块内地址访问Cache;

➤ 块表大小

□ 单元数: 与cache的行数相同

□ 单元字长: 主存块号+Cache行号+1



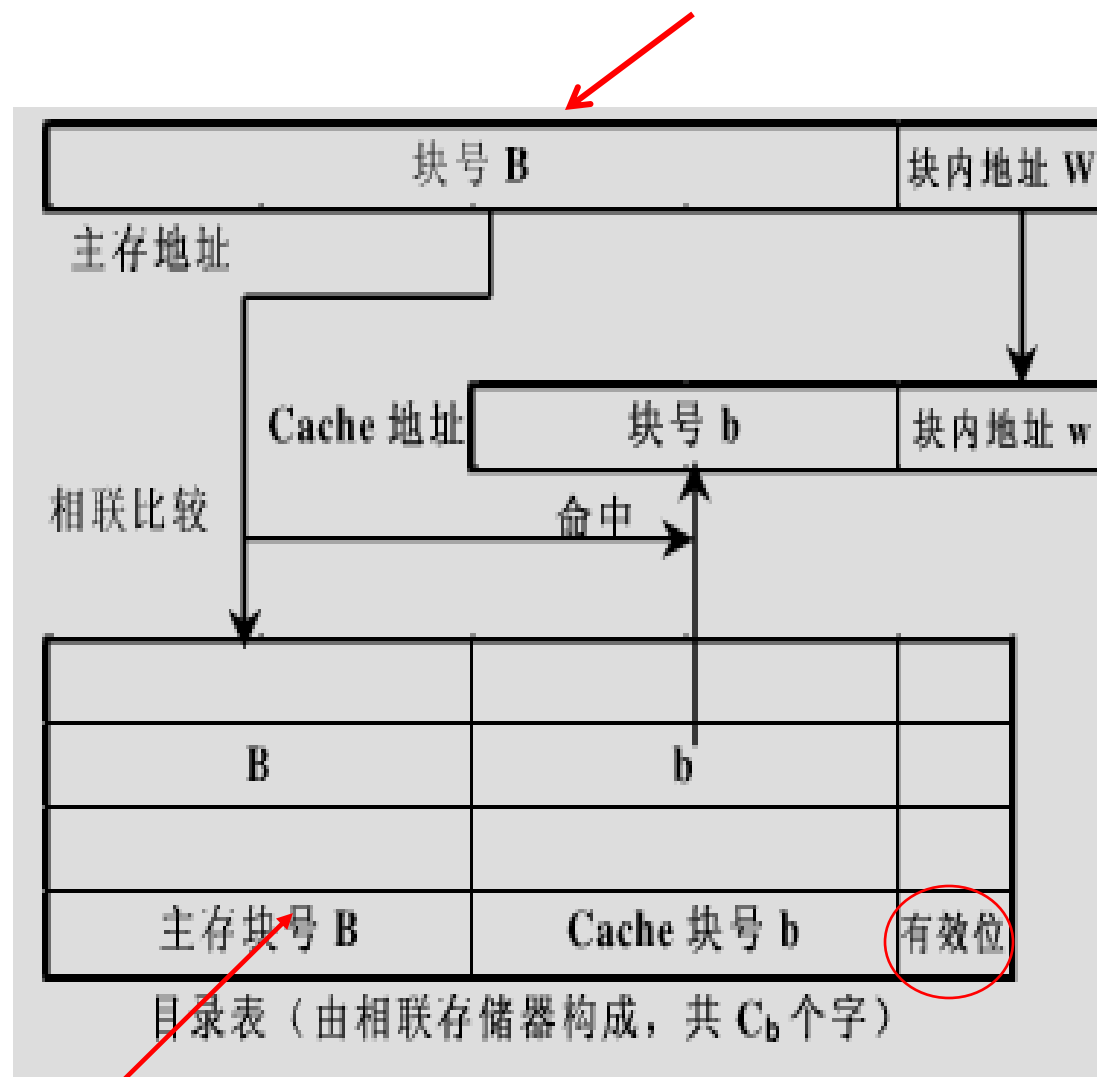
主存0—255块共256块, Cache标记位8位

>>> 全相联地址变换示意图

CPU

➤ 地址变换过程

- CPU发出主存地址;
- 在Cache块表中使用主存块号进行内容检索;
- 若命中, 则读出对应单元中的Cache行号;
- 使用Cache行号和主存地址中的块内地址访问Cache;



主存块号也可以理解为标记

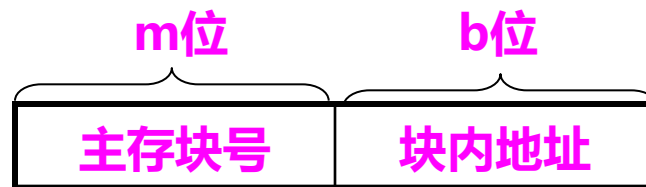
>>> 全相联映射的地址与块表格式

➤ 设主存共 2^m 个块，每块单元数为 2^b 个

□ 主存地址格式：

✓ 主存块号，也称为主存标记

□ Cache地址格式：



➤ 块表的基本结构

□ 按内容检索的相联存储器；

□ 块表大小： $2^c \times (m + c + 1)$ 位

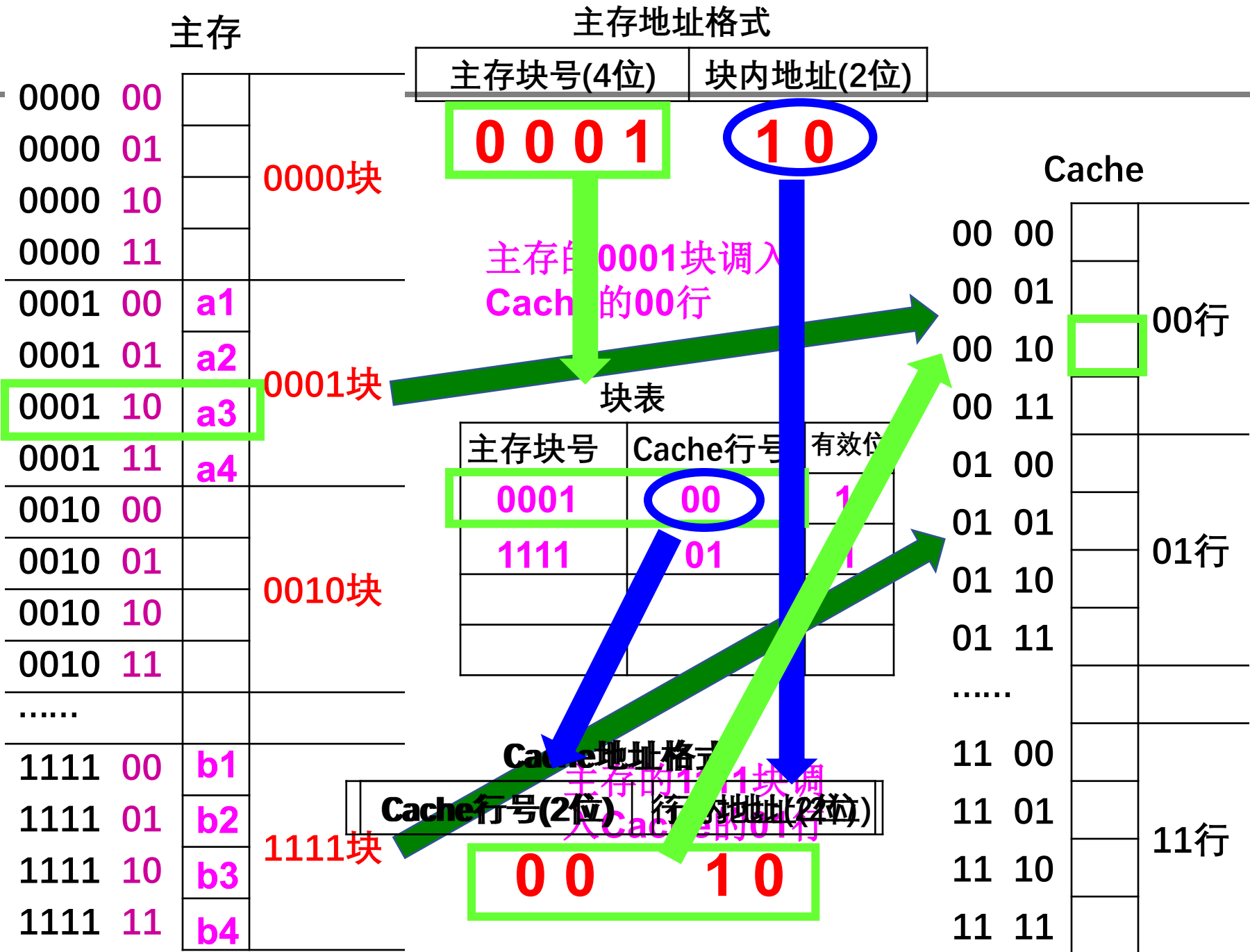
块表

主存块号	Cache行号	有效位
m位	c位	1位
...	...	

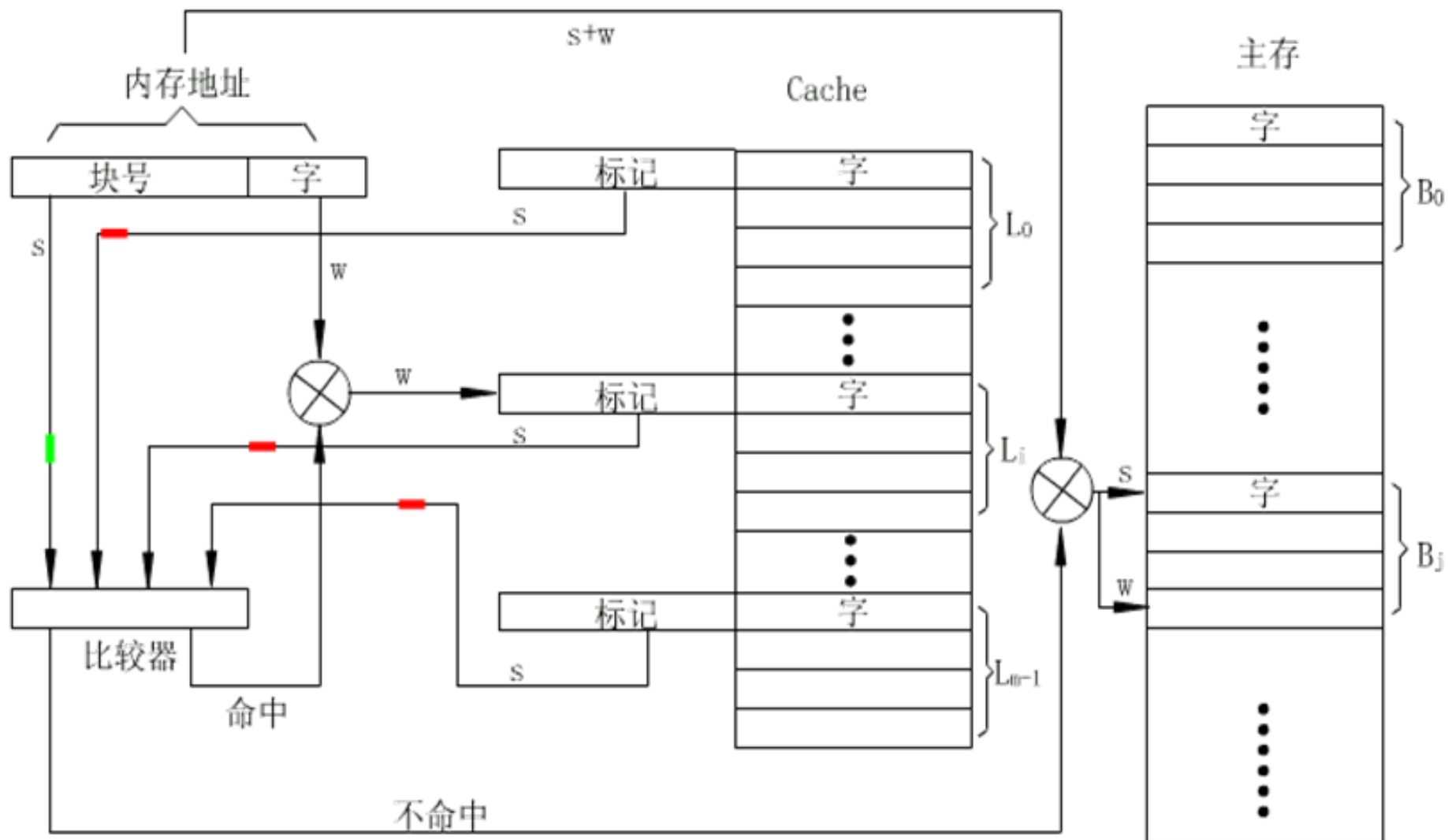
} 2^c



全相联映射方式举例



>>> 全相联映射的组织 (访存) [动画演示: 全相联映射.swf](#)



所有标记和主存地址中的标记位进行比较

>>> 全相联映射Cache的特点

➤ 优点

□ 灵活性好(最理想)

- ✓ Cache中只要有空行，就可以调入所需要的主存数据块；

➤ 缺点

□ 成本高

- ✓ 块表单元字长为 $m+c+1$ 位，需要较大容量的Cache块表；

□ 速度太慢

- ✓ 访问Cache时，需将所有标记比较一遍，才能最后判出所需主存中的字块是否在Cache中；

➤ 一般较少使用。

>>> 【例1】 设主存容量1MB， Cache容量16KB， 块的大小为512B，
采用全相联映射方式。

- ① 写出Cache的地址格式。
- ② 写出主存的地址格式。
- ③ 块表的容量多大？
- ④ 画出地址映射及变换示意图。
- ⑤ 主存地址为CDE8FH的单元， 在Cache中的什么位置？

>>> 【例1】 设主存容量1MB，Cache容量16KB，块的大小为512B，采用**全相联映射方式**。

① 写出Cache的地址格式

□ Cache地址格式

□ Cache的容量16KB

□ 块（行）的大小为512B

□ 行地址为 $14 - 9 = 5$ 位，c为5



→ Cache地址为14位

→ 行内地址为9位

→ Cache共32行

② 写出主存的地址格式

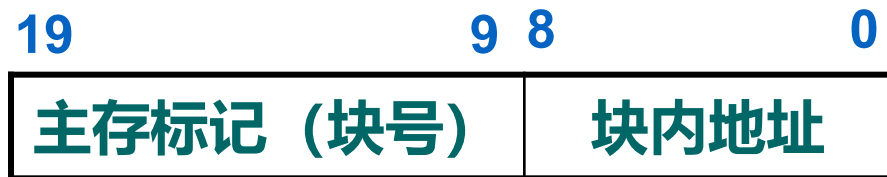
□ 主存的地址格式为

□ 主存容量1MB

□ 块的大小为512B

□ 块地址为 $20 - 9 = 11$ 位

□ m为11



→ 主存地址为20位

→ 块内地址为9位

→ 主存共2048块



>>> 【例1】设主存容量1MB，Cache容量16KB，块的大小为512B，采用全相联映射方式。

③ 块表的容量多大？

□ 块表的大小应为 $2^c \times (m + c + 1)$ 位，即 $2^5 \times 17$ 位；

块表

主存块号	Cache行号	有效位
m位	c位	1位
...	...	

} 2^c

④ 画出地址映射及变换示意图（自学）。

主存地址为CDE8FH的单元，在Cache中的什么位置？

□ 主存地址为CDE8FH的单元可映射到Cache中的任何一个字块位置；

✓ CDE8FH= 1100 1101 1110 1000 1111 B

□ 其块/行内地址为：010001111。

>>> 2、直接映射 (Direct Mapping)

➤ 映射关系

□ 主存中的每一块数据只能调入Cache的特定行中;

□ 直接映射函数为: $i = j \bmod 2^c$

主存块号为j Cache行号为i

c是Cache行地址的位数

➤ 地址映射

□ 主存中数据块调入Cache时, 只能调入Cache的特定行;

□ 同时, 将主存标记写入块表中与Cache行地址相同的单元;

➤ 地址变换

□ CPU访存时, 发出主存地址;

□ 从主存地址中截取出Cache行号, 访问块表的对应单元;

□ 若该单元中数据与主存标记相同, 则命中, 否则未命中;

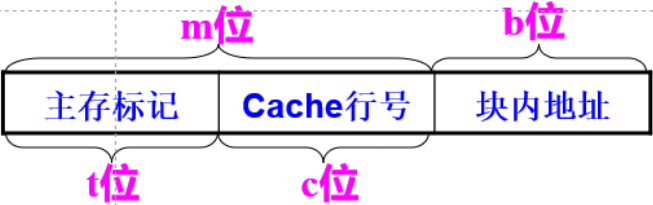
□ 命中时, 使用Cache行号和块内地址 (即主存地址中除主存标记位之外的其余位) 访问Cache;

利用行号选择
块表相应行



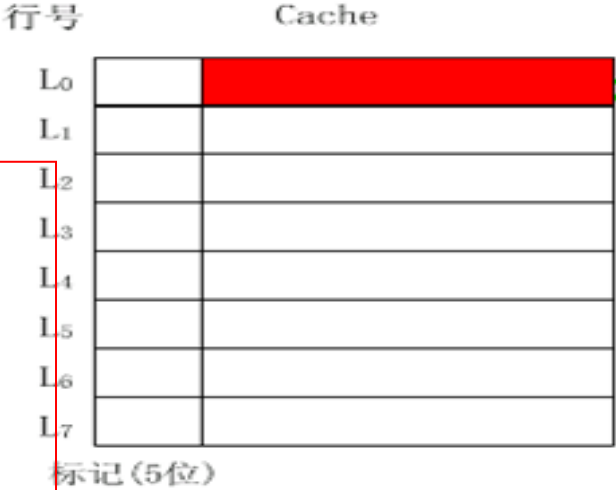
直接相联映射示意图

如果Cache行中满了（比如第0块），B8主存块映射时就要先务必替换出第0块

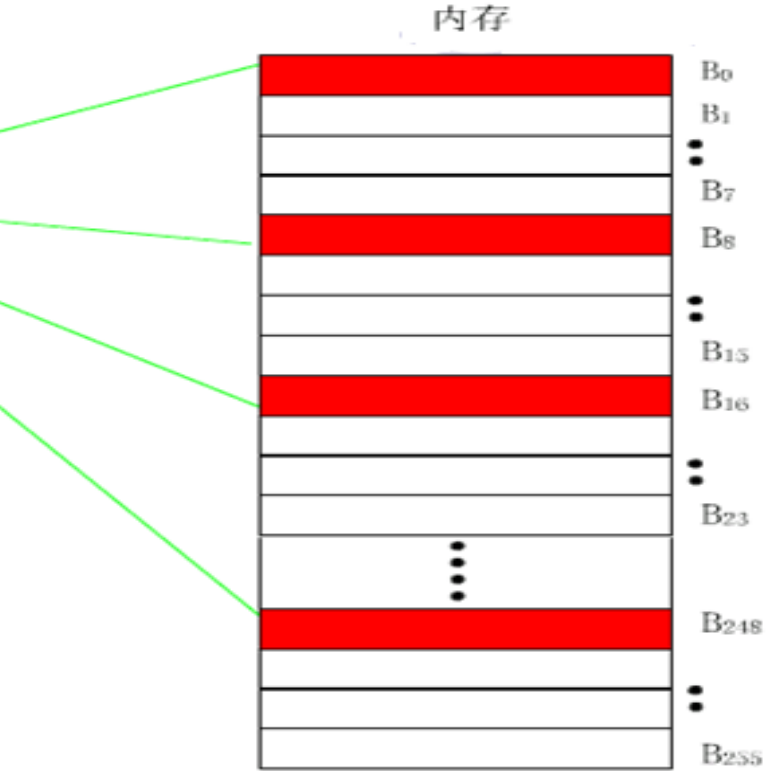


$i = j \bmod 2^c$,
这个主存块号 13, cache行数8
13—1101 8—1000
Cache行号=13% $2^3 = 5$
 1101 101
发现：主存块号的低C位即为对
应cache的行号

$i = j \bmod 2^3$, 这个模运算相当于留下最后三位二进制数，例如000恰好对应cache的第0行，同理001；这样主存块号作为标记就可以减少这三位（主存块号末尾三位直接反映在cache的位置），用橙色位表示标记即可,即加快标记检索的速度

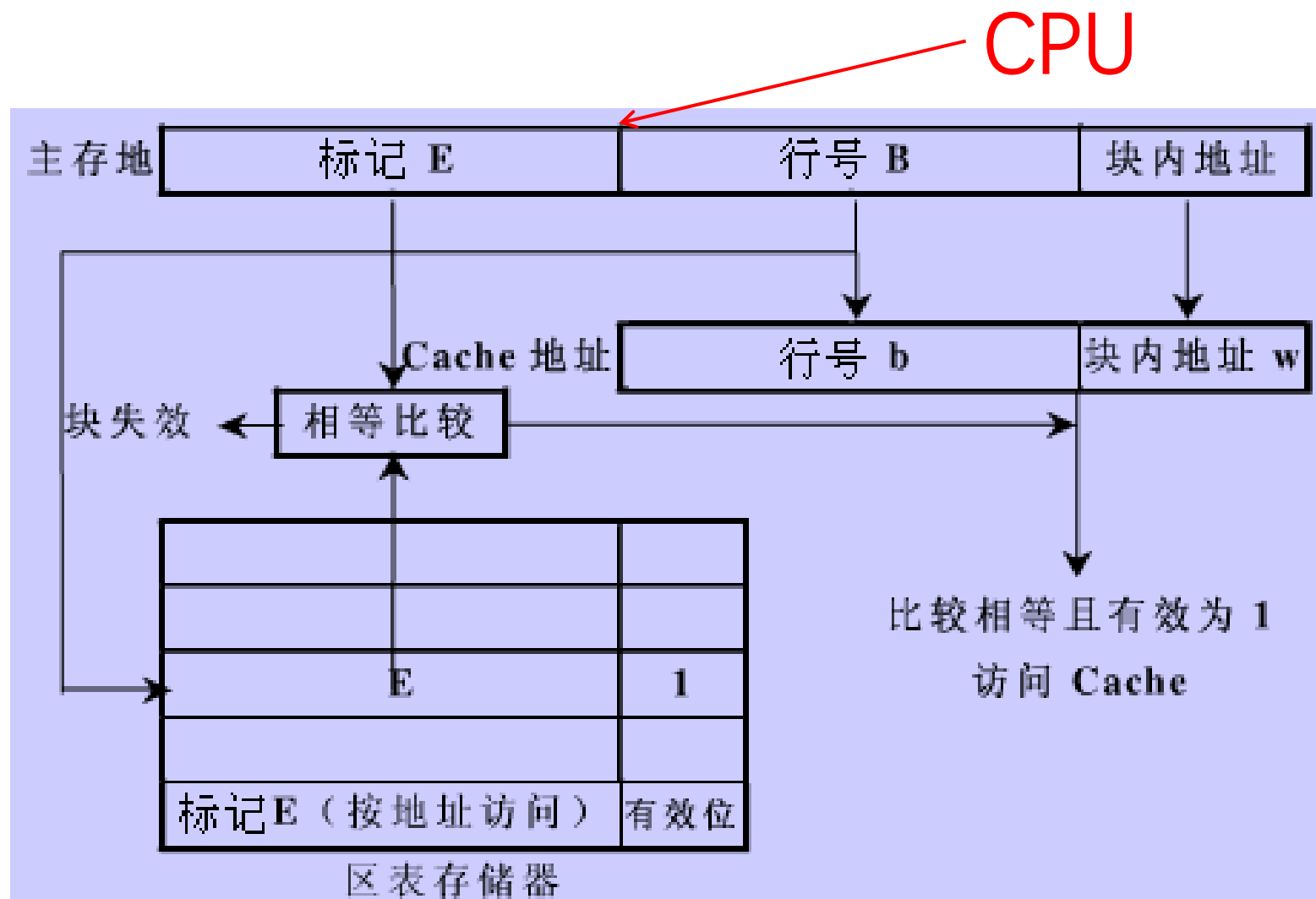


如图, B₀, B₈, . . . , B_{8k}主存块只可映射到Cache的第0块.



Cache	第0块	0	0	0	0	0
第0块	第8块	0	1	0	0	0
	第16块	1	0	0	0	0
	第1块	0	0	0	0	1
第1块	第9块	0	1	0	0	1
	第17块	1	0	0	0	1

>>> 直接映射地址变换示意图

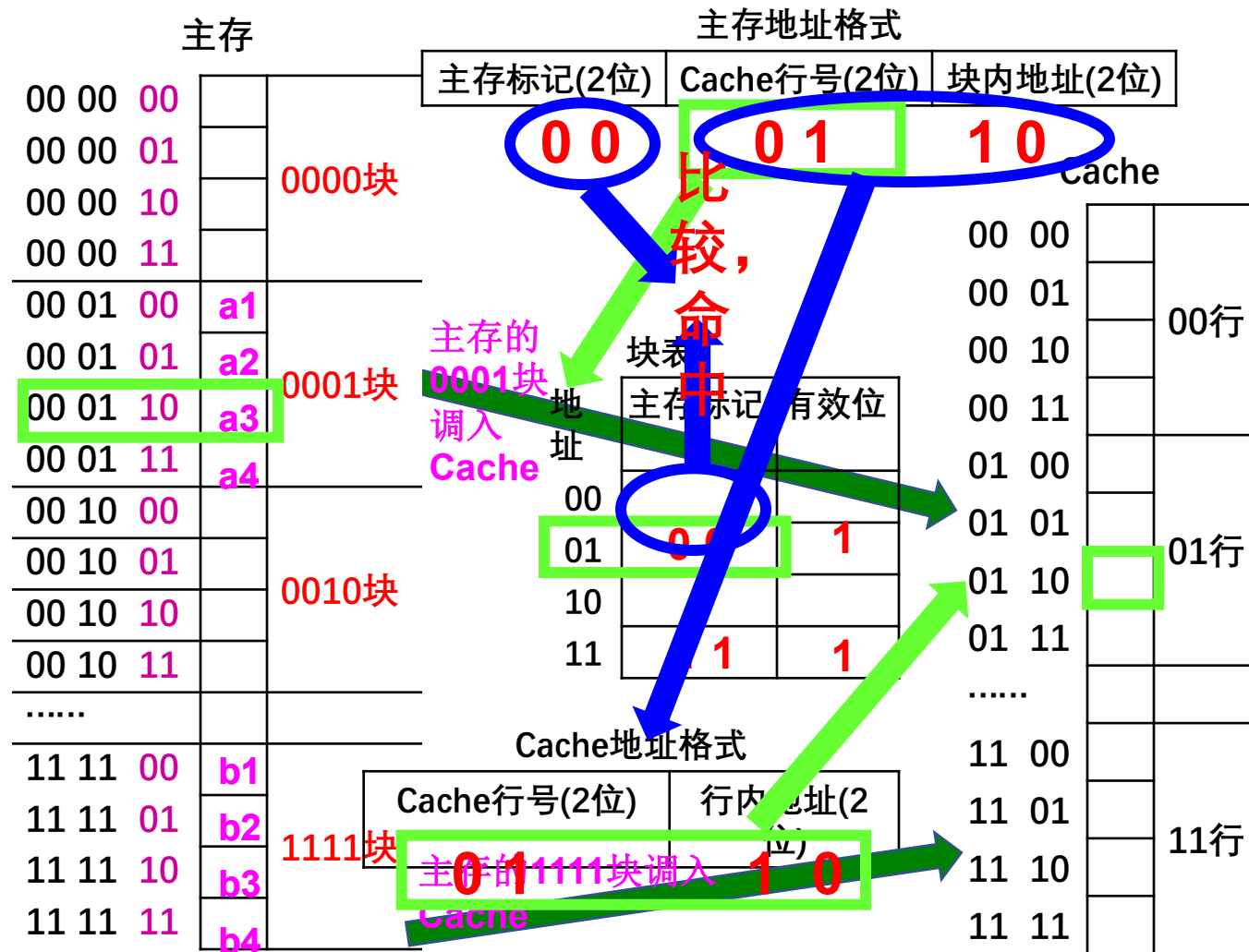


从主存地址中取出行号，查块表，找出对应的标记与比较器中另一端主存中取出的标记进行比较



例子包含两个过程：映射与地址变换

直接映射方式举例



>>> 直接映射方式下的主存地址格式

➤ 假定主存共 2^m 块，Cache共 2^c 行，每块/行单元数为 2^b 个

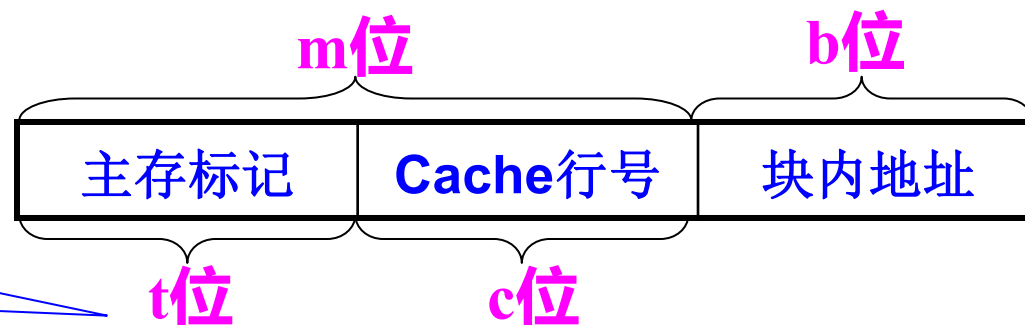
□ 主存地址为 $m+b$ 位；Cache地址为 $c+b$ 位；

➤ 直接映射中主存块与Cache行的关系：

□ Cache地址格式为：



□ 主存的地址格式为：



主存中有 2^t 块对应于同一Cache行

➤ 块表的基本结构 不需要使用相联存储器

□ 单元数目与Cache的行数一致；

□ 每个单元保存：主存块号；

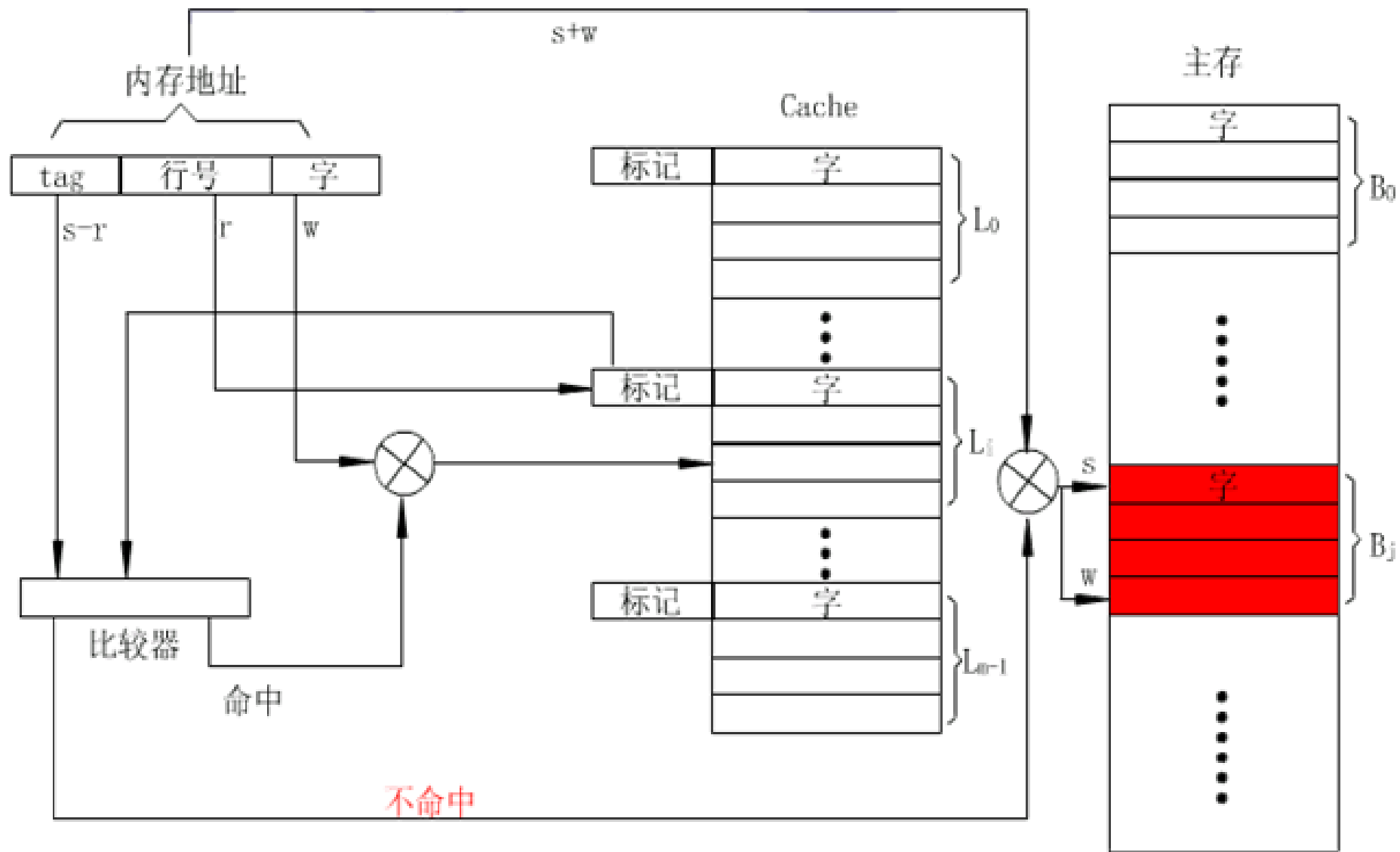
块表的大小应为

$2^c \times (m-c+1)$ 位



直接映射Cache的组织

动画演示: [直接映射.swf](#)



>>> 直接映射Cache的特点

➤ 特点

- 一个主存块只能调入Cache的一个特定行中。

➤ 优点

- 该映射函数实现简单，查找速度快；

- ✓ 主存地址的中间 c 位即为Cache的行地址；

- ✓ 在块表对应单元中，使用高 t 位地址（主存标记）进行比较，决定是否命中；

➤ 缺点

- 灵活性差；

- ✓ 主存的 2^t 个字块只能对应唯一的Cache字块，即使Cache中别的字块空着也不能占用。

>>> 直接映射举例 (1/3)

设主存共512个单元（字节），Cache共32个单元，块大小为8个字节，试用直接映射方式组织Cache。

➤ Cache共32个单元，每行（块）8字节

□ Cache共 $32/8=4$ 行

□ Cache地址需5位：Cache行号 $c=2$ ，行内地址 $b=3$

2位Cache行号	3位行内地址
-----------	--------

➤ 主存512个单元，每块8字节；

□ 主存共 $512/8=64$ 块

□ 主存地址需9位($2^9=512$)，地址包括：主存块号 $m=6$ ，块内地址 $b=3$

4位主存标记	2位Cache行号	3位块内地址
--------	-----------	--------

➤➤➤【例1】设主存共512个单元（字节），Cache共32个单元，块大小为8个字节，采用直接映射方式，试写出主存和Cache地址格式并举例说明。

➤ 若CPU发出的主存地址为0000 01 001；

- 若Cache块表中01行的有效位为1，则取主存的高4位地址（主存标记0000）送往比较器的一端；
- 再用中间的2位地址（Cache行号01），在块表中取出该单元中保存的主存标记，送往比较器的另一端；
- 若二者相等，则命中，直接访问Cache第01行中地址为001的单元，读取数据；
- 若二者不相等，则为未命中；
 - ✓ 直接使用0000 01 001地址访问主存单元；
 - ✓ 同时，将主存地址0000 01 000~0000 01 111的8个字节内容送到Cache的01行，即01000~01111单元；

>>> 【例2】设主存容量1MB，Cache容量16KB，块的大小为512B，采用**直接映射方式**。



- ① 块表的容量多大?
- ② 主存地址为CDE8FH的单元在Cache中的什么位置?

➤➤➤【例2】设主存容量1MB，Cache容量16KB，块的大小为512B，采用直接映射方式。

①块表的容量多大？

□ 块表的大小为 $2^5 \times (\text{主存标记位} + \text{有效位}) \times 6$ 位；



②主存地址为CDE8FH的单元在Cache中的什么位置？

□ 主存地址CDE8FH = 1100 1101 1110 1000 1111

□ 对应于Cache的第01111行，行内地址为010001111



>>> 3、组相联映射(Set-associative Mapping)

➤组相联映射是直接映射和全相联映射的一种折中方案。

➤映射关系

□将Cache中的行等分为若干组，主存中的每一块只能映射到Cache的特定组中，但是可调入到该组的任一行中；

□组间为直接映射，组内为全相联映射。

➤设Cache共u组，每组r行，则映射函数如下

组号 $q = j \bmod u$ j ——主存块号

Cache地址格式

组号	行号	行内地址
----	----	------

➤当Cache的一组包含r行时，通常称为r路组相联映射。

□若Cache为4路组相联映射，则表示每组中有4个Cache行；

➤主存地址

主存标记	Cache组号	块内地址
------	---------	------

>>> 组相联映射中的地址映射和变换

➤ 地址映射

- 主存中数据块调入Cache时，只能调入Cache的特定组；
- 在该组内，可选择任一行调入数据；
- 调入的同时，将主存标记和Cache的组内行号写入块表；

➤ 地址变换（类比直接映射）

- CPU访存时，发出主存地址；
- 从主存地址中截取出Cache组号，找到块表的对应组；
- 在组内，使用主存标记进行检索；
- 命中时，使用Cache组号、Cache组内行号和块内地址访问Cache；

块表

主存块号	Cache组号	Cache组内行号	有效位
0000	00	0	0/1
...	...	...	...
...	...	...	...
1111	11	1	0/1

主存标记

Cache组号

块内地址

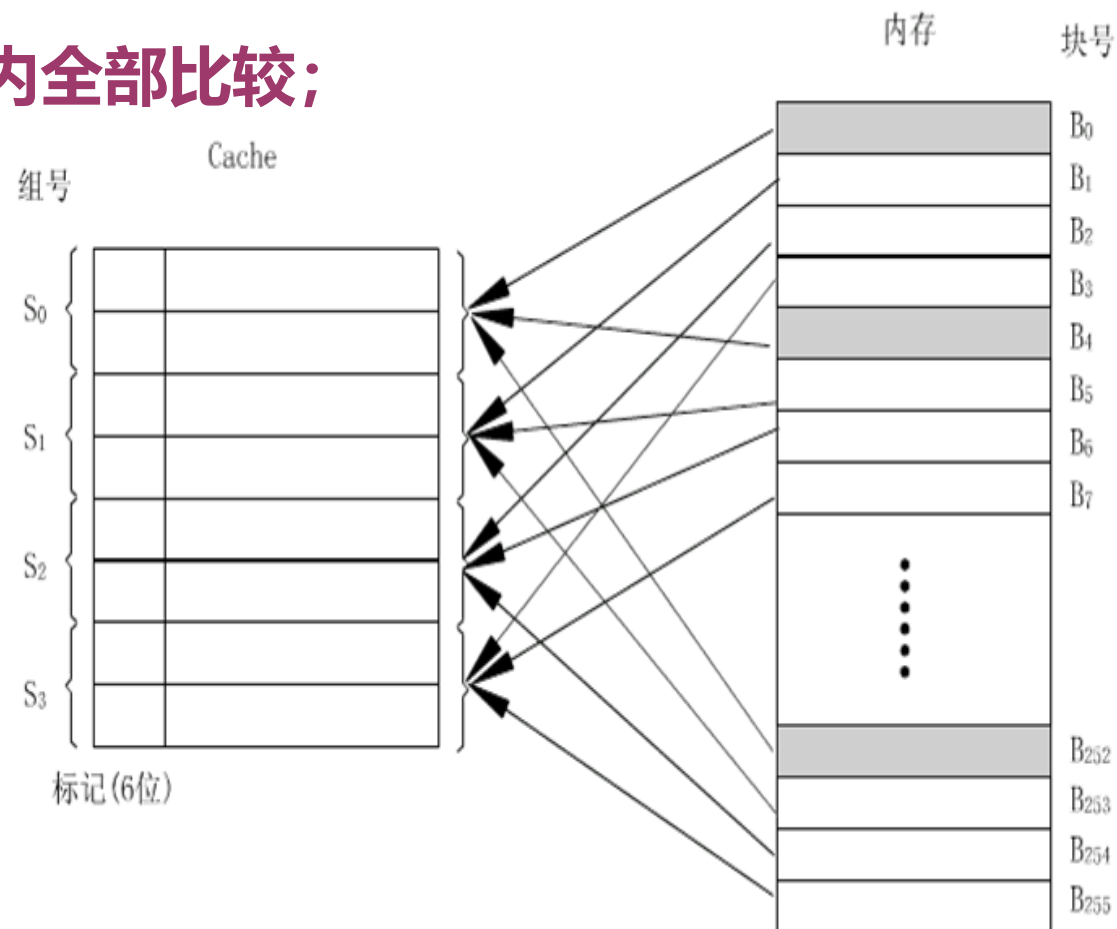
>>> 组相联映射的特点

➤ 组相联映射特点:

□ 灵活性: 比直接映射灵活 (主存可映射到组内任一块);

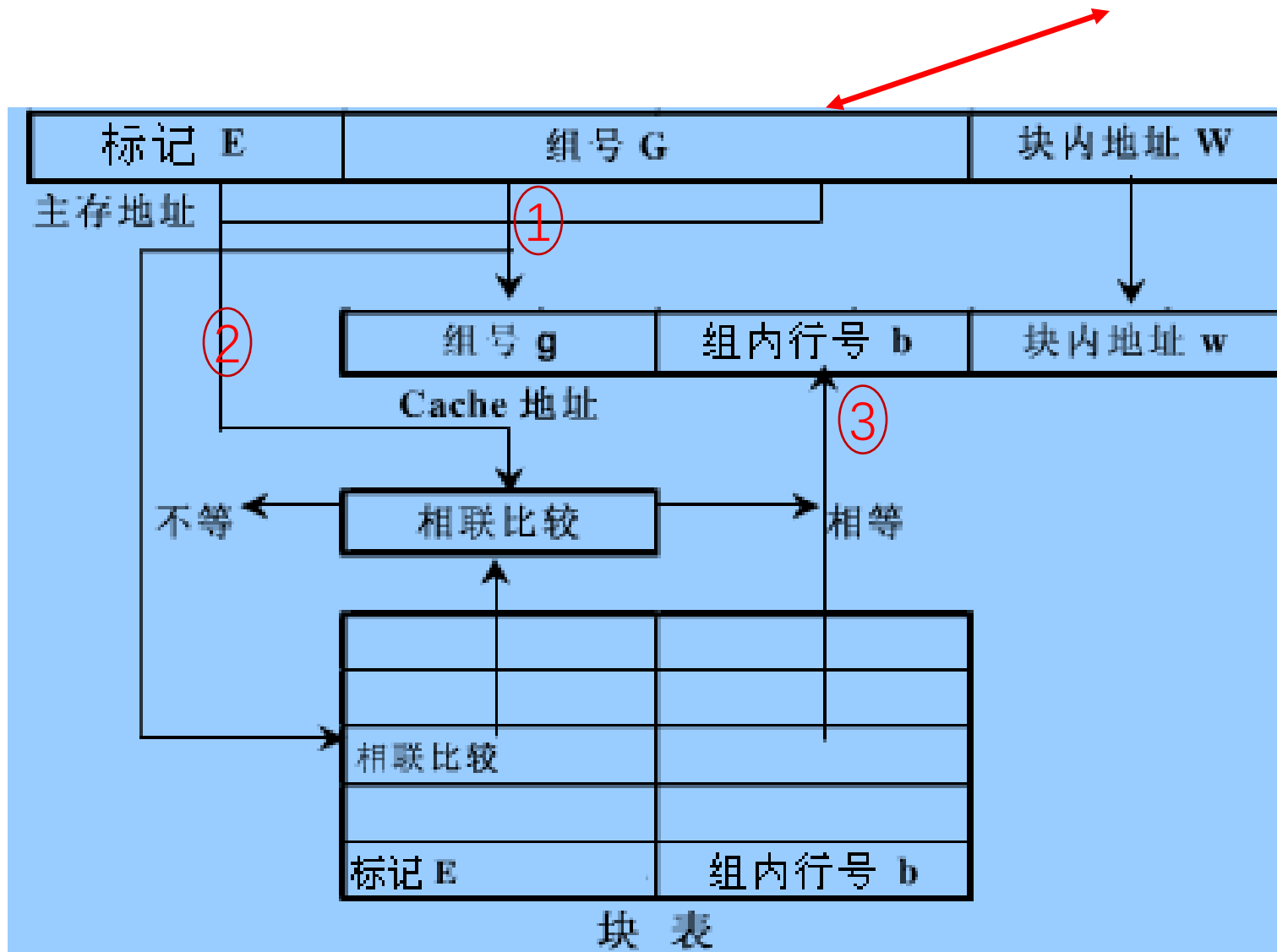
□ 快速性: 比全相联比较次数少, 只需组内全部比较;

✓ 由于比较次数少, 电路也较易于实现。



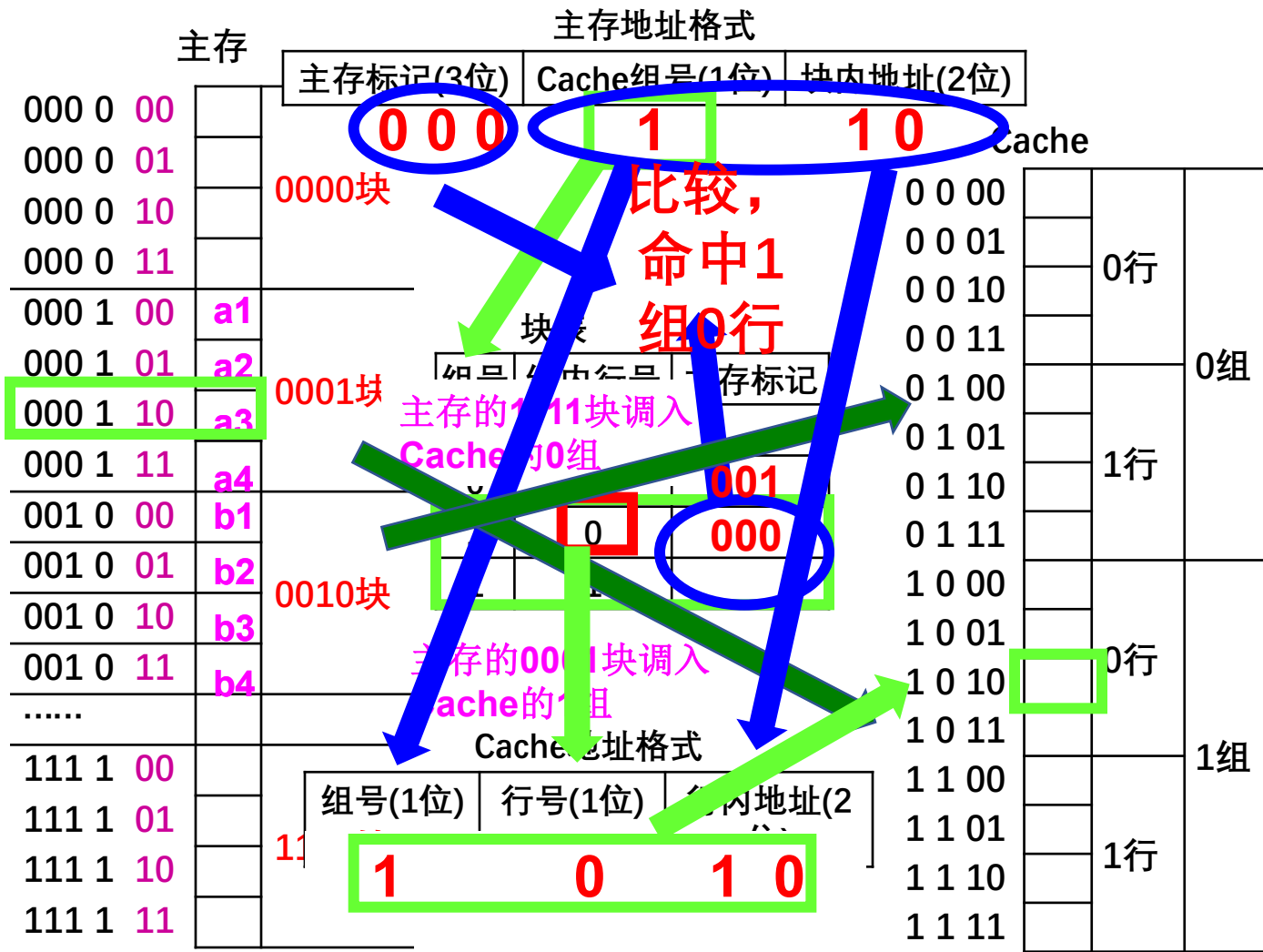
>>> 组相联映射地址变换示意图

CPU





组相联映射方式举例



例： 某系统的存储器为2MB，每字块为8个字，每字32位，
若Cache为16KB，采用字节编址方式。

问： (1) 采用直接映射，主存地址格式是什么？
(2) 采用全相联映射，主存地址格式是什么？
(3) 采用16路组相联映射，主存地址格式是什么？

➤ 2MB存储器共21位地址

□ 每个内存块共 $8 \times 4B = 32B$ ，共有 $2MB / 32B = 2^{16}$ 块

□ 16位块地址，3位块内字地址，2位字节地址

➤ 16KB的Cache共14位地址

□ 每个字块共 $8 \times 4B = 32B$ ，共有 $16KB / 32B = 512$ 行

□ 9位行地址，3位行内字地址，2位字节地址

➤ 采用直接映射，主存地址格式：

主存标记	Cache行号	块内地址
------	---------	------

7位主存标记	9位行地址	3位块内地址	2位字节地址
--------	-------	--------	--------



例： 某系统的存储器为2MB，每字块为8个字，每字32位，
若Cache为16KB，采用字节编址方式。

问： (1) 采用直接映射，主存地址格式是什么？
(2) 采用全相联映射，主存地址格式是什么？
(3) 采用16路组相联映射，主存地址格式是什么？

➤ 采用全相联映射，主存地址格式为：

主存标记	块内地址
------	------

16位主存标记	3位块内地址	2位字节地址
---------	--------	--------

➤ 采用16路组相联映射，主存地址格式为：

11位主存标记	5位组地址	3位块内地址	2位字节地址
---------	-------	--------	--------

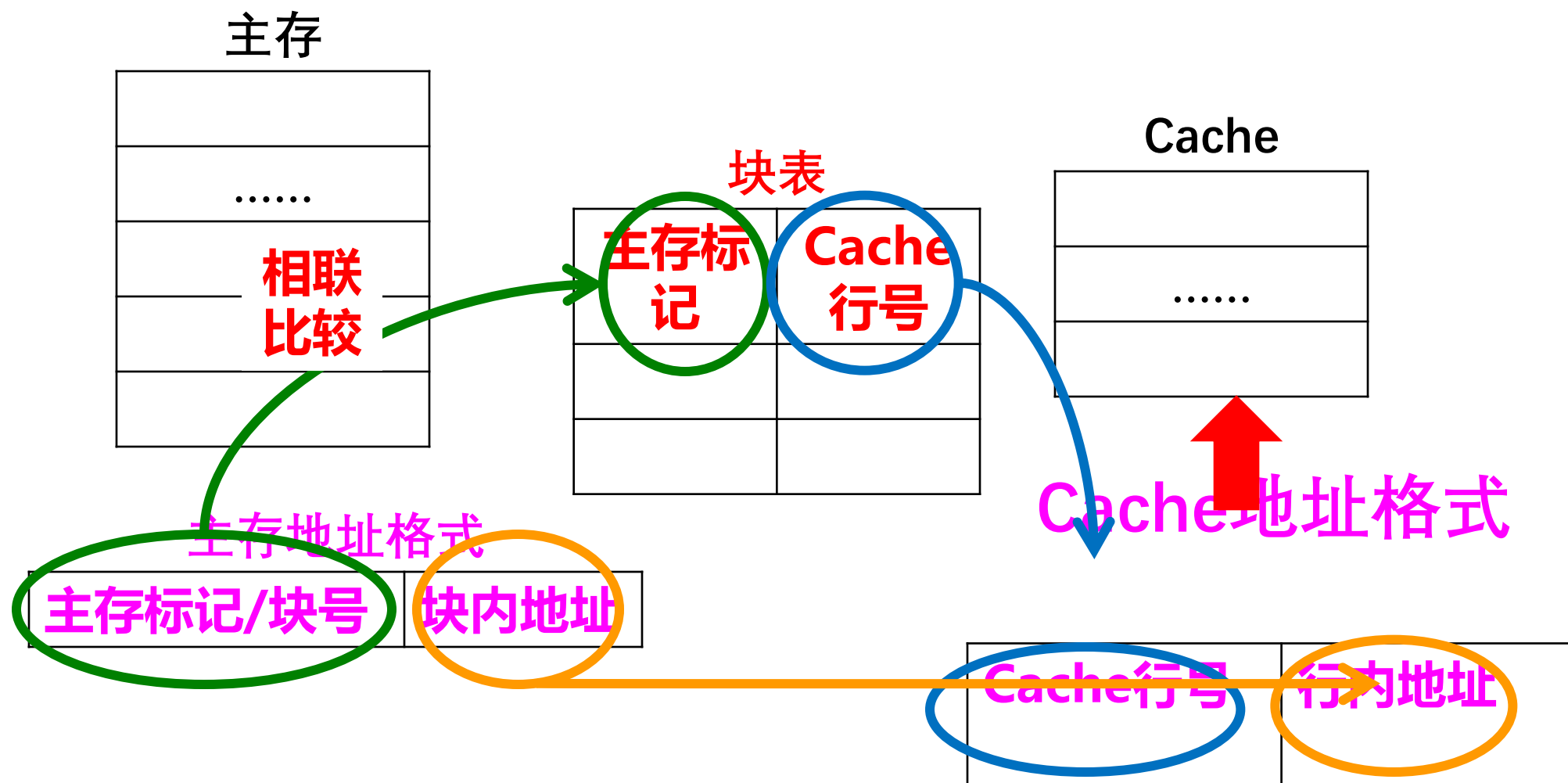
主存标记	Cache组号	块内地址
------	---------	------

1.每个字块的单元数：每个字占4B，8个字共32B，即每行为32B

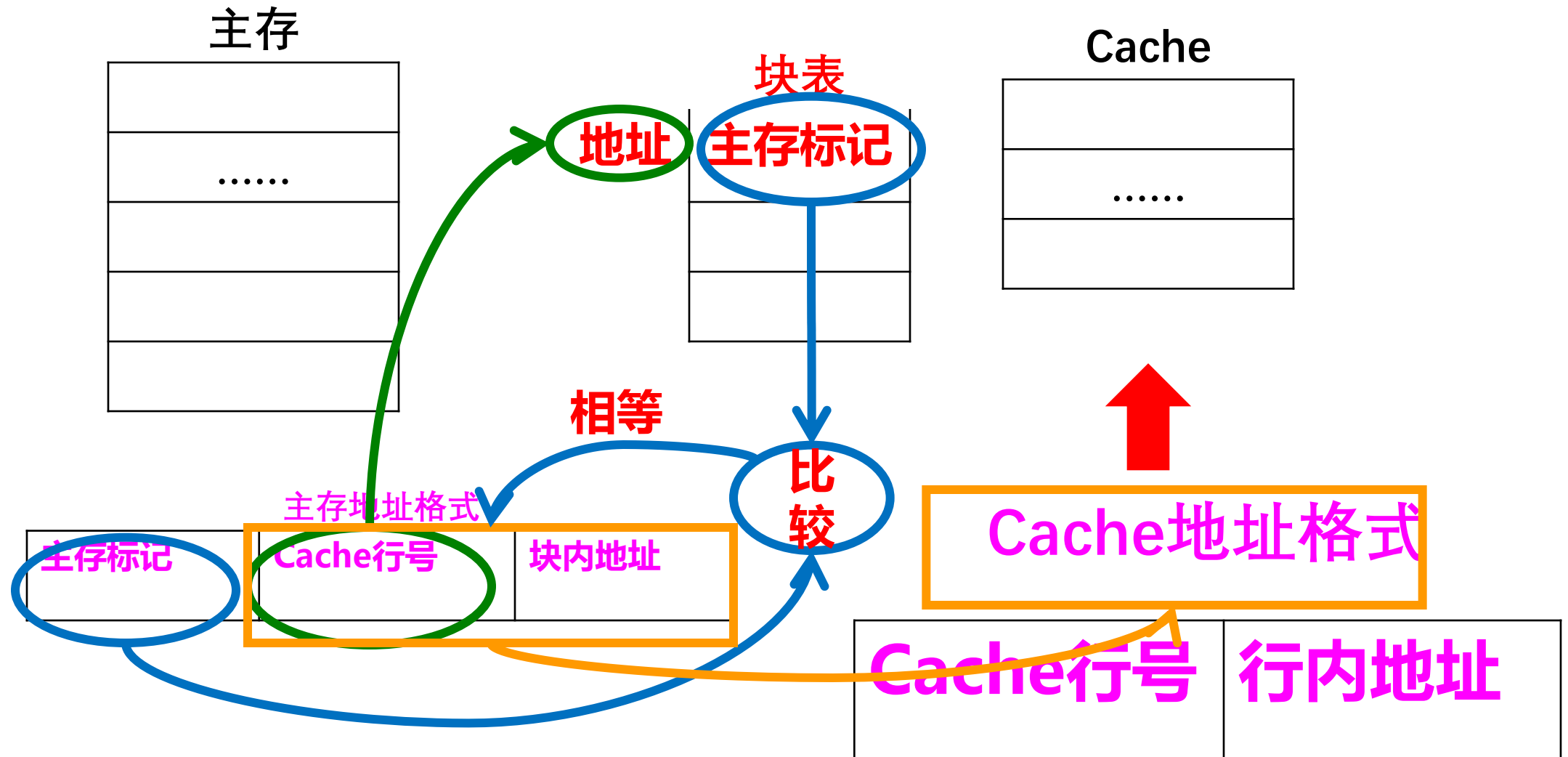
2.Cache中共有多少行： $(16KB)/32B=2^9$ 行

3.Cache中共有多少组：16路即一组16行，共 $2^9/2^4=2^5$ 组，所以组的位数为5位

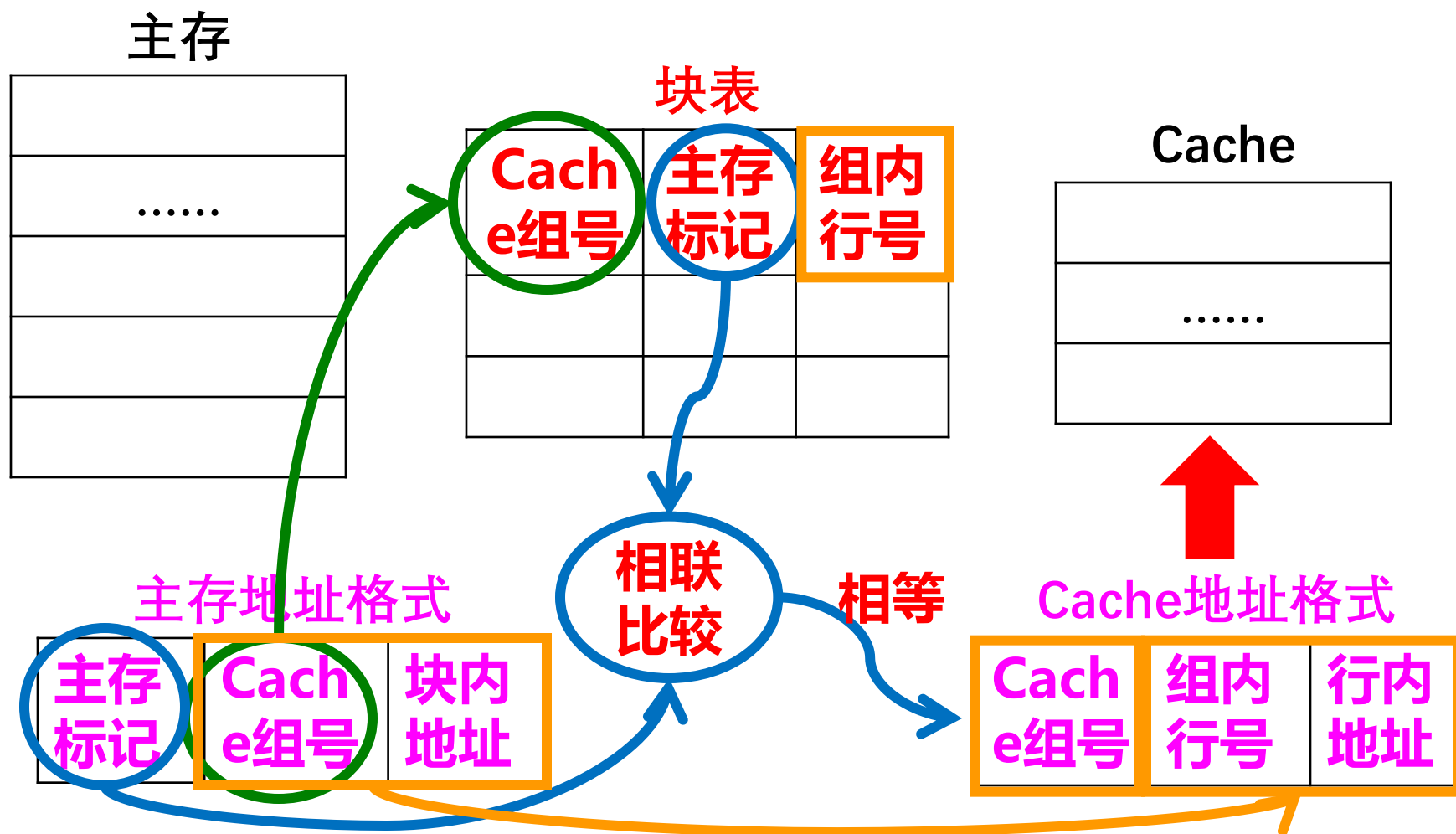
>>> 三种地址映射方式的对比——全相联映射方式



>>> 三种地址映射方式的对比 —— 直接映射方式



>>> 三种地址映射方式的对比 —— 组相联映射方式



>>> 2009年考研真题

14. 某计算机的Cache共有16块，采用2路组相连映射方式，每个主存块大小为32字节，按字节编址。主存号129号单元所在主存块应装入到cache的组号是（ C ）

- A. 0 B. 2 C. 4 D. 6

Cache共16块，2路组相联，共8组，组号3位

主存地址格式

主存标记	Cache组号	块内地址
若干位	3位	5位

21. 假设某计算机的存储系统由Cache和主存组成。某程序执行过程中访存1000次，其中访问cache缺失（未命中）50次，则Cache的命中率是（ D ）

- A. 5% B. 9.5% C. 50% D. 95%

某计算机的Cache共有16块，采用2路组相连映射方式，每个主存块大小为32字节，按字节编址。主存号129号单元所在主存块应装入到cache的组号是（ ）

A

0

B

2

C

4

D

6

解答：主存块32字节，即块内地址5位
2路组相联，即 $r=2$ ，
组数=总块数/每组行数
=16/2
=8 组 即组的位数3位
129: 1000 0001

提交

有如下C语言程序段：

```
for (k=0; k<1000; k++)
```

```
    a[k]=a[k]+32;
```

若数组a及变量k均为int型，int型占4B，数据Cache采用直接映射方式，数据区大小为1KB，块大小为16B，该程序段执行过程中访问数组a的Cache缺失率约为（ ）

- ☐ A 1.25%
 ☐ B 2.5 %
 ☒ C 12.5%
 ☐ D 25 %

提交

有如下C语言程序段：

```
for (k=0; k<1000; k++)
```

```
    a[k]=a[k]+32;
```

若数组a及变量k均为int型，int型占4B，数据Cache采用直接映射方式，数据区大小为1KB，块大小为16B，该程序段执行前**Cache**为空，该程序段执行过程中访问数组a的Cache缺失率约为（ ）

1
2
3
4

解答：一个数据占4B，一个块大小是16B，所以一个块有4个数据，

开始cache为空，证明第一次cache缺失，分析语句“**a[k]=a[k]+32**”：首先读取**a[k]**需要访问一次**a[k]**，之后将结果赋值给**a[k]**需要访问一次，共访问两次。第一次访问**a[k]**未命中，并将该字所在的主存块调入**Cache**对应的块中，对于该主存块中的**4个**整数的两次访问中只在访问第一次的第一个元素时发生缺失，其他的7次访问中全部命中，故该程序段执行过程中访问数组a的**Cache**缺失率约为1/8（即12.5%）。

>>> Cache的替换

➤ Cache本身容量小，满了（没有空行或者有空行但不能随意更换）如何处理

➤ 替换

□ 当出现cache未命中，而cache对应行已满，便淘汰该行中的某一组以腾出位置存放新调入的组，称为替换。

➤ 替换算法/策略

□ 确定替换的规则叫替换算法，常用的替换算法有：近期最少使用(Least Recently Used) LRU、最不经常使用((Least Frequently Used), LFU)算法和随机法 (RAND) 等。

➤ 替换算法的目标

□ 使Cache获得更高的命中率

>>> 3.6.3 Cache的替换策略

——近期最少使用(Least Recently Used , LRU)算法

➤ 替换原则

- 将**近期内**长久未被访问过的行替换出去。

➤ 使用方法

- 每行也设置一个计数器;

- 每访问一次，被访行的计数器清零，其它各行计数值均加1;

- 当需要替换时，将计数值最大的行换出。

➤ 特点

- 这种算法保护了刚调入到Cache中的新数据行，使Cache的使用率较高。

>>> 3.7 虚拟存储器

3.7.1 虚拟存储器的基本概念

3.7.2 页式虚拟存储器

3.7.3 段式虚拟存储器 和段页式虚拟存储器

3.7.4 虚存的替换算法

>>> 3.7.1 虚拟存储器的基本概念

➤ 虚存的使用原理

- 结合存储系统的多层次结构，物理内存容量有限，难以满足实际使用的需要（研究主存与辅存层次关系）；
- 利用辅存空间，将数据在辅存与主存之间调度；
- 用户实际访问的仍然是主存，利用辅存的部分空间保存暂时不用的主存内的数据；

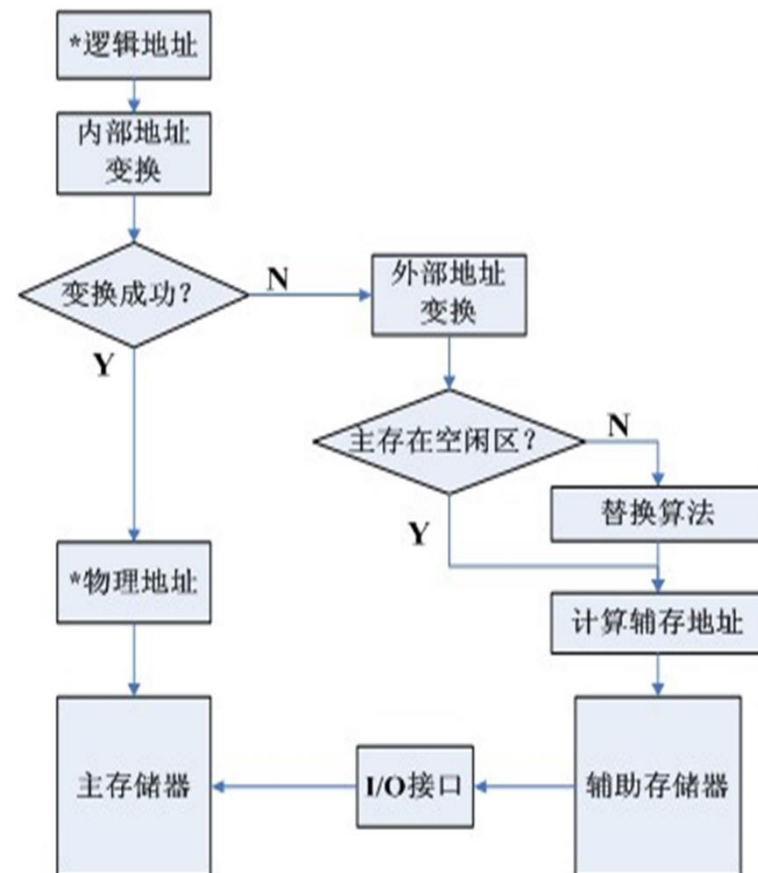
➤ 实地址与虚地址

- 实地址（物理地址）：主存空间的地址，对应于物理内存；
- 虚地址（逻辑地址）：用户编程时可用的地址（例如 `mov AX,x`），对应的存储空间称为虚拟空间；
- 再定位：虚地址到实地址的转换过程；

➤ 虚拟存储器按组织形式分为：页式虚拟存储器、段式虚拟存储器、段页式虚拟存储器

>>>1、虚存的访问过程

- 用户程序按照虚地址存放在辅存中；
- 程序运行时，将程序或程序的一部分从辅存调入实存；
 - 同时更新页表/段表；
- CPU执行程序访存时，发出**虚地址**；
 - **查询页表/段表**，判断是否命中实存；
 - 若命中，则将虚地址转换成实地址，**访问实存**；
 - 若未命中，则执行调度算法，将辅存中的内容调入实存，再**访问实存**；
- 虚存对于应用程序员来说，是透明的。
- 程序虚地址空间与实地址空间没有要求（可大可小）



>>> 2、虚存的访问过程

- 根据虚存机制，应用程序员透明地使用虚存空间。
- 若主存命中率很高，虚存的访问速度接近于主存访问速度，但虚存的大小仅仅依赖于辅存大小。
- 因此每个程序都可以拥有一个虚拟的存储器，它是由主存、辅存以及辅存管理部件构成的**概念模型**，不是实际物理存储器。
- 虚存是主存和辅存以及硬件和软件实现的，所以，基于软件的引入
 - 对设计存储给管理软件的系统程序员是不透明的
 - 对应用程序员仍然是透明的

>>> 3、Cache—主存与主存—辅存

➤相同点

- **目的相同**：构成一个具有高速度、大容量、低成本的存储系统；
- **原理相同**：慢速存储器与快速存储器之间的数据调度；地址映射/变换，数据替换；

➤不同点

- **侧重点不同**：cache侧重于速度，虚存侧重于容量；
- **传送信息单位不同**：cache-主存以块为单位，虚存以页、段、段页式虚拟存储器组织数据
- **数据通路不同**：
 - cache未命中，可以直接访问主存；
 - 主存未命中，只能通过调页解决，需要先调入主存，再访问；
- **透明性不同**：cache由硬件系统管理，对系统程序员和应用程序员完全透明
虚存管理有软硬件完成，虚存对于存储管理系统程序员不透明，对应用程序员透明
- **未命中的损失不同**：主存未命中损失大于Cache未命中（速度差异）；

>>> 4、虚存机制要解决的关键问题

➤ 调度问题

- 决定哪些程序和数据应被调入主存；

➤ 地址映射问题

- 在访问主存时把虚地址变为主存物理地址（这一过程称为内地址变换）；在访问辅存时把虚地址变成辅存的物理地址（这一过程称为外地址变换），以便换页。此外还要解决主存分配、存储保护与程序再定位等问题。

➤ 替换问题

- 决定哪些程序和数据应被调出主存。

➤ 更新问题

- 确保主存与辅存的一致性。

>>> 3.7.2 页式虚拟存储器

➤ 如cache和主存之间的传递单位为块一样，虚拟空间与主存之间的传递单位可以为页，也可以为段

➤ 虚存空间与主存空间按照同样的容量（页）划分；

□ 虚存空间的页被称为**逻辑页**；主存空间的页被称为**物理页（实页、页框）**；

□ 虚地址格式：

逻辑页号	页内地址
------	------

□ 实地址格式：

物理页号	页内地址
------	------

➤ **页表：实现逻辑页到物理页的变换，大多驻留在内存中；**

□ 每个**进程**对应一个页表；每个进程被分为多个页

□ 每个逻辑页有一个表项；

□ 逻辑页调入物理页时，写该表项；

□ 地址变换时，根据虚页号查该表；

□ 命中时，将虚地址中的虚页号替换为主存页号，然后访问主存；（课表P106 图3.34）

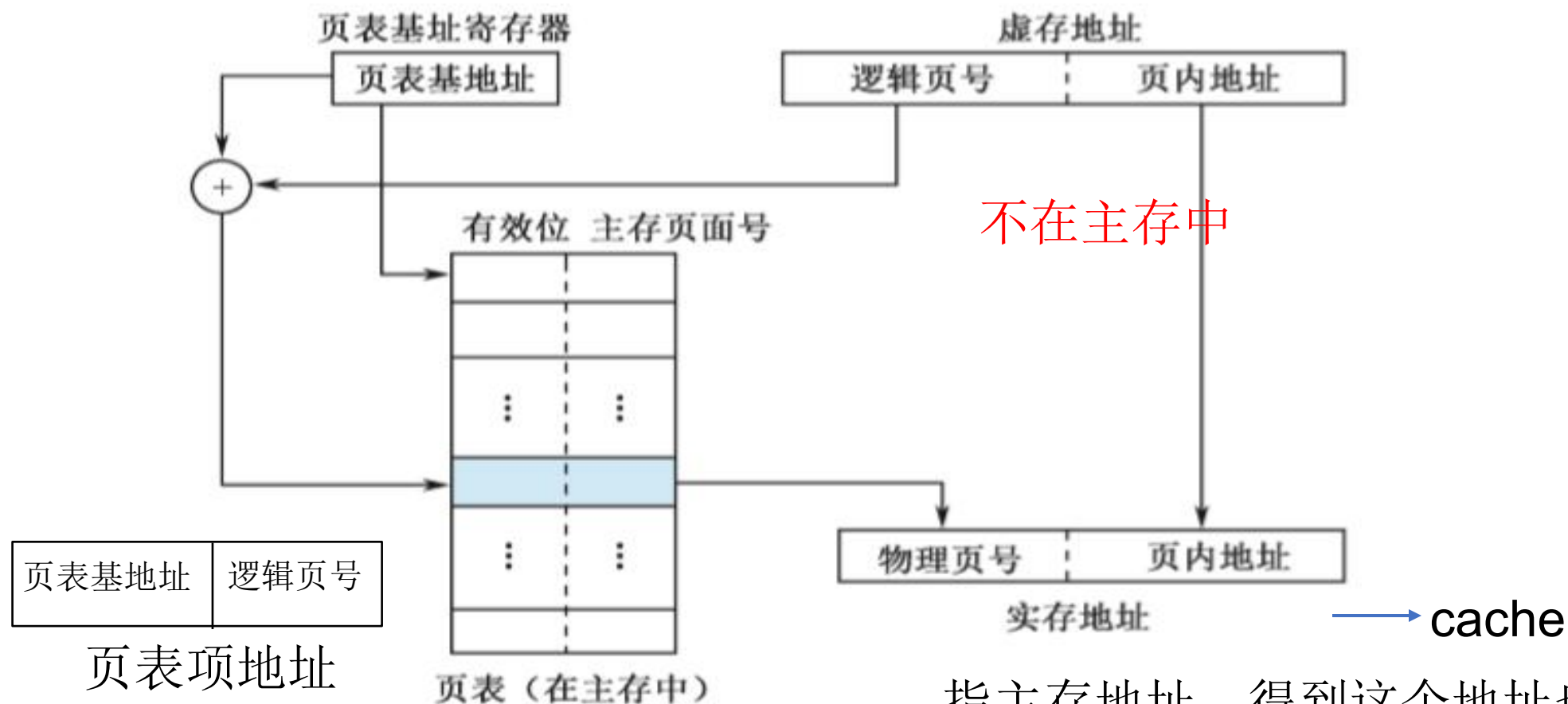
□ 有效位为1表示当前虚拟地址指向的数据已经调入内存，否则在辅存

页 表

虚页号	主存页号	有效位	修改位	引用位
0...00		0		
0...01		1		
		...		
1...11		...		

有效位：访问时，指示该页数据是否可用；
修改位：写回时，指示该页数据是否被修改；
引用位：替换时，指示该页被访问的次数；

>>> 页式虚拟存储器的地址映射过程



页表项地址指主存地址

指主存地址，得到这个地址也不一定说明数据就在cache中

页表寄存器存放的是页表的起始地址，根据其内容然后与逻辑页号进行拼接，查到页表中对应的实页号地址，再结合其页内地址就找到了主存地址，然后就可以去访问cache了

>>> 关于页表的存放

- 每个进程所需的页数并不固定，所以页表的长度是可变的，因此通常的实现方法是把页表的基地址保存在寄存器中，而页表本身则放在主存中。
- 页表保存在主存中，页表起始地址保存在寄存器中；
 - 页表大小与该进程的虚拟空间大小有关；
 - 若某进程虚地址空间2GB，页大小512B，则页表所需的存储单元数目为 $2^{31}/2^9=2^{22}$ ；（多少页）
- 活跃部分页表保存在主存中，其他部分保存在辅存中
 - 页表进行分页管理
 - 为了节省页表所占的主存空间

>>> 关于TLB（转换后援缓冲器，快表）

- 页表保存在主存中，页表起始地址保存在寄存器中；
- CPU访问一次数据需要两次访存，一次访问页表，一次根据页表的访问最终转化为物理地址访问该单元的数据
 - 降低了计算机的处理速度
 - 用降低处理速度换取存储器空间利用率的提高得不偿失
- TLB是一个内存管理单元用于改进虚拟地址到物理地址转换速度的缓存。（快表是慢表的副本，注意和cache的使用结合理解）
 - 把页表中活跃的部分存放在高速存储器中。这个专用于页表缓存的高速存储部件通常称为转换后援缓冲器(TLB)，又称为快表。而保存在主存中的完整页表则称为慢表。
 - TLB是位于内存中的页表的cache，如果没有TLB，则每次取数据都需要两次访问内存,即查页表获得物理地址和取数据.

>>> TLB (转换后援缓冲器, 快表)

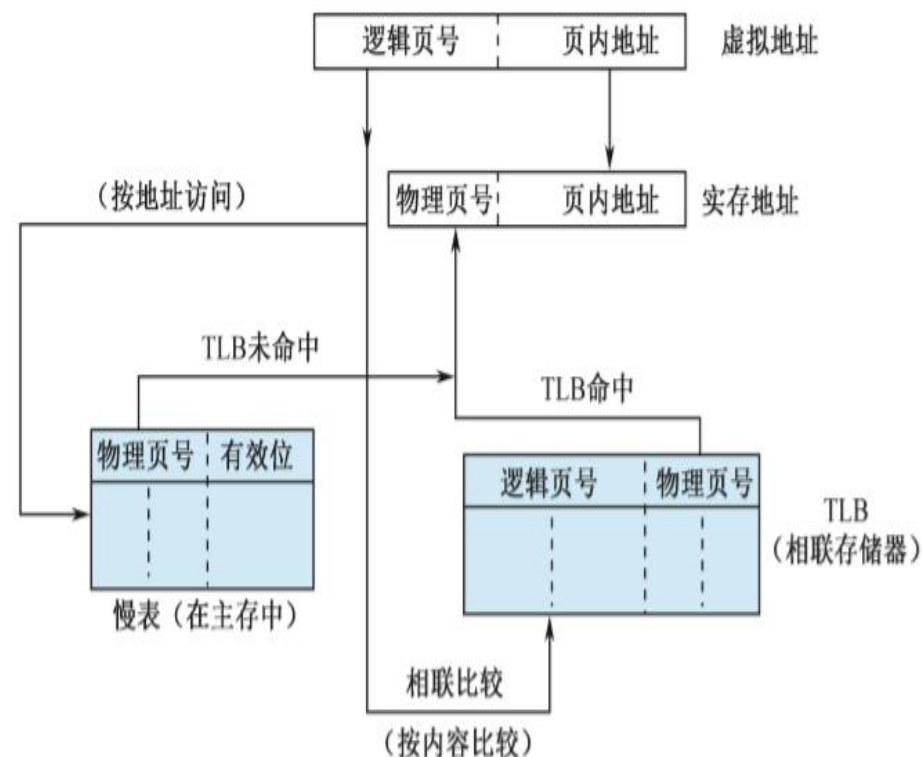
➤ TLB: 页表的Cache, 常使用相联存储器实现;

- 其作用同cache (速度、数据副本等)
- 将页表中最活跃的部分放在Cache中, 从而加快物理地址与虚地址的转换过程;
- 保存在主存中的页表, 称为慢表;

➤ 地址变换 (课表P107 图3.35)

- 由逻辑页号同时检索快表和慢表, 快表命中时, 可快速得到物理地址;
- 未命中, 则由慢表给出物理地址;
- 查询慢表若仍然命中, 从辅存调入内存
- TLB通常采用**全相联映射**或**组相联映射**方式;

TLB的地址映射过程



快表的作用就是加快地址变换



例3.9 有页式虚拟存储器、TLB和cache组成的存储器层次结构中，访问存储器可能会遇到三种不同类型的缺失： TLB缺失、cache缺失、页缺失。研究这三种缺失会发生一个或多个时所有可能的组合（7种可能给性）。对每种可能性，说明这种情况是否真的会发生，在什么条件下会发生？

解、表3.9说明了可能的发生背景以及事实上它们是否真的可能发生，这些组合有三种是可能发生的，四种是可能的，注意页表指的是慢表。



课表P109 【例3.9】

虚拟存储器、TLB和Cache发生事件的可能组合

TLB	页表	Cache	发生可能性
命中	命中	缺失	可能。数据在主存中，未调入Cache，不需要检查页表
缺失	命中	命中	可能。数据在Cache中，页表信息在慢表中
缺失	命中	缺失	可能。数据在主存中，页表信息在慢表中
缺失	缺失	缺失	可能。数据在辅存中，页表信息不存在
命中	缺失	缺失	不可能。TLB是页表子集，Cache是主存子集
命中	缺失	命中	不可能。TLB是页表子集。
缺失	缺失	命中	不可能。数据不在主存中，一定不在Cache中

>>> 3.7.3 段式虚拟存储器和段页式虚拟存储器

➤ 段式虚拟存储器

□ 段：程序的自然分界；

✓ 例如8086中的代码段、数据段等；

□ 虚地址格式

□ 段表

段号	段内地址
----	------

段号 段地址 有效位 段长

0		0	
1		1	
		...	
n		...	

➤ 段页式虚拟存储器

□ 段式与页式虚拟存储器的结合；

□ 实存分页，程序先分段，段内再分页；

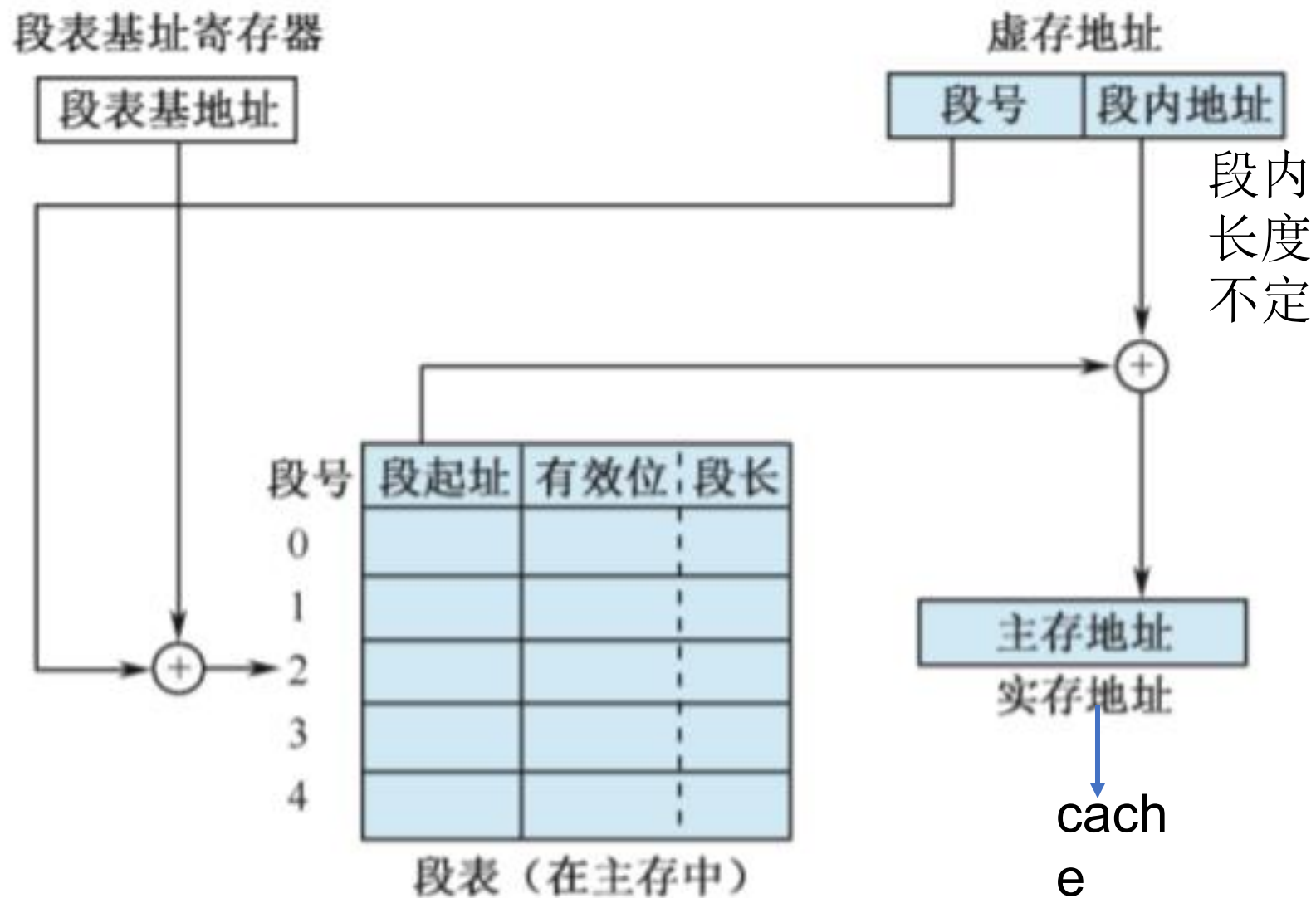
□ 按页进行数据调度，按段进行编程、保护和共享；

□ 虚地址格式

段号	段内页号	页内地址
----	------	------

✓ 多任务系统，虚地址前有“基号”，保存段表起始位置的地址；

>>> 段式虚拟存储器



17、下列命令组合中，一次访存过程中，不可能发生的是

()

数据在辅存中

A、TLB未命中，Cache未命中，Page未命中

数据在Cache中，慢表命中

B、TLB未命中，Cache命中，Page命中

数据在主存中，快表命中

C、TLB命中，Cache未命中，Page命中

Cache中数据必须是主存中数据的子集

D、TLB命中，Cache命中，Page未命中

16、某计算机主存地址空间大小为256MB，按字节编址。虚拟地址空间大小为4GB，采用页式存储管理，页面大小为4KB，TLB（快表）采用全相联映射，有4个页表项，内容如下表所示。则对虚拟地址03FF F180H进行虚实地址变换的结果是（ ）

有效位	标记	页框号	
0	FF180H	0002H	
1	3FFF1H	0035H	
0	02FF3H	0351H	
1	03FFFH	0153H	

A

015 3180H

B

003 5180H

C

TLB缺失

D

缺页



2013年考研真题

16、某计算机主存地址空间大小为256MB，按字节编址。虚拟地址空间大小为4GB，采用页式存储管理，页面大小为4KB，TLB（快表）采用全相联映射，有4个页表项，内容如下表所示。则对虚拟地址03FF F180H进行虚实地址变换的结果是 (A)

有效位	标记	页框号	
0	FF180H	0002H	
1	3FFF1H	0035H	
0	02FF3H	0351H	
1	03FFFH	0153H	

A、015 3180H B、003 5180H C、TLB缺失 D、缺页

解答：实地址：实页号+页内地址，虚地址：虚页号+页内地址

虚拟地址为03FFF180H，其中^{16位}页号为03FFFH，^{12位}页内地址为180H，根据题目中给出的页表项可知页标记为03FFFH所对应的页框号为0153H，页框号与页内地址之和即为物理地址0153180H

>>> 本章综合举例

1. CPU访问存储器的时间是由存储器的容量决定的，存储容量越大，访问存储器所需要的时间越长。

□ 错误。

□ CPU可直接访问的是随机存储器，随机存储器是按地址访问的，其访问时间和存储容量无关。

2. 半导体存储器加电后才能存储数据，断电后数据就丢失了，因此，EPROM做成的存储器，加电后必须重写原来的内容。

□ 错误。

□ EPROM（可擦除的可编程的只读存储器）是非易失性存储器，断电后数据是不会丢失的。

3. 大多数个人计算机中可配置的内存容量受地址总线位数限制。

□ 正确。地址总线的位数决定了最大的内存容量。